

Basi di dati

Introduzione

Docente: Stefano Paraboschi

parabosc@unibg.it

Corso di “Sistemi informativi” AA 2005/2006

- Orario:
 - Lunedì 10.45-12.15, aula 10
 - Martedì 8.45-10.15, aula 8
- Pagina Web del corso: raggiungibile dal sito dell'Università.
- URL diretto: <http://193.204.253.55/sistinf.html>

2

Corso di “Sistemi informativi” AA 2005/2006

- Titolare del corso: Prof. Stefano Paraboschi
 - ricevimento: Merc. 17.30-19.30
 - email: parabosc@unibg.it
 - tel: 035-205-2356
- Resp. esercitazioni: Dott. Mario Molinari
 - email: marimol@tin.it

3

Corso di “Sistemi informativi” AA 2005/2006

- Termine lezioni: 30/5/06
- E' possibile sostenere l'esame con delle prove in itinere
 - I prova: martedì 11/4/06
 - II prova: mercoledì 6/6/06
 - La partecipazione alle prove in itinere non pone vincoli rispetto alla partecipazione ai normali appelli
- La consegna del compito a un appello implica la rinuncia a un eventuale voto positivo ottenuto nelle prove in itinere

4

Corso di “Sistemi informativi” AA 2005/2006

- Calendario esercitazioni:
 - 14/3
 - 21/3
 - 3/4 (lab.)
 - 4/4
 - 19/5
 - 23/5 (ven.)
 - 29/5 (lab.)
 - 30/5

5

Corso di “Sistemi informativi” AA 2005/2006

- E' possibile preparare un elaborato (facoltativo)
- Al massimo 3 persone per elaborato
- Il voto assegnato all'elaborato viene sommato al voto ottenuto nei compiti o all'esame, con arrotondamento per eccesso

6

Corso di “Sistemi informativi” AA 2005/2006

- Libro di testo: Paolo Atzeni, Stefano Ceri, Stefano Paraboschi, Riccardo Torlone “Basi di dati: Modelli e linguaggi di interrogazione, II ed.”, McGraw-Hill Libri Italia, € 25.00 (?)
- Esiste anche la versione in inglese, degli stessi autori, titolo “Database systems”, McGraw-Hill

7

Corso di “Sistemi informativi” AA 2005/2006

- Sul sito del corso sono disponibili i lucidi usati durante le lezioni
- Eserciziari:
 - Elena Baralis, Alberto Belussi, Giuseppe Psaila, “Esercizi di basi di dati”, Esculapio
 - Chiara Francalanci, Fabio Schreiber, Letizia Tanca, “Esercizi di progettazione concettuale”, Esculapio

8

Corso di “Sistemi informativi” AA 2005/2006

- Altro materiale
 - soluzioni di tutti gli esercizi del libro sul sito della versione italiana del libro <http://www.ateneonline.it>
 - raccolta completa dei temi d'esame passati su <http://193.204.253.55/TemiSI>

9

Sistemi informativi e sistemi informatici una premessa

- Che cos'è l'informatica? Una definizione:
 - Scienza del trattamento razionale, specialmente per mezzo di macchine automatiche, dell'informazione, considerata come supporto alla conoscenza umana e alla comunicazione (Académie Française).
- L'informatica ha due anime:
 - tecnologica: i calcolatori elettronici e i sistemi che li utilizzano;
 - metodologica: i metodi per la soluzione di problemi e la gestione delle informazioni.

10

Sistema informativo

- Componente (sottosistema) di una organizzazione che gestisce (acquisisce, elabora, conserva, produce) le informazioni di interesse (cioè utilizzate per il perseguimento degli scopi dell'organizzazione stessa).
 - ogni organizzazione ha un sistema informativo, eventualmente non esplicitato nella struttura;
 - quasi sempre, il sistema informativo è di supporto ad altri sottosistemi, e va quindi studiato nel contesto in cui è inserito
 - il sistema informativo è di solito suddiviso in sottosistemi (in modo gerarchico o decentrato), più o meno fortemente integrati

11

Sistema organizzativo

- insieme di risorse e regole per lo svolgimento coordinato delle attività al fine del perseguimento degli scopi
- il sistema informativo è parte del sistema organizzativo
- il sistema informativo esegue/gestisce processi informativi (cioè i processi che coinvolgono informazioni)

12

Risorse

- le risorse di una azienda (o amministrazione):
 - persone
 - denaro
 - materiali
 - informazioni

13

Funzioni di un sistema informativo

- raccolta, acquisizione delle informazioni
- archiviazione, conservazione delle informazioni
- elaborazione delle informazioni
- distribuzione, scambio di informazioni

14

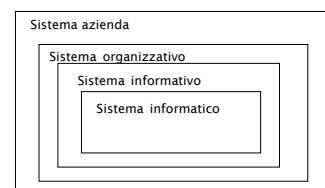
Sistemi informativi e automazione

- Il concetto di "sistema informativo" è indipendente da qualsiasi automatizzazione: esistono organizzazioni la cui ragion d'essere è la gestione di informazioni (p. es. servizi anagrafici e banche) e che operano da secoli.

15

Sistema Informatico

- porzione automatizzata del sistema informativo
- il sistema informatico è la parte del sistema informativo che gestisce informazioni per mezzo della tecnologia informatica.



16

Sistema informativo e sistema informatico

- Anche prima di essere automatizzati, molti sistemi informativi si sono evoluti verso una razionalizzazione e standardizzazione delle procedure e dell'organizzazione delle informazioni.

17

Gestione delle informazioni

- Nelle attività umane, le informazioni vengono gestite (registrate e scambiate) in forme diverse, a seconda delle necessità e capacità:
 - idee informali
 - linguaggio naturale (scritto o parlato, formale o colloquiale, in una lingua o in un'altra)
 - disegni, grafici, schemi
 - numeri
 - codici (anche segreti)
- e su vari supporti, dalla memoria umana alla carta

18

Gestione delle informazioni

- Nelle attività standardizzate dei sistemi informativi complessi, sono state introdotte col tempo forme di organizzazione e codifica delle informazioni.
- Ad esempio, nei servizi anagrafici si è iniziato con registrazioni discorsive e sono state poi introdotte informazioni via via più precise (e in un certo senso artificiali):
 - nome e cognome;
 - estremi anagrafici;
 - codice fiscale;

19

Informazioni e dati

- Nei sistemi informatici (e non solo in essi), le informazioni vengono rappresentate in modo essenziale, spartano: attraverso i dati.
 - **informazione** : notizia, dato o elemento che consente di avere conoscenza più o meno esatta di fatti, situazioni, modi di essere.
 - **dato** : ciò che è immediatamente presente alla conoscenza, prima di ogni elaborazione; (in informatica) elementi di informazione costituiti da simboli che debbono essere elaborati.

(Vocabolario della lingua italiana, 1987)

20

Dati e informazioni

- Quindi, i dati sono fatti elementari, informazioni codificate, che hanno bisogno di essere interpretate per fornire conoscenza

Esempio

'Mario' '275' su un foglio di carta sono due **dati**.
Se il foglio di carta viene fornito in risposta alla domanda "A chi mi devo rivolgere per il problema X; qual è il suo numero di telefono?", allora i dati possono essere interpretati ed arricchire la conoscenza.

21

Perché i dati?

- La rappresentazione precisa di forme più ricche di informazione e conoscenza è difficile
- I dati costituiscono spesso una risorsa strategica, perché più stabili nel tempo di altre componenti (processi, tecnologie, ruoli umani)

22

Base di dati

(accezione generica)

- **collezione di dati, utilizzati per rappresentare le informazioni di interesse per una o più applicazioni di una organizzazione.**

(accezione specifica)

- **collezione di dati gestita da un DBMS**

23

Sistema di gestione di basi di dati DataBase Management System — DBMS

Sistema (**prodotto software**) in grado di gestire **collezioni di dati** che siano (anche):

- **grandi** (di dimensioni (molto) maggiori della memoria centrale dei sistemi di calcolo utilizzati)
- **persistenti** (con un periodo di vita indipendente dalle singole esecuzioni dei programmi che le utilizzano)
- **condivise** (utilizzate da applicazioni diverse)

garantendo **affidabilità** (resistenza a malfunzionamenti hardware e software) e **privatezza** (con una disciplina e un controllo degli accessi). Come ogni prodotto informatico, un DBMS deve essere **efficiente** (utilizzando al meglio le risorse di spazio e tempo del sistema) ed **efficace** (rendendo produttive le attività dei suoi utilizzatori).

24

Condivisione

- Ogni organizzazione (specie se grande) è divisa in settori o comunque svolge diverse attività. A ciascun settore o attività corrisponde un (sotto-)sistema informativo (privato o porzione di un sistema più grande).
- Possono esistere sovrapposizioni fra i dati di interesse dei vari settori.
- Una base di dati è una risorsa **integrata**, condivisa fra i vari settori.

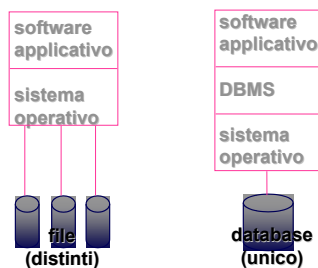
25

Condivisione (segue)

- L'integrazione e la condivisione permettono di ridurre la **ridondanza** (evitando ripetizioni) e, di conseguenza, le possibilità di incoerenza (o **inconsistenza**) fra i dati.
- Poiché la condivisione non è mai completa (o comunque non opportuna) i DBMS prevedono meccanismi di definizione della **privatezza** dei dati e di limitazioni all'accesso (**autorizzazioni**).
- La condivisione richiede un opportuno coordinamento degli accessi: **controllo della concorrenza**.

26

File system e DBMS



27

Efficienza

- Si misura (come in tutti i sistemi informatici) in termini di tempo di esecuzione (tempo di risposta) e spazio di memoria (principale e secondaria).
- I DBMS, a causa della varietà di funzioni, non sono necessariamente più efficienti dei file system.
- L'efficienza è il risultato della qualità del DBMS e delle applicazioni che lo utilizzano.

28

DBMS vs file system

- La gestione di insiemi di dati grandi e persistenti è possibile anche attraverso sistemi più semplici — gli ordinari **file system** dei sistemi operativi, che permettono di realizzare anche rudimentali forme di condivisione.
- Non esiste una linea netta di separazione fra DBMS e non-DBMS.
- I DBMS estendono le funzionalità dei file system, fornendo più servizi ed in maniera integrata (cfr. **efficacia**).
- I file system prevedono forme di condivisione, permettendo accessi contemporanei in lettura ed esclusivi in scrittura: se è in corso un'operazione di scrittura su un file, altri non possono accedere affatto al file. Nei DBMS, c'è maggiore flessibilità: si può accedere contemporaneamente a record diversi di uno stesso file o addirittura allo stesso record (in lettura).

29

DBMS vs file system (2)

- Nei programmi tradizionali che accedono a file, ogni programma contiene una descrizione della struttura del file stesso, con i conseguenti rischi di incoerenza fra le descrizioni (ripetute in ciascun programma) e i file stessi.
- Nei DBMS, esiste una porzione della base di dati (il **catalogo** o **dizionario**) che contiene una descrizione centralizzata dei dati, che può essere utilizzata dai vari programmi.

30

Descrizioni dei dati nei DBMS

- Esistono descrizioni e rappresentazioni dei dati a livelli diversi, che permettono l'**indipendenza dei dati** dalla rappresentazione fisica: i programmi fanno riferimento alla struttura a livello più alto, e le rappresentazioni sottostanti possono essere modificate senza necessità di modifica dei programmi.
- Precisiamo attraverso il concetto di modello dei dati.

31

Modello dei dati

- insieme di costrutti utilizzati per organizzare i dati di interesse e descriverne la dinamica
- componente fondamentale: **meccanismi di strutturazione** (o **costruttori di tipo**)
- come nei linguaggi di programmazione esistono meccanismi che permettono di definire nuovi tipi, così ogni modello dei dati prevede alcuni costruttori
- ad esempio, il **modello relazionale** prevede il costruttore **relazione**, che permette di definire insiemi di record omogenei

32

Tabelle: rappresentazione di relazioni

CORSI	Corso	Docente	Aula
	Basi di dati	Rossi	DS3
	Sistemi	Neri	N3
	Reti	Bruni	N3
	Controlli	Bruni	G

AULE	Nome	Edificio	Piano
	DS1	Ex-OMI	Terra
	N3	Ex-OMI	Terra
	G	Pincherle	Primo

33

Schemi e istanze

In ogni base di dati esistono:

- lo **schema**, sostanzialmente invariante nel tempo, che ne descrive la struttura (aspetto **intensionale**);
nell'esempio, le intestazioni delle tabelle
- l'**istanza**, costituita dai valori attuali, che possono cambiare molto e molto rapidamente (aspetto **estensionale**);
nell'esempio, il "corpo" di ciascuna tabella

34

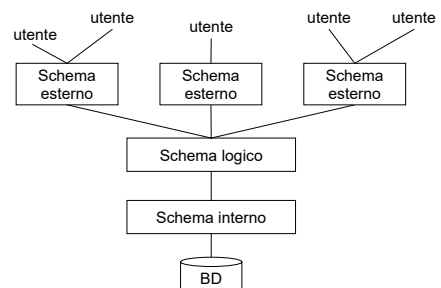
Due tipi (principali) di modelli

- modelli logici**: utilizzati nei DBMS esistenti per l'organizzazione dei dati; ad essi fanno riferimento i programmi; sono indipendenti dalle strutture fisiche;
esempi: relazionale, reticolare, gerarchico, a oggetti
- modelli concettuali**: permettono di rappresentare i dati in modo indipendente da ogni sistema, cercando di descrivere i concetti del mondo reale; sono utilizzati nelle fasi preliminari di progettazione;
il più noto è il modello **Entity-Relationship**

Di per sé, un modello potrebbe essere concettuale e logico al tempo stesso

35

Architettura standard (ANSI/SPARC) a tre livelli per DBMS



36

Architettura ANSI/SPARC: schemi

schema logico: descrizione dell'intera base di dati nel modello logico "principale" del DBMS

schema esterno: descrizione di parte della base di dati in un modello logico ("viste" parziali, derivate, anche in modelli diversi)

schema fisico: rappresentazione dello schema logico per mezzo di strutture fisiche di memorizzazione.

37

Una vista

Corsi

Corso	Docente	Aula
Basi di dati	Rossi	DS3
Sistemi	Neri	N3
Reti	Bruni	N3
Controlli	Bruni	G

Aule

Nome	Edificio	Piano
DS1	Ex-OMI	Terra
N3	Ex-OMI	Terra
G	Pincherle	Primo

CorsiSedi

Corso	Aula	Edificio	Piano
Sistemi	N3	Ex-OMI	Terra
Reti	N3	Ex-OMI	Terra
Controlli	G	Pincherle	Primo

38

Indipendenza dei dati

conseguenza della articolazione in livelli:

l'accesso avviene solo tramite il livello esterno (che può coincidere con il livello logico); due forme:

fisica: il livello logico e quello esterno sono indipendenti da quello fisico; una relazione è utilizzata nello stesso modo qualunque sia la sua realizzazione fisica (che può anche cambiare nel tempo senza che debbano essere modificate le forme di utilizzo)

logica: il livello esterno è indipendente da quello logico

- aggiunte o modifiche alle viste non richiedono modifiche al livello logico;
- modifiche allo schema logico che lascino inalterato lo schema esterno sono trasparenti

39

Linguaggi per basi di dati

- Un altro contributo all'efficacia: disponibilità di vari linguaggi e interfacce diverse.

L'accesso ai dati può avvenire

1. con linguaggi testuali interattivi
2. con comandi (come quelli del linguaggio interattivo) immersi in un linguaggio **ospite** (Pascal, C, Cobol, etc.)
3. con comandi (come quelli del linguaggio interattivo) immersi in un linguaggio ad hoc, con anche altre funzionalità (p.es. per grafici o stampe strutturate), anche con l'ausilio di strumenti di sviluppo (p. es. per la gestione di maschere)
4. con interfacce amichevoli (senza linguaggio testuale)

40

SQL, un linguaggio interattivo

```
SELECT Corso, Aula, Piano
FROM Aule, Corsi
WHERE Nome = Aula
AND Piano="Terra"
```

Corso	Aula	Piano
Reti	N3	Terra
Sistemi	N3	Terra

41

SQL immerso in Pascal (o altro linguaggio ad alto livello)

```
write('nome della città?'); readln(citta);
EXEC SQL DECLARE P CURSOR FOR
  SELECT NOME, REDDITO
  FROM PERSONE
  WHERE CITTA = :citta ;
EXEC SQL OPEN P ;
EXEC SQL FETCH P INTO :nome, :reddito ;
while SQLCODE = 0 do begin
  write('nome della persona:', nome, 'aumento?');
  readln(aumento);
  EXEC SQL UPDATE PERSONE SET REDDITO = REDDITO + :aumento
  WHERE CURRENT OF P
  EXEC SQL FETCH P INTO :nome, :reddito
end;
EXEC SQL CLOSE CURSOR P
```

42

SQL immerso in linguaggio ad hoc (Oracle PL/SQL)

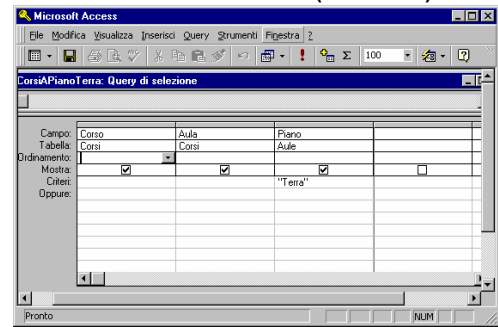
```

declare Stip number;
begin
    select Stipendio into Stip
    from Impiegato
    where Matricola = '575488'
    for update of Stipendio;
    if Stip > 30 then
        update Impiegato set Stipendio = Stipendio * 1.1 where Matricola = '575488';
    else
        update Impiegato set Stipendio = Stipendio * 1.15 where Matricola = '575488';
    end if;
    commit;
exception
    when no_data_found then
        insert into Errori
        values('Non esiste la matricola specificata',sysdate);
end;

```

43

Interazione non testuale (in Access)



44

Una distinzione terminologica (separazione fra dati e programmi)

- data definition language (DDL) :**
per la definizione di schemi (logici, esterni, fisici) e altre operazioni generali;
- data manipulation language (DML) :**
per l'interrogazione e l'aggiornamento di (istanze di) basi di dati.

45

Personaggi e interpreti

- progettisti e realizzatori di DBMS
- progettisti della base di dati e amministratori della base di dati (DBA)
- progettisti e programmatori di applicazioni
- utenti
 - utenti finali (terminalisti): eseguono applicazioni predefinite (**transazioni**)
 - utenti casuali: eseguono operazioni non previste a priori, usando linguaggi interattivi

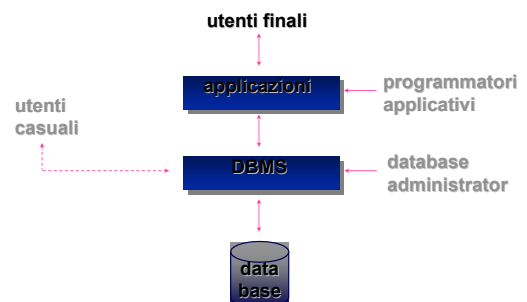
46

Database administrator (DBA)

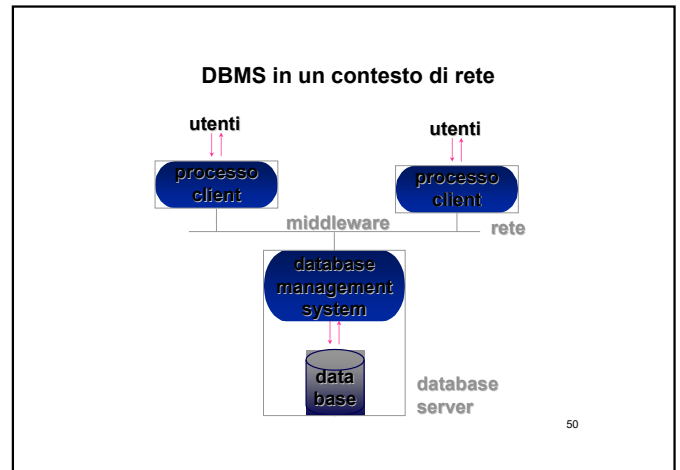
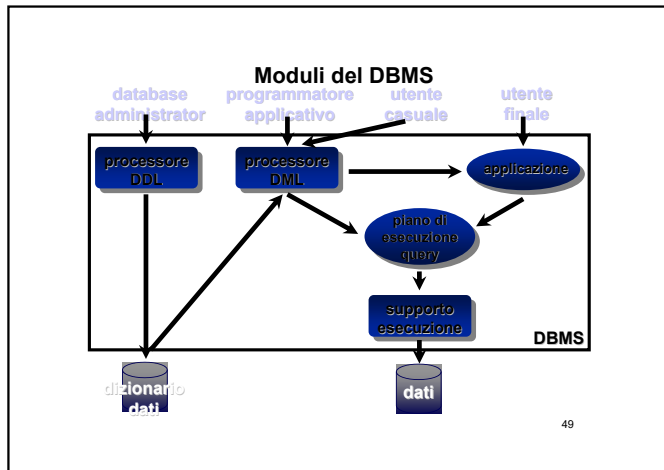
- Persona o gruppo di persone responsabile del controllo centralizzato e della gestione del sistema, delle prestazioni, dell'affidabilità, delle autorizzazioni.
- Le funzioni del DBA includono anche (parte di) quelle di progettazione.

47

Utenti del DBMS



48



Transazioni

- Programmi che realizzano attività frequenti e predefinite, con poche eccezioni, previste a priori.
- Esempi:
 - versamento presso uno sportello bancario
 - emissione di certificato anagrafico
 - dichiarazione presso l'ufficio di stato civile
 - prenotazione aerea
- Le transazioni sono di solito realizzate con programmi in linguaggio ospite (tradizionale o ad hoc).
- **N. B.:** il termine **transazione** ha un'altra accezione, più specifica: sequenza indivisibile di operazioni (o vengono eseguite tutte o nessuna).

51

Vantaggi e svantaggi dei DBMS

Pro

- dati come risorsa comune, base di dati come modello della realtà
- gestione centralizzata con possibilità di standardizzazione ed "economia di scala"
- disponibilità di servizi integrati
- riduzione di ridondanze e inconsistenze
- indipendenza dei dati (favorisce lo sviluppo e la manutenzione delle applicazioni)

Contro

- costo dei prodotti e della transizione verso di essi
- non scorporabilità delle funzionalità (con riduzione di efficienza)

52

Struttura del corso

Prima parte

- Basi di dati relazionali: modello, linguaggi formali, SQL

Seconda parte

- Progetto di basi di dati

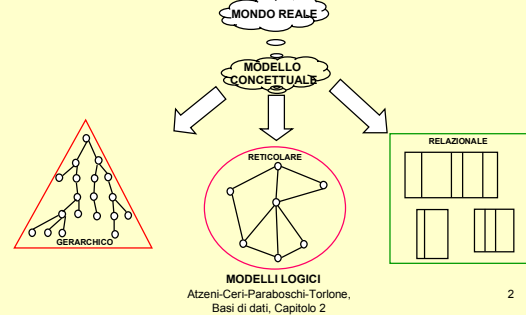
53

Atzeni, Ceri, Paraboschi, Torlone
Basi di dati
 McGraw-Hill, 1999

Capitolo 2: Il modello relazionale: strutture e vincoli

MODELLI DEI DATI

I MODELLI COSTITUISCONO UNA STRUTTURAZIONE SEMPLIFICATA DELLA REALTA' CHE NE ACCOGLIE ASPETTI SPECIFICI E AIUTA A COMPRENDERLA MEGLIO



2

MODELLI LOGICI DEI DATI

- SUPPORTANO UNA DESCRIZIONE DEI DATI CHE PUO' ESSERE **ELABORATA DAL SISTEMA** (DBMS)
- VENGONO **MAPPATI FACILMENTE** SULLE STRUTTURE FISICHE DI MEMORIZZAZIONE

Atzeni-Ceri-Paraboschi-Torlone,
 Basi di dati, Capitolo 2

3

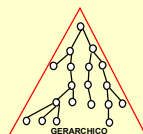
Cronologia dei modelli per la rappresentazione dei dati

- **Modello gerarchico (anni 60)**
- **Modello reticolare (anni 70)**
- **Modello relazionale (anni 80)**
- **Modello a oggetti (anni 90)**
- **Modello XML (anni 00)**

MODELLI LOGICI DEI DATI

GERARCHICO

- I DATI SONO RAPPRESENTATI COME **RECORD**
- LE ASSOCIAZIONI TRA I DATI SONO RAPPRESENTATE CON **PUNTATORI** IN UNA STRUTTURA AD **ALBERO**



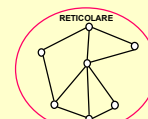
Atzeni-Ceri-Paraboschi-Torlone,
 Basi di dati, Capitolo 2

5

MODELLI LOGICI DEI DATI

RETICOLARE (CODASYL)

- I DATI SONO RAPPRESENTATI COME **RECORD**
- LE ASSOCIAZIONI TRA I DATI SONO RAPPRESENTATE CON **PUNTATORI** IN UNA STRUTTURA A **GRAFO** COMPLESSO



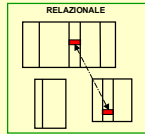
Atzeni-Ceri-Paraboschi-Torlone,
 Basi di dati, Capitolo 2

6

MODELLI LOGICI DEI DATI

RELAZIONALE

- I DATI SONO RAPPRESENTATI COME SEQUENZE DI VALORI DI ATTRIBUTI
- DATI CARATTERIZZATI DALLE STESSSE SEQUENZE DI ATTRIBUTI SONO RAGGRUPPATI IN TABELLE
- LE ASSOCIAZIONI TRA I DATI SONO OTTENUTE ASSOCIANDO VALORI DI ATTRIBUTI IN TABELLE DIVERSE



Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

7

Cronologia del modello relazionale

- Inventato da T. Codd, 1970
(IBM Research di Santa Teresa, Cal)
- Primi progetti:
SYSTEM R (IBM), Ingres (Berkeley Un.)
- Principali scoperte tecnologiche: 1978-1980
- Primi sistemi commerciali:
inizio anni '80 (Oracle, IBM-SQL DS e DB2, Ingres, Informix, Sybase)
- Successo commerciale: dal 1985.

Definizione informale

studente

colonna

schema

MATR	NOME	CITTA'	C-DIP
123	Carlo	Bologna	Inf
307	Giovanni	Milano	Log
415	Paola	Torino	Inf
702	Antonio	Roma	Log

istanza

riga

Relazione: tre accezioni

- **relazione matematica**: come nella teoria degli insiemi;
- relazione (dall'inglese **relationship**) che rappresenta una classe di fatti — una relazione matematica fra due **entità**, nel modello **Entity-Relationship**; talvolta tradotto con **associazione** o **correlazione**
- **relazione** secondo il modello relazionale dei dati.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

10

Relazione matematica

- $D_1, D_2, \dots, D_n$ (n insiemi anche non distinti)
- il prodotto cartesiano $D_1 \times D_2 \times \dots \times D_n$ è l'insieme di tutte le n -uple ordinate $(d_1, d_2, \dots, d_n)$ tali che $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$.
- una relazione matematica su $D_1, D_2, \dots, D_n$ è un sottoinsieme del prodotto cartesiano $D_1 \times D_2 \times \dots \times D_n$.
- $D_1, D_2, \dots, D_n$ sono i domini della relazione. Una relazione su n domini ha grado n .
- il numero di n -uple è la cardinalità della relazione. Nelle applicazioni reali, la cardinalità è sempre finita.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

11

Relazione matematica, esempio

- $D_1 = \{a, b\}$
- $D_2 = \{x, y, z\}$
- prodotto cartesiano $D_1 \times D_2$

a	x
a	y
a	z
b	x
b	y
b	z

- una relazione $r \subseteq D_1 \times D_2$

a	x
a	z
b	y
b	z

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

12

Relazione matematica, proprietà

- In base alla definizione, una relazione matematica è un **insieme** di n -uple **ordinate**:
($d_1, d_2, \dots, d_n$) tali che $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$
- una relazione è un **insieme**; quindi:
 - non è definito alcun ordinamento fra le n -uple;
 - le n -uple di una relazione sono distinte l'una dall'altra;
- le n -uple sono **ordinate**: l' i -esimo valore di ciascuna proviene dall' i -esimo dominio; è cioè definito un ordinamento fra i domini.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

13

Relazione matematica, esempio

$Partite \subseteq string \times string \times integer \times integer$

Juve	Lazio	3	1
Lazio	Milan	2	0
Juve	Roma	1	2
Roma	Milan	0	1

- Ciascuno dei domini ha due **ruoli** distinti, distinguibili attraverso la posizione: il primo e il terzo dominio si riferiscono a nome e reti della squadra ospitante; il secondo e il quarto a nome e reti della squadra ospitata.
- La struttura è **posizionale**

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

14

Relazioni nel modello relazionale dei dati

- A ciascun dominio associamo un nome (**attributo**), unico nella relazione, che "descrive" il ruolo del dominio.
- Nella rappresentazione tabellare, gli attributi possono essere usati come intestazioni delle colonne.

Casa	Fuori	RetiCasa	RetiFuori
Juve	Lazio	3	1
Lazio	Milan	2	0
Juve	Roma	1	2
Roma	Milan	0	1

- L'ordinamento fra gli attributi è irrilevante: la struttura è **non posizionale**

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

15

Formalizzando

- L'associazione fra domini e attributi è definita da una funzione $dom: X \in D$ che associa a ciascun attributo un dominio.
- Una **ennupla** su un insieme di attributi X è una funzione che associa a ciascun attributo A in X un valore del dominio $dom(A)$
- Una **relazione** su X è un insieme di ennuple su X

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

16

Notazioni

- Se t è una ennupla su X e $A \in X$, allora $t[A]$ (o $t.A$) indica il valore di t su A .
- Nell'esempio, se t è la prima ennupla della tabella
 $t[Fuori] = Lazio$
- La stessa notazione è estesa anche ad insiemi di attributi, nel qual caso denota ennuple: $t[Fuori, RetiF]$ è una ennupla su due attributi.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

17

Tabelle e relazioni

- Una tabella rappresenta una relazione se
 - i valori di ciascuna colonna sono fra loro omogenei (dallo stesso dominio)
 - le righe sono diverse fra loro
 - le intestazioni delle colonne sono diverse tra loro
- Inoltre, in una tabella che rappresenta una relazione
 - l'ordinamento tra le righe è irrilevante
 - l'ordinamento tra le colonne è irrilevante

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

18

Il modello relazionale è basato su valori

- i riferimenti fra dati in relazioni diverse sono rappresentati per mezzo di valori dei domini che compaiono nelle enuple.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

19

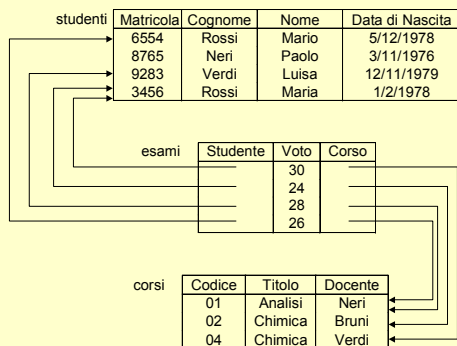
studenti	Matricola	Cognome	Nome	Data di Nascita
	6554	Rossi	Mario	5/12/1978
	8765	Neri	Paolo	3/11/1976
	9283	Verdi	Luisa	12/11/1979
	3456	Rossi	Maria	1/2/1978

esami	Studente	Voto	Corso
	3456	30	04
	3456	24	02
	9283	28	01
	6554	26	01

corsi	Codice	Titolo	Docente
	01	Analisi	Neri
	02	Chimica	Bruni
	04	Chimica	Verdi

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

20



Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

21

Vantaggi della struttura basata su valori

- indipendenza dalle strutture fisiche (si potrebbe avere anche con puntatori di alto livello) che possono cambiare anche dinamicamente
- si rappresenta solo ciò che è rilevante dal punto di vista dell'applicazione (dell'utente); i puntatori sono meno comprensibili per l'utente finale (senza, l'utente finale vede gli stessi dati dei programmatori)
- i dati sono portabili più facilmente da un sistema ad un altro
- i puntatori sono direzionali

Note:

- i puntatori possono esistere a livello fisico
- nel modello a oggetti esistono "una specie di puntatori", ad alto livello

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

22

Definizioni

Schema di relazione:

un **nome di relazione** R con un insieme di **attributi** $A_1, \dots, A_n$
 $R(A_1, \dots, A_n)$

Schema di base di dati:

insieme di schemi di relazione con nomi diversi:

$$R = \{R_1(X_1), \dots, R_n(X_n)\}$$

(Istanza di) relazione su uno schema $R(X)$:

insieme r di enuple su X

(Istanza di) base di dati su uno schema $R = \{R_1(X_1), \dots, R_n(X_n)\}$:

insieme di relazioni $r = \{r_1, \dots, r_n\}$ (con r_i relazione su R_i)

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

23

Notazione

attributi: lettere iniziali dell'alfabeto, maiuscole:

$A, B, C, A', A_1, \dots$

insiemi di attributi: lettere finali dell'alfabeto, maiuscole:

$X, Y, Z, X', X_1, \dots$

giustapposizione dei nomi degli attributi: $X=ABC$
 (anziché $X=\{A,B,C\}$)

unioni di insiemi: XY anziché $X \cup Y$

nomi di relazione: R e lettere circostanti, maiuscole, anche con
 indici e pedici: $R_i, S, S', \dots$

relazione: come il nome della relazione, ma in minuscolo

schema di base di dati: lettera maiuscola in grassetto $R, S, \dots$

base di dati: stesso simbolo dello schema, ma in minuscolo

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

24

Esempio

- sono possibili relazioni su un solo attributo

studenti	Matricola	Cognome	Nome	Data di Nascita
	6554	Rossi	Mario	5/12/1978
	8765	Neri	Paolo	3/11/1976
	9283	Verdi	Luisa	12/11/1979
	3456	Rossi	Maria	1/2/1978

Studenti lavoratori	Matricola
	6554
	8765

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

25

Strutture nidificate

"Da Filippo" Via Roma 23, Chivasso		
Ricevuta 2369 del 12/05/1997		
3	Coperti	6000
2	Antipasti	12000
3	Primi	27000
2	Bisteche	36000
Totale		81000

"Da Filippo" Via Roma 23, Chivasso		
Ricevuta 2456 del 16/05/1997		
2	Coperti	4000
1	Antipasti	6000
2	Primi	15000
2	Orate	50000
2	Caffè	3000
Totale		78000

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

26

Rappresentazione di strutture nidificate per mezzo di relazioni

ricevute	Numero	Data	Totale
	2369	12/05/1997	81000
	2456	16/05/1997	78000

dettaglio	Numero	Quantità	Descrizione	Importo
	2369	3	Coperti	6000
	2369	2	Antipasti	12000
	2369	3	Primi	27000
	2369	2	Bisteche	36000
	2456	2	Coperti	4000
	2456	1	Antipasti	6000
	2456	2	Primi	15000
	2456	2	Orate	50000
	2456	2	Caffè	3000

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

27

- Abbiamo rappresentato veramente tutti gli aspetti delle ricevute?
- Dipende da che cosa ci interessa realmente!
 - l'ordine delle righe e' rilevante?
 - possono esistere linee ripetute in una ricevuta?
- Sono possibili rappresentazioni diverse

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

28

Rappresentazione alternativa

ricevute	Numero	Data	Totale
	2369	12/05/1997	81000
	2456	16/05/1997	78000

dettaglio	Numero	Riga	Quantità	Descrizione	Importo
	2369	1	3	Coperti	6000
	2369	2	2	Antipasti	12000
	2369	3	3	Primi	27000
	2369	4	2	Bisteche	36000
	2456	1	2	Coperti	4000
	2456	2	1	Antipasti	6000
	2456	3	2	Primi	15000
	2456	4	2	Orate	50000
	2456	5	2	Caffè	3000

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

29

Informazione incompleta

- Il modello relazionale impone ai dati una struttura rigida:
 - le informazioni sono rappresentate per mezzo di ennuple
 - solo alcuni formati di ennuple sono ammessi:
 - quelli che corrispondono agli schemi di relazione
- I dati disponibili possono non corrispondere esattamente al formato previsto, per varie ragioni.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

30

Informazione incompleta: motivazioni

- Firenze è provincia, ma non conosciamo l'indirizzo della prefettura
- Tivoli non è provincia: non ha prefettura
- Prato è "nuova" provincia: ha la prefettura?

Città	Prefettura
Roma	Via IV novembre
Firenze	
Tivoli	
Prato	

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

31

Informazione incompleta: soluzioni?

- non conviene (anche se spesso si fa) utilizzare valori ordinari del dominio (0, stringa nulla, "99", etc), per vari motivi:
 - potrebbero non esistere valori "non utilizzati"
 - valori "non utilizzati" potrebbero diventare significativi
 - in fase di utilizzo (ad esempio, nei programmi) sarebbe necessario ogni volta tener conto del "significato" di questi valori

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

32

Informazione incompleta nel modello relazionale

- Si adotta una tecnica rudimentale ma efficace:
 - **valore nullo**: denota l'assenza di un valore del dominio (e non è un valore del dominio)
- Formalmente, è sufficiente estendere il concetto di ennupla: $t[A]$, per ogni attributo A , è un valore del dominio $\text{dom}(A)$ oppure il valore nullo NULL
- Si possono (e debbono) imporre restrizioni sulla presenza di valori nulli

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

33

Troppi valori nulli!

Studenti	Matricola	Cognome	Nome	Nascita
	276545	Rossi	Maria	NULL
	NULL	Neri	Anna	23/04/1972
	NULL	Verdi	Fabio	12/02/1972

Esami	Studente	Voto	Corso
	276545	28	01
	NULL	27	NULL
	200768	24	NULL

Corsi	Codice	Titolo	Docente
	01	Analisi	Giani
	03	NULL	NULL
	NULL	Chimica	Belli

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

34

Tipi di valore nullo

- (almeno) tre casi differenti
 - **valore sconosciuto**: esiste un valore del dominio, ma non è noto (Firenze)
 - **valore inesistente**: non esiste un valore del dominio (Tivoli)
 - **valore senza informazione**: non è noto se esista o meno un valore del dominio (Prato)
- I DBMS non distinguono i tipi di valore nullo (e quindi implicitamente adottano il valore **senza informazione**)

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

35

Vincoli di integrità

- Esistono istanze di basi di dati che, pur sintatticamente corrette, non rappresentano informazioni possibili per l'applicazione di interesse.

Studenti	Matricola	Cognome	Nome	Nascita
	276545	Rossi	Maria	23/04/1968
	276545	Neri	Anna	23/04/1972
	788854	Verdi	Fabio	12/02/1972

Esami	Studente	Voto	Lode	Corso
	276545	28	e lode	01
	276545	32		02
	788854	23		03
	200768	30	e lode	03

Corsi	Codice	Titolo	Docente
	01	Analisi	Giani
	03	NULL	NULL
	02	Chimica	Belli

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

36

Vincolo di integrità

- Definizione
 - proprietà che deve essere soddisfatta dalle istanze che rappresentano informazioni corrette per l'applicazione ogni vincolo può essere visto come una funzione booleana (o un predicato) che associa ad ogni istanza il valore **vero** o **falso**.
- Tipi di vincoli:
 - vincoli intrarelazionali; casi particolari:
 - vincoli su valori (o di dominio)
 - vincoli di ennuola
 - vincoli interrelazionali

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

37

Vincoli di integrità, motivazioni

- risultano utili al fine di descrivere la realtà di interesse in modo più accurato di quanto le strutture permettano;
- forniscono un contributo verso la "qualità dei dati"
- costituiscono uno strumento di ausilio alla progettazione (vedremo la "normalizzazione")
- sono utilizzati dal sistema nella scelta della strategia di esecuzione delle interrogazioni

Nota:

- non tutte le proprietà di interesse sono rappresentabili per mezzo di vincoli esprimibili direttamente

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

38

Vincoli di ennuola

- Esprimono condizioni sui valori di ciascuna ennuola, indipendentemente dalle altre ennuole.
- Una possibile sintassi: espressione booleana (con AND, OR e NOT) di atomi che confrontano valori di attributo o espressioni aritmetiche su di essi.
- Un vincolo di ennuola è un **vincolo di dominio** se coinvolge un solo attributo
- Esempi:

$(Voto \geq 18) \text{ AND } (Voto \leq 30)$
 $(Voto = 30) \text{ OR NOT } (Lode = "e lode")$

Lorde = (Ritenute + Netto)

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

39

Identificazione delle ennuole

Matricola	Cognome	Nome	Corso	Nascita
6554	Rossi	Mario	Informatica	5/12/1978
8765	Rossi	Mario	Informatica	3/11/1976
4723	Verdi	Laura	Meccanica	10/7/1979
9283	Verdi	Mario	Informatica	3/11/1976
3456	Rossi	Laura	Meccanica	5/12/1978

- il numero di matricola identifica gli studenti:
 - non ci sono due ennuole con lo stesso valore sull'attributo Matricola
- i dati anagrafici identificano gli studenti:
 - non ci sono due ennuole uguali su tutti e tre gli attributi Cognome, Nome e Data di Nascita

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

40

Vincoli di chiave

- chiave**:
 - insieme di attributi che identificano univocamente le ennuole di una relazione
- più precisamente:
 - un insieme K di attributi è **superchiave** per una relazione r se r non contiene due ennuole distinte t_1 e t_2 con $t_1[K] = t_2[K]$
 - K è **chiave** per r se è una superchiave minimale (cioè non contiene un'altra superchiave) per r

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

41

Matricola	Cognome	Nome	Corso	Nascita
6554	Rossi	Mario	Informatica	5/12/1978
8765	Rossi	Mario	Informatica	3/11/1976
4723	Verdi	Laura	Meccanica	10/7/1979
9283	Verdi	Mario	Informatica	3/11/1976
3456	Rossi	Laura	Meccanica	5/12/1978

- Matricola è una chiave:
 - Matricola è superchiave
 - contiene un solo attributo e quindi è minimale
- Cognome, Nome, Nascita è un'altra chiave:
 - l'insieme Cognome, Nome, Nascita è superchiave
 - nessuno dei suoi sottoinsiemi è superchiave

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

42

Matricola	Cognome	Nome	Corso	Nascita
6554	Rossi	Mario	Informatica	5/12/1978
8765	Rossi	Mario	Elettronica	3/11/1976
4723	Verdi	Laura	Meccanica	10/7/1979
9283	Verdi	Mario	Informatica	3/11/1976
3456	Rossi	Laura	Meccanica	5/12/1978

- la relazione non contiene ennuple fra loro uguali su Cognome e Corso:
 - in ogni corso di studio gli studenti hanno cognomi diversi;
 - l'insieme { Cognome, Corso } è superchiave minimale e quindi chiave
- possiamo dire che questa proprietà è sempre soddisfatta?
 - No! In generale ci possono essere in un corso di studio studenti con lo stesso cognome

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

43

Chiavi, schemi e istanze

- i vincoli corrispondono a proprietà del mondo reale modellato dalla base di dati;
- quindi interessano a livello di schema (con riferimento cioè a tutte le istanze):
 - ad uno schema associamo un insieme di vincoli e consideriamo **corrette** (lecite, valide, ammissibili) solo le istanze che soddisfano tutti i vincoli;
 - single istanze possono soddisfare ulteriori vincoli ("per pura coincidenza")

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

44

Individuazione delle chiavi

- definendo uno schema di relazione, associamo ad esso i vincoli di chiave che vogliamo siano soddisfatti dalle sue istanze (corrette)
- li individuiamo
 - considerando le proprietà che i dati soddisfano nell'applicazione (il "frammento di mondo reale di interesse");
 - notando quali insiemi di attributi permettono di identificare univocamente le ennuple;
 - e individuando i sottoinsiemi minimali di tali insiemi che conservano la capacità di identificare le ennuple.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

45

Individuazione delle chiavi, esempio

- Allo schema di relazione
STUDENTI(Matricola, Cognome, Nome, Corso, Nascita)
associamo i vincoli che indicano come chiavi gli insiemi di attributi Matricola e Cognome, Nome, Nascita
 - La relazione

Matricola	Cognome	Nome	Corso	Nascita
6554	Rossi	Mario	Informatica	5/12/1978
8765	Rossi	Mario	Elettronica	3/11/1976
4723	Verdi	Laura	Meccanica	10/7/1979
9283	Verdi	Mario	Informatica	3/11/1976
3456	Rossi	Laura	Meccanica	5/12/1978
- è corretta, perché soddisfa i vincoli associati allo schema.
- Ne soddisfa anche altri. Ad esempio, Cognome, Corso è chiave per essa.

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

46

Esistenza delle chiavi

- poiché le relazioni sono insiemi, ogni relazione non può contenere ennuple distinte ma uguali fra loro:
 - ogni relazione ha come superchiave l'insieme degli attributi su cui è definita;
- poiché l'insieme di tutti gli attributi è una superchiave per ogni relazione, ogni schema di relazione ha tale insieme come superchiave;
- poiché l'insieme di attributi è finito, ogni schema di relazione ha (almeno) una chiave

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

47

Importanza delle chiavi

- l'esistenza delle chiavi garantisce l'accessibilità a ciascun dato della base di dati
- ogni singolo valore è univocamente accessibile tramite:
 - nome della relazione
 - valore della chiave
 - nome dell'attributo
- le chiavi sono lo strumento principale attraverso il quale vengono correlati i dati in relazioni diverse ("il modello relazionale è basato su valori")

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

48

Chiavi e valori nulli

- In presenza di valori nulli, i valori degli attributi che formano la chiave
 - non permettono di identificare le ennuple come desiderato
 - né permettono di realizzare facilmente i riferimenti da altre relazioni

Matricola	Cognome	Nome	Nascita	Corso
NULL	Rossi	Luca	NULL	Informatica
8765	Rossi	Mario	01/05/61	Civile
4856	Neri	Mario	NULL	NULL
NULL	Neri	Mario	05/03/63	Civile

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

49

Chiave primaria

- La presenza di valori nulli nelle chiavi deve essere limitata
- Soluzione pratica: per ogni relazione scegliamo una chiave (la chiave primaria) su cui non ammettiamo valori nulli.
- Notazione per la chiave primaria: gli attributi che la compongono sono sottolineati

<u>Matricola</u>	<u>Cognome</u>	<u>Nome</u>	<u>Nascita</u>	<u>Corso</u>
6554	Rossi	Luca	NULL	Informatica
8765	Rossi	Mario	01/05/61	Civile
4856	Neri	Mario	NULL	NULL
6590	Neri	Mario	05/03/63	Civile

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

50

Vincoli di integrità referenziale ("foreign key")

- informazioni in relazioni diverse sono correlate attraverso valori comuni
- in particolare, valori delle chiavi (primarie, di solito)
- un **vincolo di integrità referenziale** fra un insieme di attributi X di una relazione R_1 e un'altra relazione R_2 impone ai valori su X di ciascuna ennupla dell'istanza di R_1 di comparire come valori della chiave (primaria) dell'istanza di R_2

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

51

Base di dati con vincoli di integrità referenziale

infrizioni	Codice	Data	Vigile	Prov	Numero
	65524	3/9/1997	343	MI	3K9886
	87635	4/12/1997	476	MI	6D5563
	82236	4/12/1997	343	RM	7C5567
	35632	6/1/1998	476	RM	7C5567
	76543	5/3/1998	548	MI	6D5563

vigili	Matricola	Cognome	Nome
	343	Rossi	Luca
	476	Neri	Pino
	548	Nicolosi	Gino

automobili	Prov	Numero	Proprietario	...
	MI	3K9886	Nestore	...
	MI	6D5563	Nestore	...
	RM	7C5567	Menconi	...
	RM	1A6673	Mussone	...
	MI	5E7653	Marchi	...

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

52

- nell'esempio, esistono vincoli di integrità referenziale fra:
 - l'attributo Vigile della relazione INFRAZIONI e la relazione VIGILI
 - gli attributi Prov e Numero di INFRAZIONI e la relazione AUTO

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

53

Base di dati che viola vincoli di integrità referenziale

infrizioni	Codice	Data	Vigile	Prov	Numero
	65524	3/9/1997	343	MI	3K9886
	87635	4/12/1997	476	MI	6D5563
	82236	4/12/1997	343	RM	7C5567
	35632	6/1/1998	476	RM	7C5567
	76543	5/3/1998	548	MI	6D5563

vigili	Matricola	Cognome	Nome
	343	Rossi	Luca
	548	Nicolosi	Gino

automobili	Prov	Numero	Proprietario	...
	MI	3K9886	Nestore	...
	RM	6D5563	Nestore	...
	MI	7C5567	Menconi	...
	RM	1A6673	Mussone	...
	MI	5E7653	Marchi	...

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

54

Vincoli di integrità referenziale: commenti

- I vincoli di integrità referenziale giocano un ruolo fondamentale nel concetto "modello relazionale basato su valori."
- Sono possibili meccanismi per il supporto alla gestione dei vincoli di integrità referenziale ("azioni" da svolgere in corrispondenza a violazioni).
- In presenza di valori nulli i vincoli possono essere resi meno restrittivi
- Attenzione ai vincoli su più attributi

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

55

Base di dati con vincoli di integrità referenziale

incidenti	Codice	Data	ProvA	NumeroA	ProvB	NumeroB
	65524	3/9/1997	MI	3K9886	RM	7C5567
	87635	4/12/1997	RM	6D5563	RM	1A6673
	82236	6/12/1997	MI	7C5567	RM	6D5563

automobili	Prov	Numero	Proprietario	...
	MI	3K9886	Nestore	...
	RM	6D5563	Nestore	...
	MI	7C5567	Menconi	...
	RM	1A6673	Mussone	...
	MI	5E7653	Marchi	...

Atzeni-Ceri-Paraboschi-Torlone,
Basi di dati, Capitolo 2

56

Basi di dati

Linguaggi di interrogazione

Docente: Stefano Paraboschi
parabosc@elet.polimi.it

Classificazione

a linguaggi formali

Algebra relazionale
Calcolo relazionale
Programmazione logica

b linguaggi programmatici

SQL: Structured Query Language
QBE: Query By Example

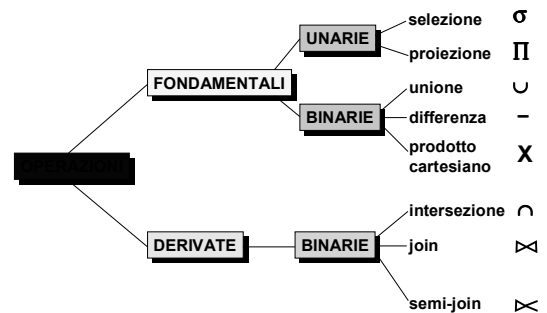
2

Algebra relazionale

- definita da Codd (70)
- molto utile per imparare a formulare query
- insieme minimo di 5 operazioni che danno l'intero potere espressivo del linguaggio

3

Una visione d'insieme



4

Esempio : gestione degli esami universitari

studente

MATR	NOME	CITTA'	C-DIP
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

esame

MATR	COD-CORSO	DATA	VOTO
123	1	7-9-97	30
123	2	8-1-98	28
702	2	7-9-97	20

corso

COD-CORSO	TITOLO	DOCENTE
1	matematica	Barozzi
2	informatica	Meo

5

Selezione

$\sigma_{\text{Nome}='Paola'}$ STUDENTE

e' una tabella (priva di nome) con

- schema :
lo stesso schema di STUDENTE
- istanza :
le tuple di STUDENTE che soddisfano il predicato di selezione

Matr	Nome	Città	CDip
415	Paola	Torino	Inf

6

Sintassi del predicato di selezione

espressione booleana di predicati semplici

operazioni booleane :

- AND (P1 AND P2) ($\wedge$)
- OR (P1 OR P2) ($\vee$)
- NOT (P1) ($\neg$)

predicati semplici :

- TRUE, FALSE
- termine
- comparatore
- termine

comparatore :

- =, !=, <, <=, >, >=

termine :

- costante, attributo
- espressione aritmetica di costanti e attributi

7

Esempio di selezione

σ

STUDENTE

(Città='Torino') OR
((Città='Roma')
AND NOT (CDip='Log'))

MATR	NOME	CITTA'	C-DIP
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

8

Proiezione

$\Pi_{\text{Nome, CDip}}$ STUDENTE

è una tabella (priva di nome) con

- schema :
gli attributi Nome e CDip
- istanza :
la restrizione delle tuple sugli attributi Nome e CDip

Nome	CDip
Carlo	Inf
Paola	Inf
Antonio	Log

9

Proiezione e duplicati

- nel modello formale la proiezione elimina i duplicati

Π_{CDip} STUDENTE

CDip
Inf
Log

- nel modello informale (e nei sistemi) la eliminazione dei duplicati va richiesta esplicitamente

10

Assegnamento

- serve per dare un nome al risultato di una espressione algebrica
- non fa parte delle operazioni algebriche

INFORMATICI = $\sigma_{\text{CDip}='Inf'}$ STUDENTI

TORINESI = $\sigma_{\text{Città}='Torino'}$ STUDENTI

11

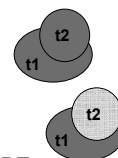
UNIONE E DIFFERENZA

- $\text{tabella1} \cup \text{tabella2}$

- $\text{tabella1} - \text{tabella2}$

tabella1 E tabella2 DEVONO ESSERE COMPATIBILI

- STESSO GRADO
- DOMINI ORDINATAMENTE DELLO STESSO TIPO



12

Unione

INFORMATICI $\cup$ TORINESI

è una tabella (priva di nome) con

- schema :
lo schema di INFORMATICI
- istanza :
la unione delle tuple di
INFORMATICI e TORINESI

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf

Per quanto riguarda le istanze, è commutativa

13

Differenza

INFORMATICI - TORINESI

è una tabella (priva di nome) con

- schema :
lo schema di INFORMATICI
- istanza :
la differenza delle tuple di
INFORMATICI e TORINESI

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf

Attenzione:
non è
commutativa

14

Prodotto cartesiano

$R \times S$

è una tabella (priva di nome) con

- schema :
gli attributi di R e S
($\text{grado}(R \times S) = \text{grado}(R) + \text{grado}(S)$)
- istanza :
tutte le possibili coppie di tuple di R e S
($\text{card}(R \times S) = \text{card}(R) * \text{card}(S)$)

15

Esempio

R1(A,B)

A	B
a	1
b	3

R2(C,D)

C	D
c	1
b	3
a	2

$R1 \times R2 (A,B,C,D)$

A	B	C	D
a	1	c	1
a	1	b	3
a	1	a	2
b	3	c	1
b	3	b	3
b	3	a	2

16

Intersezione

TABELLA1 $\cap$ TABELLA2

Come gli altri operatori insiemistici, si può fare se TABELLA1 e TABELLA2 sono compatibili

Derivabile tramite la seguente formula:

$$R \cap S = R - (R - S)$$

17

Intersezione

INFORMATICI $\cap$ TORINESI

è una tabella (priva di nome) con

- schema :
lo schema di INFORMATICI
- istanza :
la intersezione delle tuple di
INFORMATICI e TORINESI

Matr	Nome	Città	CDip
415	Paola	Torino	Inf

18

Join

STUDENTE $\bowtie$ **ESAME**
 $\text{STUDENTE.Matr} = \text{ESAME.Matr}$

è equivalente alla seguente espressione
 (operatore derivato):

$\sigma_{\text{STUDENTE.Matr} = \text{ESAME.Matr}} (\text{STUDENTE} \times \text{ESAME})$

attributi omonimi sono resi non ambigui
 usando la notazione "puntata":
ESAME.Matr, **STUDENTE.Matr**

19

Join

STUDENTE $\bowtie$ **ESAME**
 $\text{STUDENTE.Matr} = \text{ESAME.Matr}$

produce una tabella (priva di nome) con

- **schema** :
 la concatenazione degli schemi di **STUDENTE** e **ESAME**

- **istanza**:
 le tuple ottenute concatenando quelle tuple di **STUDENTE** e di **ESAME** che soddisfano il predicato

STUDENTE. Matr	Nome	Città	CDip	ESAME. Matr	Cod- Corso	Data	Voto
123	Carlo	Bologna	Inf	123	1	7-9-97	30
123	Carlo	Bologna	Inf	123	2	8-1-98	28
702	Antonio	Roma	Log	702	2	7-9-97	20

20

Sintassi del predicato di join

espressione congiuntiva di predicati
 semplici:

ATTR1 comp ATTR2

ove **ATTR1** appartiene a **TAB1**
ATTR2 appartiene a **TAB2**
 comp: =, !=, <, <=, >, >=

EQUI-JOIN :
 soli confronti di uguaglianza

21

Join naturale

equi-join di tutti gli attributi omonimi
 (si omette il predicato, si elimina la
 colonna ripetuta)

STUDENTE $\bowtie$ **ESAME**

Matr	Nome	Città	CDip	Cod- Corso	Data	Voto
123	Carlo	Bologna	Inf	1	7-9-97	30
123	Carlo	Bologna	Inf	2	8-1-98	28
702	Antonio	Roma	Log	2	7-9-97	20

22

Join naturale di tre tabelle

STUDENTE $\bowtie$ **ESAME** $\bowtie$ **CORSO**

Matr	Nome	Città	CDip	Cod- Corso	Data	Voto	Titolo	Docente
123	Carlo	Bologna	Inf	1	7-9-97	30	matem	barozzi
123	Carlo	Bologna	Inf	2	8-1-98	28	infor	meo
702	Antonio	Roma	Log	3	7-9-97	20	infor	meo

23

Semi-join

STUDENTE $\bowtie$ **ESAME**
 $\text{STUDENTE.Matr} = \text{ESAME.Matr}$

è equivalente alla seguente espressione (operatore
 derivato):

$\Pi_{\text{STUDENTE}} (\text{STUDENTE} \bowtie_{\text{STUDENTE.Matr} = \text{ESAME.Matr}} \text{ESAME})$

Produce come risultato un sottoinsieme di
STUDENTE

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
702	Antonio	Roma	Log

24

Semi-join naturale

STUDENTE $\bowtie$ ESAME

è equivalente alla seguente espressione:

$\Pi_{\text{STUDENTE}} (\text{STUDENTE} \bowtie \text{ESAME})$

E' equivalente alla query precedente

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
702	Antonio	Roma	Log

25

Espressioni algebriche

- Concatenazione di più operazioni algebriche
- Esprimono interrogazioni in modo formale
- Consentono di estrarre informazioni dai dati

26

Selezione e proiezione

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

- quali studenti sono iscritti al diploma di informatica?

$\Pi_{\text{Nome}} \sigma_{\text{CDip}='Inf'} \text{STUDENTE}$

NOME
Carlo
Paola

27

Selezione e proiezione

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

- quali studenti di Logistica non sono di Milano?

$\Pi_{\text{Nome}} \sigma_{\text{CDip}='Log' \wedge \text{Città} \neq 'Milano'} \text{STUDENTE}$

NOME
Antonio

28

Esempio : gestione degli esami universitari

STUDENTE

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

ESAME

Matr	Cod-Corso	Data	Voto
123	1	7-9-97	30
123	2	8-1-98	28
702	2	7-9-97	20

CORSO

Cod-Corso	Titolo	Docente
1	matematica	Barozzi
2	informatica	Meo

29

Ricordiamo anche che...

STUDENTE $\bowtie$ ESAME $\bowtie$ CORSO

Matr	Nome	Città	CDip	Cod-Corso	Data	Voto	Titolo	Docente
123	Carlo	Bologna	Inf	1	7-9-97	30	matem	barozzi
123	Carlo	Bologna	Inf	2	8-1-98	28	infor	meo
702	Antonio	Roma	Log	3	7-9-97	20	infor	meo

30

Selezione, proiezione e join

- quali studenti hanno preso 30 in matematica?

Matr	Nome	Città	CDip	Cod-Corso	Data	Voto	Titolo	Docente
123	Carlo	Bologna	Inf	1	7-9-97	30	matem	barozzi
123	Carlo	Bologna	Inf	2	8-1-98	28	infor	meo
702	Antonio	Roma	Log	3	7-9-97	20	infor	meo

31

Selezione, proiezione e join

- quali studenti hanno preso 30 in matematica?

$\Pi_{\text{Nome}} \sigma_{\text{Voto}=30 \wedge \text{Titolo}=\text{'matematica'}} (\text{STUDENTE} \bowtie \text{ESAME} \bowtie \text{CORSO})$

Nome
Carlo

32

Equivalenza di espressioni

- quali studenti hanno preso 30 in matematica?

STUDENTE ↑

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

ESAME

Matr	Cod-Corso	Data	Voto
123	1	7-9-97	30
123	2	8-1-98	28
702	2	7-9-97	20

CORSO

Cod-Corso	Titolo	Docente
1	matematica	Barozzi
2	informatica	Meo

33

Equivalenza di espressioni

- quali studenti hanno preso 30 in matematica?

$\Pi_{\text{Nome}} (\text{STUDENTE} \bowtie (\sigma_{\text{Voto}=30} \text{ESAME}) \bowtie (\sigma_{\text{Titolo}=\text{'matematica'}} \text{CORSO}))$

- equivalente a:

$\Pi_{\text{Nome}} \sigma_{\text{Voto}=30 \wedge \text{Titolo}=\text{'matematica'}} (\text{STUDENTE} \bowtie \text{ESAME} \bowtie \text{CORSO})$

34

Selezione, proiezione e join

- quali professori hanno esaminato Antonio?

Matr	Nome	Città	CDip	Cod-Corso	Data	Voto	Titolo	Docente
123	Carlo	Bologna	Inf	1	7-9-97	30	matem	barozzi
123	Carlo	Bologna	Inf	2	8-1-98	28	infor	meo
702	Antonio	Roma	Log	3	7-9-97	20	infor	meo

35

Selezione, proiezione e join

- quali professori hanno esaminato Antonio?

$\Pi_{\text{Docente}} \sigma_{\text{Nome}=\text{'Antonio'}} (\text{STUDENTE} \bowtie \text{ESAME} \bowtie \text{CORSO})$

Docente
Meo

36

Equivalenza di espressioni

STUDENTE			
Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

ESAME			
Matr	Cod-Corso	Data	Voto
123	1	7-9-97	30
123	2	8-1-98	28
702	2	7-9-97	20

CORSO		
Cod-Corso	Titolo	Docente
1	matematica	Barozzi
2	informatica	Meo

37

Equivalenza di espressioni

- quali professori hanno esaminato Antonio?

Π_{Docente}
 $(\text{CORSO} \bowtie (\text{ESAME} \bowtie (\sigma_{\text{Nome} = \text{'Antonio'}} \text{STUDENTE})))$

- equivalente a:

$\Pi_{\text{Docente}} \sigma_{\text{Nome} = \text{'Antonio'}} (\text{STUDENTE} \bowtie \text{ESAME} \bowtie \text{CORSO})$

38

Espressioni con unione e differenza

- estrarre la matricola degli studenti romani oppure degli studenti che hanno sostenuto un esame il giorno 8-1-01

$(\Pi_{\text{Matr}} \sigma_{\text{Città} = \text{'Roma'}} \text{STUDENTE})$

∪

$(\Pi_{\text{Matr}} \sigma_{\text{Data} = \text{'8-1-01'}} \text{ESAME})$

Matr	Matr	Matr
702	123	702
		123

39

Espressioni con unione e differenza

- estrarre la matricola degli studenti che hanno preso almeno un voto superiore a 28 e non sono mai scesi sotto il 25

$(\Pi_{\text{Matr}} \sigma_{\text{Voto} \geq 28} \text{ESAME})$

-

$(\Pi_{\text{Matr}} \sigma_{\text{Voto} < 25} \text{ESAME})$

Matr	Matr	Matr
123	702	123

40

Espressioni complesse

- estrarre il nome degli studenti che non hanno mai preso meno di 28

$\Pi_{\text{Nome}} \text{STUDENTE} \bowtie$
 $(\Pi_{\text{Matr}} \text{ESAME})$

-

$\Pi_{\text{Matr}} \sigma_{\text{Voto} < 28} \text{ESAME})$

- spiegazione: prima trovo le matricole di tutti gli studenti meno le matricole di coloro che hanno preso meno di 28, poi trovo i loro nomi.

41

Espressioni complesse

- estrarre il nome degli studenti che hanno sostenuto "informatica" e "matematica" lo stesso giorno

$\Pi_{\text{Nome}} \text{STUDENTE} \bowtie$

$((\text{ESAME} \bowtie \sigma_{\text{Titolo} = \text{'informatica'}} \text{CORSO})$

$\bowtie \text{Matr} = \text{Matr} \wedge \text{Data} = \text{Data}$

$(\text{ESAME} \bowtie \sigma_{\text{Titolo} = \text{'matematica'}} \text{CORSO}))$

- spiegazione: prima trovo le matricole di coloro che hanno dato i due esami nello stesso giorno, poi trovo i loro nomi.

42

Espressioni complesse

- estrarre l'ultimo esame di ciascuno studente

ESAME

-

(ESAME $\triangleright < \text{Matr} = \text{Matr} \wedge \text{Data} < \text{Data}$ ESAME)

- spiegazione: prima trovo gli esami che non sono ultimi, cioè che sono seguiti da qualche esame a pari studente in data superiore, poi sottraggo da tutti gli esami questi "esami non ultimi" e trovo gli ultimi esami di ciascuno studente

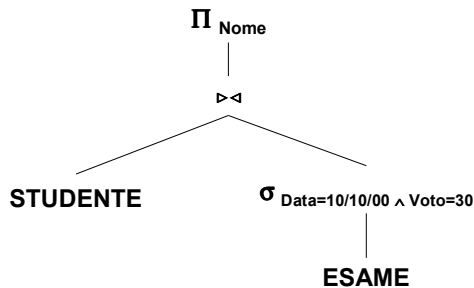
43

Rappresentazione delle espressioni tramite alberi

- Ogni espressione dell'algebra relazionale può essere rappresentata in modo grafico da un albero, che esplicita l'ordine di valutazione degli operatori
- Ogni operatore corrisponde a un nodo
 - operatori unari con un ramo in ingresso e uno in uscita
 - operatori binari con due rami in ingresso e uno in uscita

44

Esempio di albero



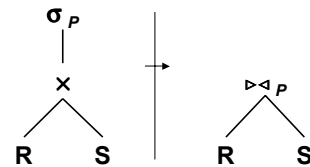
Corrisponde a:

$\Pi_{\text{Nome}} \text{STUDENTE} \triangleright \triangleleft \sigma_{\text{Data}=10/10/00 \wedge \text{Voto}=30} \text{ESAME}$

45

Trasformazioni di equivalenza per espressioni algebriche

1) Eliminazione dei prodotti cartesiani

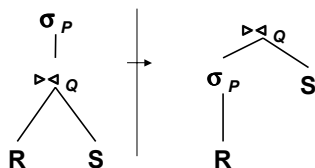


Vale se p è una congiunzione di predicati del tipo ATTR comp ATTR

46

Trasformazioni di equivalenza per espressioni algebriche

2) Push della selezione rispetto al join

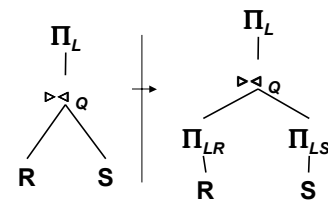


Vale se p è un predicato che si applica ai soli attributi di R

47

Trasformazioni di equivalenza per espressioni algebriche

3) Push della proiezione rispetto al join

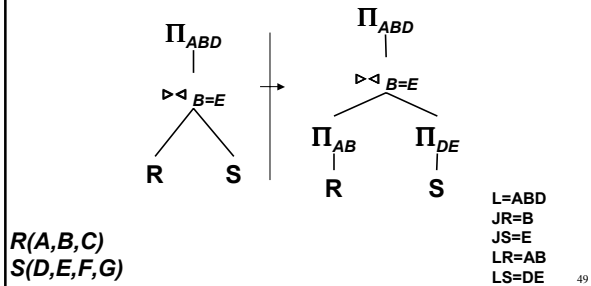


JR e JS sono gli attributi di R e S necessari a valutare Q
 $LR = L - \text{schema}(S) + JR$
 $LS = L - \text{schema}(R) + JS$

48

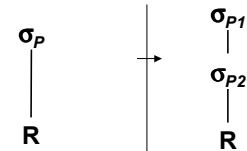
Esempio

3) Push della proiezione rispetto al join



Trasformazioni di equivalenza per espressioni algebriche

4) Idempotenza della selezione

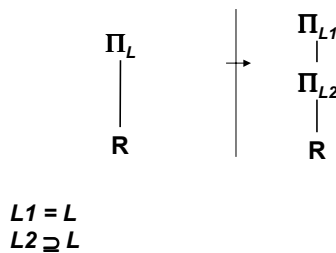


$$P = P1 \wedge P2$$

50

Trasformazioni di equivalenza per espressioni algebriche

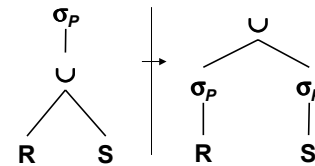
5) Idempotenza della proiezione



51

Trasformazioni di equivalenza per espressioni algebriche

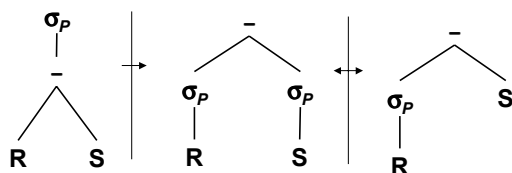
6) Push della selezione rispetto all'unione



52

Trasformazioni di equivalenza per espressioni algebriche

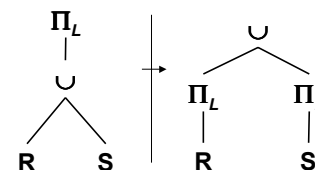
7) Push della selezione rispetto alla differenza



53

Trasformazioni di equivalenza per espressioni algebriche

8) Push della proiezione rispetto all'unione

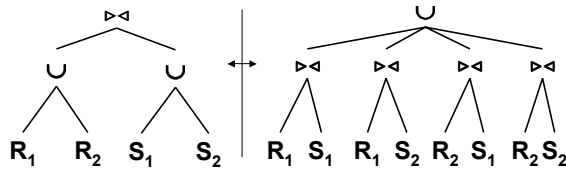


Attenzione: non vale il push della proiezione rispetto alla differenza e all'intersezione

54

Trasformazioni di equivalenza per espressioni algebriche

9) Commutazione di join e unione



55

Formule utili

$$R \bowtie R = R$$

$$R \cup R = R$$

$$R - R = \emptyset$$

$$R \bowtie \sigma_P R = \sigma_P R$$

$$R \cup \sigma_P R = R$$

$$R - \sigma_P R = \sigma_{\neg P} R$$

$$\sigma_{P1} R \bowtie \sigma_{P2} R = \sigma_{P1 \wedge P2} R$$

$$\sigma_{P1} R \cup \sigma_{P2} R = \sigma_{P1 \vee P2} R$$

$$\sigma_{P1} R - \sigma_{P2} R = \sigma_{P1 \wedge \neg P2} R$$

$$\sigma_P \emptyset = \emptyset$$

$$\Pi_L \emptyset = \emptyset$$

$$R \cup \emptyset = R$$

$$R - \emptyset = R$$

$$\emptyset - R = \emptyset$$

$$R \cap \emptyset = \emptyset$$

$$R \times \emptyset = \emptyset$$

$$R \bowtie \emptyset = \emptyset$$

56

Ottimizzazione algebrica

- Tra tutte le rappresentazioni equivalenti, conviene scegliere quella meno costosa da eseguire
- Criterio informale: minimizzare la dimensione dei risultati intermedi
- Tecnica:
 - utilizzare dove possibile le trasformazioni di push (2, 3, 6, 7, 8);
 - usare le trasformazioni di idempotenza (4, 5) per generare nuove selezioni e proiezioni

57

Esempio di ottimizzazione

Si ha lo schema:

$R(A,B,C)$

$S(C,D,E)$

$T(D,E,F,G)$

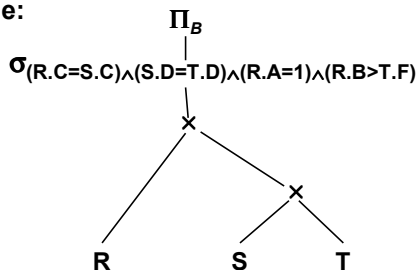
Si deve ottimizzare l'espressione algebrica:

$$\Pi_B \sigma_{(R.C=S.C) \wedge (S.D=T.D) \wedge (R.A=1) \wedge (R.B>T.F)} R \times S \times T$$

58

Esempio di ottimizzazione

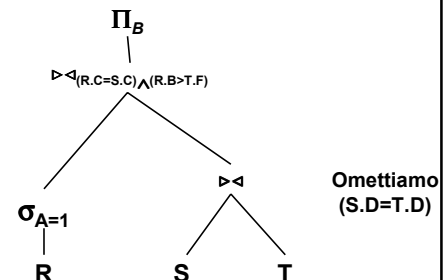
- Con la rappresentazione ad albero, si ottiene:



59

Esempio di ottimizzazione

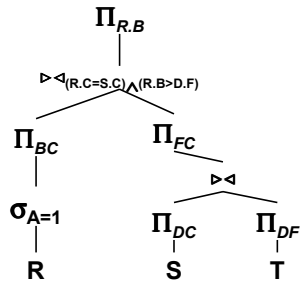
- Applicando le trasformazioni, si ottiene:



60

Esempio di ottimizzazione

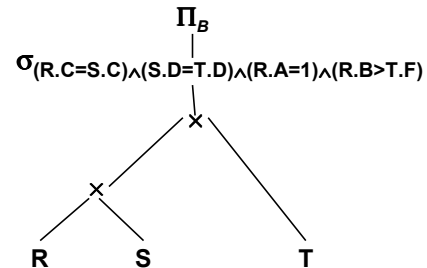
- Aggiungendo le proiezioni, si ottiene:



61

Esercizio

- Ripetere con:



62

Esercizi di ottimizzazione

- 1) Si ha lo schema: R(A,B)
S(A,B)
- Ottimizzare l'espressione algebrica:
 $\sigma_{(S.A=R.A) \wedge (R.A>2) \wedge (S.A=1)} R \times S$

Il risultato è una relazione vuota, perchè il predicato è una CONTRADDIZIONE!!!!

63

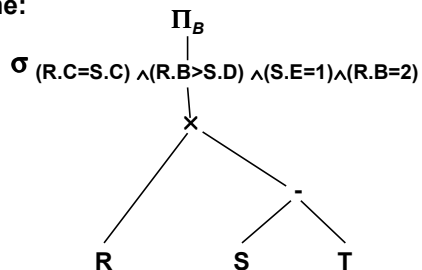
Esercizi di ottimizzazione

- 2) Si ha lo schema: R(A,B,C)
S(C,D,E)
T(C,D,E)
- Ottimizzare l'espressione algebrica:
 $\Pi_{AD} \sigma_{(R.C=S.C) \wedge (S.E=1) \wedge (R.B=2) \wedge (R.B>S.D)} (R \times (S - T))$

64

Esempio di ottimizzazione

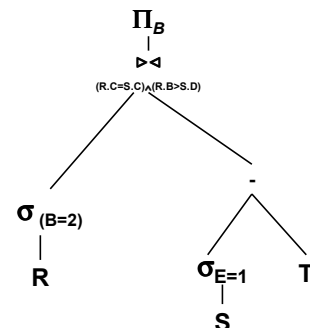
- Con la rappresentazione ad albero, si ottiene:



65

Esempio di ottimizzazione

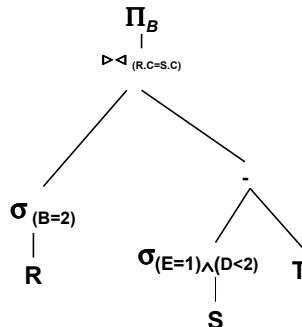
- Applicando le trasformazioni, si ottiene:



66

Esempio di ottimizzazione

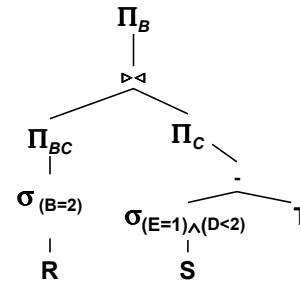
- Ragionando sui predicati, si ottiene:



67

Esempio di ottimizzazione

- Aggiungendo le proiezioni, si ottiene:



68

Ricerca della minima espressione contenitore (contain-view)

- Date n query, è un'espressione che consente di estrarre i risultati delle n query tramite selezioni e proiezioni su di essa

- Esempio: $R(A,B,C)$ $S(B,C,D,E,F)$ $T(D,G)$

1) $\Pi_{R,C,G}$

$$\sigma_{(A=1) \wedge (R.B=S.B) \wedge (S.D=T.D) \wedge (E=1) \wedge (G>3)} R \times S \times T$$

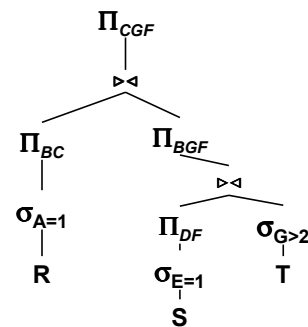
2) $\Pi_{R,C,F}$

$$\sigma_{(R.C>G) \wedge (A=1) \wedge (E=1) \wedge (F>3) \wedge (G>2) \wedge (S.D=T.D) \wedge (R.B=S.B)} R \times S \times T$$

69

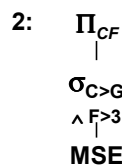
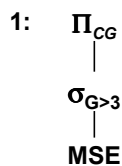
Estrazione della contain-view

- La massima sub espressione comune (MSE):



70

Calcolo delle query tramite la MSE



71

Calcolo relazionale

- Una famiglia di linguaggi formali
- Due tipi principali
 - Calcolo delle tuple (TRC, Tuple Relational Calculus)
 - Calcolo dei domini (DRC, Domain Relational Calculus)
- TRC in due versioni:
 - Con tuple ristrette sul range
 - Con tuple arbitrarie
- Guarderemo il TRC con tuple arbitrarie (anche se non è descritto nel libro!)

72

TRC è dichiarativo

- Esprime cosa si vuole nel risultato ma non come ottenerlo
- E' diverso dall'algebra, che è procedurale
- La dichiaratività è una caratteristica tipica dei linguaggi relazionali

73

Definizione formale del TRC

- **Forma standard:** $\{ t \mid p(t) \}$
- $p(t)$ è una *formula*, costruita tramite *atomi*
- **Atomi**
 - $t \in R$
 - $t1 [A1] \text{ comp } t2 [A2]$
 - $t [A] \text{ comp } k$
- **comp** e' un *comparatore*: $=, <, >, >=, <=$
- k è una *costante*
- $t[A]$ è una *restrizione sull'attributo A della tupla t*

74

Definizione formale del TRC

- **Regole di costruzione delle formule**
 - un atomo è una formula
 - se p è una formula, lo sono anche $\neg p$ e (p)
 - se $p1$ e $p2$ sono formule, lo sono anche $p1 \wedge p2, p1 \vee p2, p1 \Rightarrow p2$
 - se p è una formula in cui s è una variabile, lo sono anche $\exists s \in R (p(s)), \forall s \in R (p(s))$

75

Proprietà del TRC

- Legge di De Morgan
$$p1 \wedge p2 \equiv \neg (\neg p1 \vee \neg p2)$$
- Corrispondenze tra quantificatori
$$\forall t \in R (p(t)) \equiv \neg \exists t \in R (\neg p(t))$$
- **Definizione di implicazione**
$$p1 \Rightarrow p2 \equiv \neg p1 \vee p2$$

76

Forme Normali

- **Dalle tre leggi segue che è possibile scrivere tutte le possibili espressioni senza implicazione e con:**
 - Un solo quantificatore
 - Un solo operatore binario
- La forma normale più usata (simile ad SQL) usa quantificatore esistenziale e congiunzione

77

Esempi di TRC

- Nome degli studenti che hanno preso 30 in "matematica"

$\{ t \mid \exists t1 \in \text{STUDENTE},$
 $\exists t2 \in \text{ESAME},$
 $\exists t3 \in \text{CORSO}$
 $($
 $t[\text{Nome}] = t1[\text{Nome}] \wedge$
 $t1[\text{Matr}] = t2[\text{Matr}] \wedge$
 $t2[\text{CodCorso}] = t3[\text{CodCorso}] \wedge$
 $t2[\text{Voto}] = 30 \wedge$
 $t3[\text{Titolo}] = \text{'matematica'}) \}$

78

Esempi di TRC

- Matricole degli studenti che hanno sostenuto “matematica” ma non “basi di dati”

$$\{ t \mid \exists t1 \in \text{ESAME}, \exists t2 \in \text{CORSO} \\ (t[\text{Matr}] = t1[\text{Matr}] \wedge \\ t1[\text{CodCorso}] = t2[\text{CodCorso}] \wedge \\ t2[\text{Titolo}] = \text{'matematica'}) \wedge \\ \neg (\exists t3 \in \text{ESAME}, \exists t4 \in \text{CORSO} \\ (t[\text{Matr}] = t3[\text{Matr}] \wedge \\ t3[\text{CodCorso}] = t4[\text{CodCorso}] \wedge \\ t4[\text{Titolo}] = \text{'basi di dati'}))) \}$$

79

Correttezza

- Si devono evitare formule *unsafe*:
 $\{ t \mid t \notin R \}$ dà un risultato infinito
- Si considerano corrette solo formule *indipendenti dal dominio*
 - la soluzione non dipende dal dominio degli attributi, ma solo dall'istanza del DB

80

AR è esprimibile tramite TRC

E' sufficiente mostrare che si possono realizzare i cinque operatori fondamentali:

- Selezione, $\sigma_{A=1} R$:
 $\{ t \mid \exists t \in R (t[A]=1) \}$
- Proiezione, $\Pi_{AC} R$:
 $\{ t \mid \exists t1 \in R (t[A,C]=t1[A,C]) \}$

81

AR è esprimibile tramite TRC

- Prodotto cartesiano, $R(A,B,C) \times S(D,E,F)$:
 $\{ t \mid \exists t1 \in R, \exists t2 \in S \\ (t[A,B,C]=t1[A,B,C] \wedge \\ t[D,E,F]=t2[D,E,F]) \}$

Esempio di join, $R(A,C) \bowtie_{A=B} S(B,D)$:
 $\{ t \mid \exists t1 \in R, \exists t2 \in S \\ (t[A,C] = t1[A,C] \wedge \\ t[B,D] = t2[B,D] \wedge \\ t[A] = t[B]) \}$

82

AR è esprimibile tramite TRC

- Unione, $R \cup S$:
 $\{ t \mid \exists t1 \in R, \exists t2 \in S \\ (t = t1 \vee t = t2) \}$
- Differenza, $R - S$:
 $\{ t \mid \exists t \in R (t \notin S) \}$

83

Anche TRC è esprimibile tramite AR

- La prova è più complicata
- Si devono escludere espressioni unsafe e dipendenti dal dominio
- Sotto queste ipotesi TRC e AR hanno lo stesso potere espressivo

84

Linguaggi di query basati sulla programmazione logica

- La programmazione logica è un paradigma di programmazione basato su regole
- Linguaggio di riferimento: Prolog (1970)
- Datalog: Prolog per basi di dati (1984)
- Differenze principali rispetto a Prolog (per chi lo conosce):
 - mancano i simboli di funzione
 - modello di valutazione pienamente dichiarativo

85

Base di dati d'esempio per Datalog

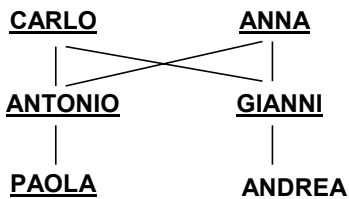
GENITORE

Genitore	Figlio
Carlo	Antonio
Carlo	Gianni
Anna	Antonio
Anna	Gianni
Gianni	Andrea
Antonio	Paola

PERSONA

Nome	Età	Sesso
Carlo	65	M
Antonio	40	M
Anna	60	F
Gianni	43	M
Andrea	22	M
Paola	20	F

86



87

Regole Datalog

- Ogni regola è composta da una testa (head o LHS) e da un corpo (body o RHS)
ES: $P :- P_1, P_2, \dots, P_n$

- Ogni P rappresenta un predicato (chiamato letterale), così composto:

- nome
 - argomenti:
 - costanti
 - variabili
 - simbolo "don't care" (non nella testa)
- ES: $\text{persona}(X, _, "M")$

88

Regole Datalog

Hanno l'interpretazione che la testa è vera se il corpo è vero

- Esempi di regole

$\text{Padre}(X, Y) :- \text{Persona}(X, _, 'M'), \text{Genitore}(X, Y)$

$\text{Madre}(X, Y) :- \text{Persona}(X, _, 'F'), \text{Genitore}(X, Y)$

- Le variabili del LHS devono apparire nel RHS

89

Fatti Datalog

- Ciascuna tupla corrisponde a un fatto (letterale *ground*)
- Ad esempio:

$\text{Genitore}(\text{"Carlo"}, \text{"Antonio"})$.

90

Query in Datalog

- Esempio di query (detta *goal*)
?- *Padre*("Carlo",X)
- Valutazione: si cerca una regola che abbia per testa il predicato *Padre* e si cercano valori per la variabile *X* (*unificazione*)
- Si ottiene $X = \{\text{"Antonio"}, \text{"Gianni"}\}$
- Un goal senza variabili restituisce *True* o *False*
 - ?- *Padre*("Carlo", "Antonio") $\Rightarrow$ *True*
 - ?- *Padre*("Carlo", "Andrea") $\Rightarrow$ *False*

91

Altre regole

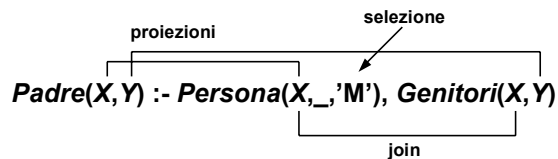
Nonno(X,Z) :- *Padre*(X,Y), *Genitore*(Y,Z)

Fratello(X,Y) :- *Genitore*(Z,X), *Genitore*(Z,Y), $X \neq Y$

Zio(X,Y) :- *Persona*(X,_'M'), *Fratello*(X,Z),
Genitore(Z,Y)

92

Corrispondenza tra Datalog e l'algebra relazionale



- Espressione corrispondente in algebra relazionale:
 $PADRE = \Pi_{1,5} \sigma_{3 = 'M'} PERSONA \bowtie_{1=1} GENITORI$

93

Database estensionale e intensionale

- Database estensionale (EDB)
 - insieme delle tabelle presenti nel DB
- Database intensionale (IDB)
 - insieme dei predicati che sono a sinistra in una regola
 - è la conoscenza dedotta a partire da EDB
- Normalmente si impone $EDB \cap IDB = \emptyset$

94

Negazione in Datalog

- Alcuni letterali del corpo possono essere negati
es: *Padre*(X,Y) :- *Genitore*(X,Y), not *Madre*(X,Y)
- L'uso della negazione in Datalog aumenta il potere espressivo del linguaggio
- Anche l'uso della negazione richiede cautela:
 $q(X) :- \neg p(X)$
 $p(0).$
 ?- $q(X)$ produce un risultato infinito!

95

Potere espressivo di Datalog

- Datalog senza negazione permette di rappresentare gli operatori $\{\sigma, \Pi, \times, \cup\}$
- $\{\sigma, \Pi, \times\}$ già visti
- Per l'unione si usano più regole con la stessa testa; $P = R \cup S$:
 $P(X,Y) :- R(X,Y)$
 $P(X,Y) :- S(X,Y)$
- Per la differenza serve il not; $P = R - S$:
 $P(X,Y) :- R(X,Y), \neg S(X,Y)$

96

Query ricorsive

- Datalog con negazione ha quindi un potere espressivo almeno pari all'algebra relazionale
- In effetti ha un potere superiore, perché permette l'espressione di query ricorsive
- Una query ricorsiva presenta il letterale della testa all'interno del corpo della regola:

$\text{Antenato}(X, Y) \text{ :- } \text{Genitore}(X, Y)$ ← start
 $\text{Antenato}(X, Y) \text{ :- } \text{Antenato}(X, Z), \text{Genitore}(Z, Y)$
 ← ricorsione

97

Meccanismo di valutazione

- Si può immaginare che venga seguito il seguente processo iterativo:

$\text{ANTENATO}^0 \Leftarrow \text{GENITORI}$
 $\text{ANTENATO}^1 \Leftarrow (\Pi_{1,4} \text{ANTENATO}^0 \triangleright_{2=3} \text{GENITORI}) \cup \text{ANTENATO}^0$
 $\text{ANTENATO}^2 \Leftarrow (\Pi_{1,4} \text{ANTENATO}^1 \triangleright_{2=3} \text{GENITORI}) \cup \text{ANTENATO}^1$
 ...

fino a che ANTENATO^n risulta pari a ANTENATO^{n-1} (punto fisso)

98

Valutazione delle query ricorsive

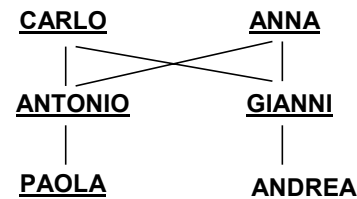
- Nella base dati d'esempio si ottiene il risultato illustrato a fianco

Nuovi elementi rispetto a GENITORE

ANTENATO

Carlo	Antonio
Carlo	Gianni
Anna	Antonio
Anna	Gianni
Gianni	Andrea
Antonio	Paola
Carlo	Andrea
Carlo	Paola
Anna	Andrea
Anna	Paola

99



100

Predicati e funzioni

- Nella definizione delle regole si possono usare predicati speciali
 - operatori di confronto
 - = (unificazione), ≠, <, ≤, ≥
 - funzioni aritmetiche
 - +, -, *, /
- Bisogna fare molta attenzione:
 - $n(X) \text{ :- } n(X - 1)$
 - $n(0)$
 - ?- $n(X)$ produce un risultato infinito!

101

Regole corrette

- La negazione deve essere *safe*
 - tutte le variabili di un letterale negato devono comparire in un letterale positivo del corpo della regola
 - $S(X) \text{ :- } R(X), \neg Q(Y)$ non va bene
- La negazione deve essere *stratificata*
 - sinteticamente, non ci devono essere cicli di dipendenza tra letterali negati
 - $P(X) \text{ :- } R(X)$
 - $P(X) \text{ :- } R(X), \neg P(X)$ non va bene



102

Riepilogo della terminologia Datalog

**Modello
relazionale
standard**

Datalog

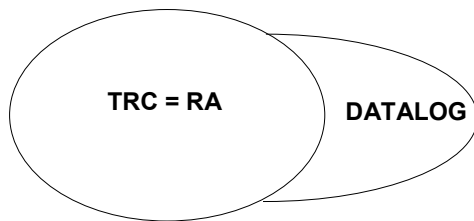
Relazione	Predicato
Attributo	Argomento
Tupla	Fatto
Query	Regole
Interrogazione	Goal

103

Sintesi sui poteri espressivi

- TRC = RA, con TRC safe e non dipendente dal dominio
- Datalog più espressivo di TRC e RA (a causa della ricorsione), con espressioni Datalog safe, stratificate e che non producono infiniti simboli-

104



105

In che modo si differenziano i linguaggi formali?

- L'algebra è un linguaggio PROCEDURALE: si dice esattamente in che modo valutare le interrogazioni (e si può ottimizzare)
- TRC è un linguaggio DICHIARATIVO: si dice cosa si vuole ottenere ma non come ottenerlo.
- DATALOG è pure dichiarativo, ma la costruzione delle dipendenze tra le regole può essere fatta in modo procedurale.

106

Perchè vi abbiamo raccontato tutto ciò?

- SQL assomiglia al calcolo relazionale
- L'algebra assomiglia a quanto viene effettivamente valutato da un DBMS
- Datalog avvicina le basi di dati al mondo della logica e della deduzioni

107

Basi di dati

Capitolo 4

1

SQL

- Il nome stava per *Structured Query Language*
- Più che un semplice linguaggio di query: si compone di una parte DDL e di una DML
 - DDL: definizione di domini, tabelle, indici, autorizzazioni, viste, vincoli, procedure, trigger
 - DML: linguaggio di query, linguaggio di modifica, comandi transazionali
- Storia:
 - Prima proposta: SEQUEL (IBM Research, 1974)
 - Prima implementazione commerciale in SQL/DS (IBM, 1981)

2

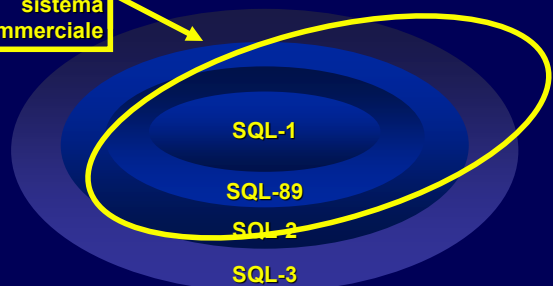
Standardizzazione di SQL

- La standardizzazione è stata cruciale per la diffusione di SQL (nell'ambito di ANSI e ISO)
 - Dal 1983, standard de facto
 - Prima versione ufficiale nel 1986 (SQL-1), rivista nel 1989 (SQL-89)
 - Seconda versione nel 1992 (SQL-2 o SQL-92)
 - Terza versione nel 1999 (SQL-3 o SQL:1999)
- In SQL-92 si distinguono tre livelli:
 - Entry SQL (più o meno equivalente a SQL-89)
 - Intermediate SQL
 - Full SQL
- La maggior parte dei sistemi è conforme al livello Entry e offre delle estensioni proprietarie per le funzioni avanzate

3

Potere espressivo di standard e sistemi commerciali

un tipico
sistema
commerciale



4

Domini

- I domini specificano i valori ammissibili per gli attributi
 - Simili ai meccanismi di definizione dei tipi dei linguaggi di programmazione
- Due categorie
 - Elementari (predefiniti dallo standard, elementary o built-in)
 - SQL-2 prevede 6 famiglie
 - Definiti dall'utente (user-defined)

5

Domini elementari, 1

- Caratteri
 - Caratteri singoli o stringhe
 - Le stringhe possono avere lunghezza variabile
 - Possono usare una famiglia di caratteri (character set) diversa da quella di default (es., Latin, Greek, Cyrillic, etc.)
 - Sintassi:

```
character [varying] [(Lunghezza)]
[character set FamigliaCaratteri]
```
 - Si possono usare le alternative più compatte `char` e `varchar`, rispettivamente per `character` e `character varying`
 - Esempi:
 - `char (6)`
 - `varchar (50)`

6

Domini elementari, 2

- Bit
 - Valori booleani (vero/falso), singoli o in sequenza (la sequenza può essere di lunghezza variabile)
 - Sintassi:
`bit [varying] [(Lunghezza)]`
- Domini numerici esatti
 - Valori esatti, interi o con una parte razionale
 - 4 alternative:
`numeric [(Precisione) [, Scala]]`
`decimal [(Precisione) [, Scala]]`
`integer`
`smallint`

7

Domini elementari, 3

- Domini numerici approssimati
 - Valori reali approssimati
 - Basati su una rappresentazione a virgola mobile: mantissa + esponente
`float [(Precisione)]`
`real`
`double precision`

8

Domini elementari, 4

- Istanti temporali
 - Ammettono dei campi
`date (campi year, month, day)`
`time [(Precisione)] [with time zone] : (campi hour, minute, second)`
`timestamp [(Precisione)] [with time zone]`
con timezone si hanno i due ulteriori campi `timezone_hour` e `timezone_minute`
- Intervalli temporali
 - `interval PrimaUnitàDiTempo [to UltimaUnitàDiTempo]`
 - Le unità di tempo sono divise in 2 gruppi:
 - year, month
 - day, hour, minute, second
 - Esempi:
 - `interval year to month`
 - `interval second`

9

Domini elementari, 5

- Nuovi domini semplici introdotti in SQL:1999
 - Boolean
 - BLOB Binary Large Object
 - CLOB Character Large Object
- SQL:1999 introduce anche dei costruttori (REF, ARRAY, ROW; vanno al di là del modello relazionale e non ne parliamo)

10

Domini definiti dagli utenti

- Paragonabile alla definizione dei tipi nei linguaggi di programmazione: si definiscono i valori ammissibili per un oggetto
- Un dominio è caratterizzato da
 - nome
 - dominio elementare
 - valore di default
 - insieme di vincoli (constraint)
- Sintassi:
`create domain NomeDominio as DominioElementare [ ValoreDefault ] [ Constraints ]`

11

Valori di default per il dominio

- Definiscono il valore che deve assumere l'attributo quando non viene specificato un valore durante l'inserimento di una tupla
- Sintassi:
`default <ValoreGenerico | user | null>`
- *ValoreGenerico* rappresenta un valore compatibile con il dominio, rappresentato come una costante o come un'espressione
- *user* è la login dell'utente che effettua il comando ¹²

Definizione di domini

- Esempio:
`create domain Voto as smallint default null`
- Rispetto ai linguaggi di programmazione
 - + vincoli, valori di default, domini di base più ricchi
 - costruttori assenti (solo ridenominazione di domini)
- La definizione di domini permette di riusare le definizioni e rende lo schema comprensibile e modificabile

13

Esempi di create domain

```
create domain OreLezione
as smallint default 40
```

```
create domain UserLog
as varchar(25) default user
```

14

Il valore "null"

`null`
è un valore polimorfo (che appartiene a tutti i domini) col significato di valore non noto

- il valore esiste in realtà ma è ignoto al database (es.: data di nascita)
- il valore è inapplicabile (es.: numero patente per minorenni)
- non si sa se il valore è inapplicabile o meno (es.: numero patente per un maggiorenne)

15

Vincoli intra-relazionali

- I vincoli sono condizioni che devono essere verificate da ogni istanza della base di dati
- I vincoli intra-relazionali coinvolgono una sola relazione (distinguibili ulteriormente a livello di tupla o di tabella)
 - `not null` (su un solo attributo; a livello di tupla)
 - `unique`: permette la definizione di chiavi candidate (opera quindi a livello di tabella); sintassi:
 - per un solo attributo:
`unique, dopo il dominio`
 - per diversi attributi:
`unique( Attributo {, Attributo } )`
 - `primary key`: definisce la chiave primaria (una volta per ogni tabella; implica `not null`); sintassi come per `unique`
 - `check`: descritto più avanti (può rappresentare vincoli di ogni tipo)

16

Definizione dei domini applicativi

```
create domain PrezzoQuotidiani
as decimal(4,2)
    default 0.90
    not null
```

17

Esempi di vincoli intra-relazionali

- Ogni coppia di attributi Nome e Cognome identifica univocamente ogni tupla
`Nome varchar(20) not null,`
`Cognome varchar(20) not null,`
`unique(Nome,Cognome)`
- Si noti la differenza con la seguente definizione (più restrittiva):
`Nome varchar(20) not null unique,`
`Cognome varchar(20) not null unique,`

18

Definizione di schemi

- Uno *schema* è una collezione di oggetti:
 - domini, tabelle, indici, asserzioni, viste, privilegi
- Uno schema ha un nome e un proprietario

- Sintassi:

```
create schema [ NomeSchema ]  
[ [ authorization ] Autorizzazione ]  
{ DefinizioneElementoSchema }
```

19

Definizione di tabelle

- Una tabella SQL consiste di:
 - un insieme ordinato di attributi
 - un insieme di vincoli (eventualmente vuoto)
- Comando `create table`
 - definisce lo schema di una relazione, creandone un'istanza vuota

- Sintassi:

```
create table NomeTabella  
(  
  NomeAttributo Dominio [ ValoreDiDefault ] [ Constraints ]  
  {, NomeAttributo Dominio [ ValoreDiDefault ] [ Constraints ] }  
  [ AltriConstraints ]  
)
```

20

Esempio di `create table` (1)

```
create table Studente  
(  
  Matr      character(6) primary key,  
  Nome      varchar(30) not null,  
  Città     varchar(20),  
  CDip      char(3) )
```

21

Esempio di `create table` (2)

```
create table Esame  
(  
  Matr      char(6),  
  CodCorso  char(6),  
  Data      date not null,  
  Voto      smallint not null,  
  primary key(Matr,CodCorso) )  
  
create table Corso  
(  
  CodCorso  char(6) primary key,  
  Titolo    varchar(30) not null,  
  Docente   varchar(20) )
```

22

Sintassi per l'integrità referenziale

- `references` e `foreign key` per l'integrità referenziale; sintassi:
 - per un solo attributo
 `references` dopo il dominio
 - per diversi attributi
 `foreign key ( Attributo {, Attributo } )`
 `references ...`
- Gli attributi descritti come `foreign key` nella tabella figlio devono presentare valori presenti come valori di chiave nella tabella padre
- Si possono associare anche delle politiche di reazione alle violazioni dei vincoli di integrità referenziale

23

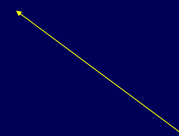
Esempio: studente - esame

Studente

Matr	
123	
415	
702	

Esame

Matr	
123	
123	
702	



24

Il problema degli orfani

Studente

Matr	
123	
415	
702	

orfani:

tuple che restano prive di padre a causa di cancellazioni e modifiche della tabella padre

Esame

Matr	
123	
123	
702	

25

Gestione degli orfani

- Le reazioni operano sulla tabella interna, in seguito a modifiche alla tabella esterna
- Le violazioni possono essere introdotte (1) da aggiornamenti (update) dell'attributo cui si fa riferimento oppure (2) da cancellazioni di tuple
- Reazioni previste:
 - `cascade`: propaga la modifica
 - `set null`: annulla l'attributo che fa riferimento
 - `set default`: assegna il valore di default all'attributo
 - `no action`: impedisce che la modifica possa avvenire
- La reazione può dipendere dall'evento; sintassi:
 on < delete | update >
 < cascade | set null | set default | no action >

26

Gestione degli orfani: cancellazione

Cosa succede degli esami se si cancella uno studente?

- `cascade`**
si cancellano anche gli esami dello studente
- `set null`**
si pone a null la matricola dei relativi esami
- `set default`**
si pone al valore di default la matricola dei relativi esami
- `no action`**
si impedisce la cancellazione dello studente

27

Gestione degli orfani: modifica

Cosa succede degli esami se si modifica la matricola di uno studente?

- `cascade`**
si modifica la matricola degli esami dello studente
- `set null`**
si pone a null la matricola dei relativi esami
- `set default`**
si pone al valore di default la matricola dei relativi esami
- `no action`**
si impedisce la modifica della matricola dello studente

28

Definizione: nella tabella figlio

```
create table Esame
( ....
  ....
  foreign key Matr
    references Studente
    on delete cascade
    on update cascade )
```

29

E' lecito essere figli di più padri

```
create table Esame
( ....
  primary key (Matr, CodCorso)
  foreign key Matr
    references Studente
    on delete cascade
    on update cascade
  foreign key CodCorso
    references Corso
    on delete no action
    on update no action )
```

30

Una istanza scorretta

Matr	Nome	Città	CDip
123			
415			
702			

Esame

Matr	Cod Corso	Data	Voto
123	1	7-9-97	30
123	2	8-1-98	28
123	2	1-8-97	28
702	2	7-9-97	20
702	1	NULL	NULL
714	1	7-9-97	28

viola la chiave

viola il NULL

viola la integrità referenziale

Una istanza corretta

Matr	Nome	Città	CDip
123			
415			
702			

Esame

Matr	Cod Corso	Data	Voto
123	1	7-9-97	30
123	2	8-1-98	28
702	2	7-9-97	20

32

Esempio: gestione ordini

Cliente

CODCLI	INDIRIZZO	PIVA

Ordine

CODORD	CODCLI	DATA	IMPORTO

Dettaglio

CODORD	CODPROD	QTA

Prodotto

CODPROD	NOME	PREZZO

33

Definizione della tabella Cliente

```
create table Cliente
( CodCli    char(6)  primary key,
  Indirizzo varchar(50),
  Piva      char(12) unique )
```

34

Definizione della tabella Ordine

```
create table Ordine
( CodOrd   char(6)  primary key,
  CodCli   char(6)  not null
                        default='999999',
  Data     date,
  Importo  integer,
  foreign key CodCli
      references Cliente
      on delete set default
      on update set default)
```

35

Definizione della tabella Dettaglio

```
create table Dettaglio
( CodOrd   char(6),
  CodProd  char(6),
  Qta      smallint,
  primary key(CodOrd,CodProd)
  foreign key CodOrd
      references Ordine
      on delete cascade
      on update cascade
  foreign key CodProd
      references Prodotto
      on delete no action
      on update no action)
```

36

Definizione della tabella Prodotto

```
create table Prodotto
( CodProd   char(6) primary key,
  Nome      varchar(20),
  Prezzo    smallint )
```

37

Modifiche degli schemi

- Necessarie per garantire l'evoluzione della base di dati a fronte di nuove esigenze
- Ci sono due comandi SQL appositi:
 - alter (alter domain ..., alter table ...)
modifica oggetti persistenti
 - drop
drop < schema | domain | table | view | assertion >
NomeComponente [restrict | cascade]
cancella oggetti dallo schema

38

Modifica degli oggetti DDL

- **alter**

- Si applica su domini e tabelle

```
es.: alter table Ordine
      add column NumFatt char(6)
```

```
es.: alter table Ordine
      alter column Importo
      add default 0
```

```
es.: alter table Ordine
      drop column Data
```

39

Cancellazione degli oggetti DDL

- **drop**

- si applica su domini, tabelle, indici, view, asserzioni, procedure, trigger

```
es.: drop table Ordine
```

```
es.: drop index DataIx
```

- Opzioni restrict e cascade

- **restrict**: impedisce drop se gli oggetti comprendono istanze
- **cascade**: applica drop agli oggetti collegati

40

Cataloghi relazionali

- Il catalogo contiene il dizionario dei dati (data dictionary), ovvero la descrizione della struttura dei dati contenuti nel database
- È basato su una struttura relazionale
 - Il modello relazionale è senz'altro noto agli utenti del sistema
 - Il sistema è in grado di gestire tabelle in modo efficiente
 - Ogni sistema rappresenta il catalogo tramite il proprio modello dei dati (es.: sistemi ad oggetti avranno un catalogo con schema a oggetti)
- Lo standard SQL-2 organizza il catalogo su due livelli
 - **Definition_Schema** (composto da tabelle, non vincolante)
 - **Information_Schema** (composto da viste, vincolante)

41

Information Schema

- Nell'Information_Schema compaiono viste come:
 - Domains
 - Domain_Constraints
 - Tables
 - Views
 - Columns
 -
- SQL-2 prevede 23 viste

42

La vista Columns

- Ad esempio, la vista Columns può avere uno schema con attributi:
 - Table_Name
 - Column_Name
 - Ordinal_Position
 - Column_Default
 - Is_Nullable(più molti altri)

43

Riflessività del catalogo

- Il catalogo è normalmente riflessivo (le strutture del catalogo sono descritte nel catalogo stesso)
- Ad esempio, un frammento di Columns:

```
Title:
columns.dvi
Creator:
dvips(k) 5.86 Copyright 1999 Radical Eye Software
Preview:
This EPS picture was not saved
with a preview included in it.
Comment
```

44

Catalogo da usare solo in lettura

- Ogni comando DDL viene quindi realizzato da opportuni comandi DML che operano sullo schema della base di dati
- Non per questo il DDL è inutile!
- Il DDL permette di descrivere gli oggetti dello schema in modo
 - affidabile
 - consistente
 - efficiente
 - portabile
- Il catalogo non deve MAI essere modificato direttamente

45

SQL come linguaggio di interrogazione

- Le interrogazioni SQL sono dichiarative
 - l'utente specifica **quale** informazione è di suo interesse, ma **non come** estrarla dai dati
- Le interrogazioni vengono tradotte dall'ottimizzatore (query optimizer) nel linguaggio procedurale interno al DBMS
- Il programmatore si focalizza sulla leggibilità, non sull'efficienza
- È l'aspetto più qualificante delle basi di dati relazionali

46

Interrogazioni SQL

- Le interrogazioni SQL hanno una struttura select-from-where
- Sintassi:

```
select AttrEspr [[ as ] Alias ] {, AttrEspr [[ as ] Alias ] }
from Tabella [[ as ] Alias ] {, Tabella [[ as ] Alias ] }
[ where Condizione ]
```
- Le tre parti della query sono chiamate:
 - clausola select / target list
 - clausola from
 - clausola where
- La query effettua il prodotto cartesiano delle tabelle nella clausola from, considera solo le righe che soddisfano la condizione nella clausola where e per ogni riga valuta le espressioni nella target list

47

Interpretazione algebrica delle query SQL

- La query generica:

```
select T_1.Attributo_11, ..., T_h.Attributo_hm
from Tabella_1 T_1, ..., Tabella_n T_n
where Condizione
```

- corrisponde all'interrogazione in algebra relazionale:

$$\pi_{T_1.Attributo_{11}, \dots, T_h.Attributo_{hm}} (\sigma_{Condizione}(Tabella_1 \times \dots \times Tabella_n))$$

48

Esempio: gestione degli esami universitari

Studente

MATR	NOME	CITTA'	CDIP
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

Esame

MATR	COD-CORSO	DATA	VOTO
123	1	7-9-97	30
123	2	8-1-98	28
702	2	7-9-97	20

Corso

COD-CORSO	TITOLO	DOCENTE
1	matematica	Barozzi
2	informatica	Meo

49

Interrogazione semplice

```
select *
from Studente
```

MATR	NOME	CITTA'	CDIP
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf
702	Antonio	Roma	Log

50

Interrogazione semplice

Studente	Matr	Nome	Città	CDip
----------	------	------	-------	------

```
select Nome
from Studente
where CDip = 'Log'
```

interpretazione algebrica
(a meno dei duplicati)

$\Pi_{\text{Nome}} \sigma_{\text{CDip}='Log'} \text{Studente}$

51

Sintassi nella clausola select

```
select *
select Nome, Città
select distinct Città
select Città as LuogoDiResidenza
select RedditoCatastale * 0.05
       as TassaIci
select sum(Salario)
```

52

Sintassi della clausola from

```
from Studente
from Studente as X
from Studente, Esame
from Studente join Esame
       on Studente.Matr=Esame.Matr
```

53

Sintassi della clausola where

- Espressione booleana di predicati semplici (come in algebra)
- Alcuni predicati aggiuntivi:
 - between:


```
Data between 1-1-90 and 31-12-99
```
 - like:


```
CDip like 'Lo%'
Targa like 'MI_777_8%'
```

54

Congiunzione di predicati

- Estrarre gli studenti di informatica originari di Bologna:

```
select *
from Studente
where CDip = 'Inf' and
      Città = 'Bologna'
```

- Risultato:

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf

55

Disgiunzione di predicati

- Estrarre gli studenti originari di Bologna o di Torino:

```
select *
from Studente
where Città = 'Bologna' or
      Città = 'Torino'
```

- Risultato:

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf
415	Paola	Torino	Inf

56

Espressioni booleane

- Estrarre gli studenti originari di Roma che frequentano il corso in Informatica o in Logistica:

```
select *
from Studente
where Città = 'Roma' and
      (CDip = 'Inf' or
       CDip = 'Log')
```

- Risultato:

Matr	Nome	Città	CDip
702	Antonio	Roma	Log

57

Operatore like

- Estrarre gli studenti con un nome che ha una 'a' in seconda posizione e finiscono per 'o':

```
select *
from Studente
where Nome like '_a%o'
```

- Risultato:

Matr	Nome	Città	CDip
123	Carlo	Bologna	Inf

58

Duplicati

- In algebra relazionale e nel calcolo, i risultati delle interrogazioni non contengono elementi duplicati
- In SQL, le tabelle prodotte dalle interrogazioni possono contenere più righe identiche tra loro
- I duplicati possono essere rimossi usando la parola chiave `distinct`

59

Duplicati

```
select
distinct CDip
from Studente
```

CDip
Inf
Log

```
select CDip
from Studente
```

CDip
Inf
Inf
Log

60

Gestione dei valori nulli

- I valori nulli rappresentano tre diverse situazioni:
 - un valore non è applicabile
 - un valore è applicabile ma sconosciuto
 - non si sa se il valore è applicabile o meno
- SQL-89 usa una logica a due valori
 - un confronto con *null* restituisce FALSE
- SQL-2 usa una logica a tre valori
 - un confronto con *null* restituisce UNKNOWN
- Per fare una verifica sui valori nulli:
Attributo is [not] null

61

Predicati e valori nulli

- logica a tre valori (V,F,U)
- $P = (Città \text{ is not null}) \text{ and } (CDip \text{ like 'Inf\%'})$

$V \text{ and } U = U$

$V \text{ or } U = V$

$F \text{ and } U = F$

$F \text{ or } U = U$

$U \text{ and } U = U$

$U \text{ or } U = U$

$\text{not } U = U$

Città	CDip	P	TUPLA SELEZ
Milano	Inf	V	si
Milano	NULL	U	no
NULL	Inf	F	no
Milano	Log	F	no

62

Interrogazioni sui valori nulli

```
select *
from Studente
where Città is [not] null
```

se Città ha valore *null*
 (Città = 'Milano') ha valore Unknown

63

Interrogazioni sui valori nulli

```
select *
from Studente
where Cdip = 'Inf' or
       Cdip <> 'Inf'
```

è equivalente a:

```
select *
from Studente
where Cdip is not null
```

64

Interrogazione semplice con due tabelle

Estrarre il nome degli studenti di "Logistica" che hanno preso almeno un 30

```
select Nome
from Studente, Esame
where Studente.Matr = Esame.Matr
      and CDip like 'Lo%' and Voto = 30
```

NOME
Carlo

65

Join in SQL-2

- SQL-2 ha introdotto una sintassi alternativa per i join, rappresentandoli esplicitamente nella clausola from:


```
select AttrEspr [[ as ] Alias ] {, AttrEspr [[ as ] Alias ] }
from Tabella [[ as ] Alias ]
{ [ TipoJoin ] join Tabella [[ as ] Alias ] on Condizioni }
[ where AltreCondizioni ]
```
- TipoJoin* può essere inner, right [outer], left [outer] oppure full [outer], consentendo la rappresentazione dei join esterni
- La parola chiave natural può precedere *TipoJoin* (però è implementato di rado)

66

Join di due tabelle in SQL-2

```
select Nome
from Studente, Esame
where Studente.Matr = Esame.Matr
and CDip like 'Lo%' and Voto = 30
```

```
select Nome
from Studente join Esame
on Studente.Matr = Esame.Matr
where CDip like 'Lo%' and Voto = 30
```

67

Database d'esempio: guidatori e automobili

DRIVER	FirstName	Surname	DriverID
	Mary	Brown	VR 2030020Y
	Charles	White	PZ 1012436B
	Marco	Neri	AP 4544442R

AUTOMOBILE	CarRegNo	Make	Model	DriverID
	ABC 123	BMW	323	VR 2030020Y
	DEF 456	BMW	Z3	VR 2030020Y
	GHI 789	Lancia	Delta	PZ 1012436B
	BBB 421	BMW	316	MI 2020030U

68

Left join

- Estrarre i guidatori con le loro macchine, includendo i guidatori senza macchine:

```
select FirstName, Surname, Driver.DriverID
CarRegNo, Make, Model
from Driver left join Automobile on
(Driver.DriverID=Automobile.DriverID)
```

- Risultato:

FirstName	Surname	DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	GHI 789	Lancia	Delta
Marco	Neri	AP 4544442R	NULL	NULL	NULL

69

Full join

- Estrarre tutti i guidatori e tutte le automobili, mostrando le possibili relazioni tra di loro:

```
select FirstName, Surname, Driver.DriverID
CarRegNo, Make, Model
from Driver full join Automobile on
(Driver.DriverID=Automobile.DriverID)
```

- Risultato:

FirstName	Surname	DriverID	CarRegNo	Make	Model
Mary	Brown	VR 2030020Y	ABC 123	BMW	323
Mary	Brown	VR 2030020Y	DEF 456	BMW	Z3
Charles	White	PZ 1012436B	GHI 789	Lancia	Delta
Marco	Neri	AP 4544442R	NULL	NULL	NULL
NULL	NULL	NULL	BBB 421	BMW	316

70

Interrogazione semplice con tre tabelle

- Estrarre il nome degli studenti di "Matematica" che hanno preso almeno un 30

```
select Nome
from Studente, Esame, Corso
where Studente.Matr = Esame.Matr
and Corso.CodCorso = Esame.CodCorso
and Titolo like 'Mat%' and Voto = 30
```

$\Pi_{Nome} \sigma_{(Titolo \text{ like 'Mat\%'}) \wedge (Voto=30)} (Studente \bowtie Esame \bowtie Corso)$

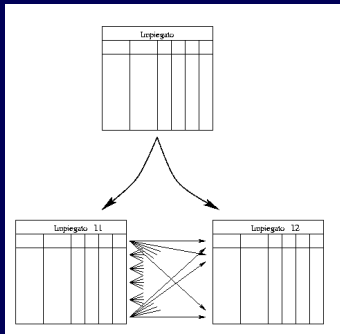
71

Variabili in SQL

- Gli alias di tabella possono essere interpretati come variabili, con valore che rappresenta la generica tupla di una tabella
- L'uso delle variabili corrisponde all'operatore di ridenominazione ρ dell'algebra relazionale

72

Variabili in SQL



73

Interrogazione semplice con variabili relazionali

Chi sono i dipendenti di Giorgio?

Impiegato

Matr	Nome	DataAss	Salario	MatrMgr
1	Piero	1-1-95	3 M	2
2	Giorgio	1-1-97	2,5 M	null
3	Giovanni	1-7-96	2 M	2

74

Chi sono i dipendenti di Giorgio?

```
select I1.Nome, I1.MatrMgr, I2.Matr, I2.N
from Impiegato as I1, Impiegato as I2
where I1.MatrMgr = I2.Matr
and I2.Nome = 'Giorgio'
```

I1.Nome	I1.MatrMgr	I2.Matr	I2.Nome
Piero	2	2	Giorgio
Giovanni	2	2	Giorgio

75

Classificazione delle interrogazioni complesse

- Query con ordinamento
- Query con aggregazioni
- Query con raggruppamento
- Query binarie
- Query nidificate

76

Esempio: gestione ordini

Cliente

<u>CODCLI</u>	INDIRIZZO	P-IVA	NOME

Ordine

<u>CODORD</u>	CODCLI	DATA	IMPORTO

Dettaglio

<u>CODORD</u>	<u>CODPROD</u>	QTA

Prodotto

<u>CODPROD</u>	NOME	PREZZO

77

Istanza di ordine

Ordine

CODORD	CODCLI	DATA	IMPORTO
1	3	1-6-97	50.000.000
2	4	3-8-97	8.000.000
3	3	1-9-97	5.500.000
4	1	1-7-97	12.000.000
5	1	1-8-97	1.500.000
6	3	3-9-97	27.000.000

78

Ordinamento

- La clausola `order by`, che compare in coda all'interrogazione, ordina le righe del risultato
- Sintassi:

```
order by AttributoOrdinamento [ asc | desc ]  
        {, AttributoOrdinamento [ asc | desc ] }
```
- Le condizioni di ordinamento vengono valutate in ordine
 - a pari valore del primo attributo, si considera l'ordinamento sul secondo, e così via

79

Query con ordinamento

```
select *  
from Ordine  
where Importo > 100.000  
order by Data
```

CODORD	CODCLI	DATA	IMPORTO
1	3	1-6-97	50.000.000
4	1	1-7-97	12.000.000
5	1	1-8-97	1.500.000
2	4	3-8-97	8.000.000
3	3	1-9-97	1.500.000
6	3	3-9-97	5.500.000

80

order by CodCli

CODORD	CODCLI	DATA	IMPORTO
4	1	1-7-97	12.000.000
5	1	1-8-97	1.500.000
1	3	1-6-97	50.000.000
6	3	3-9-97	5.500.000
3	3	1-9-97	1.500.000
2	4	3-8-97	27.000.000

81

order by CodCli asc, Data desc

CODORD	CODCLI	DATA	IMPORTO
5	1	1-8-97	1.500.000
4	1	1-7-97	12.000.000
6	3	3-9-97	5.500.000
3	3	1-9-97	1.500.000
1	3	1-6-97	50.000.000
2	4	3-8-97	27.000.000

82

Funzioni aggregate

- Le interrogazioni con funzioni aggregate non possono essere rappresentate in algebra relazionale
- Il risultato di una query con funzioni aggregate dipende dalla valutazione del contenuto di un insieme di righe
- SQL-2 offre cinque operatori aggregati:
 - `count` cardinalità
 - `sum` sommatoria
 - `max` massimo
 - `min` minimo
 - `avg` media

83

Operatore count

- `count` restituisce il numero di righe o valori distinti; sintassi:

```
count(< * | [ distinct | all ] ListaAttributi >)
```
- Estrarre il numero di ordini:

```
select count(*)  
from Ordine
```
- Estrarre il numero di valori distinti dell'attributo `CodCli` per tutte le righe di Ordine:

```
select count(distinct CodCli)  
from Ordine
```
- Estrarre il numero di righe di Ordine che posseggono un valore non nullo per l'attributo `CodCli`:

```
select count(all CodCli)  
from Ordine
```

84

sum, max, min, avg

- Sintassi:
`<sum|max|min|avg>([distinct|all] AttrEspr)`
- L'opzione `distinct` considera una sola volta ciascun valore
 - utile solo per le funzioni `sum` e `avg`
- L'opzione `all` considera tutti i valori diversi da `null`

85

Query con massimo

- Estrarre l'importo massimo degli ordini

```
select max(Importo) as MaxImp
from Ordine
```

MaxImp
50.000.000

86

Query con sommatoria

- Estrarre la somma degli importi degli ordini relativi al cliente numero 1

```
select sum(Importo) as SommaImp
from Ordine
where CodCliente = 1
```

SommaImp
13.500.000

87

Funzioni aggregate con join

- Estrarre l'ordine massimo tra quelli contenenti il prodotto con codice 'ABC':

```
select max(Importo) as MaxImportoABC
from Ordine, Dettaglio
where Ordine.CodOrd = Dettaglio.CodOrd and
      CodProd = 'ABC'
```

88

Funzioni aggregate e target list

- Query scorretta:

```
select Data, max(Importo)
from Ordine, Dettaglio
where Ordine.CodOrd = Dettaglio.CodOrd and
      CodProd = 'ABC'
```
- La data di quale ordine? La target list deve essere omogenea
- Estrarre il massimo e il minimo importo degli ordini:

```
select max(Importo) as MaxImp,
       min(Importo) as MinImp
from Ordine
```

89

Funzioni aggregate e target list

- Estrarre il massimo e il minimo importo degli ordini:

```
select max(Importo) as MaxImp,
       min(Importo) as MinImp
from Ordine
```

MaxImp	MinImp
50.000.000	1.500.000

90

Query con raggruppamento

- Nelle interrogazioni si possono applicare gli operatori aggregati a sottoinsiemi di righe
- Si aggiungono le clausole
 - `group by` (raggruppamento)
 - `having` (selezione dei gruppi)

```
select ...
from ...
where ...
group by ...
having ...
```

91

Query con raggruppamento

- Estrarre la somma degli importi degli ordini successivi al 10-6-97 per quei clienti che hanno emesso almeno 2 ordini

```
select CodCli, sum(Importo)
from Ordine
where Data > 10-6-97
group by CodCli
having count(*) >= 2
```

92

Passo 1: Valutazione where

CodOrd	CodCli	Data	Importo
2	4	3-8-97	8.000.000
3	3	1-9-97	5.500.000
4	1	1-7-97	12.000.000
5	1	1-8-97	1.500.000
6	3	3-9-97	27.000.000

93

Passo 2 : Raggruppamento

- si valuta la clausola `group by`

CodOrd	CodCli	Data	Importo
4	1	1-7-97	12.000.000
5	1	1-8-97	1.500.000
3	3	1-9-97	5.500.000
6	3	3-9-97	27.000.000
2	4	3-8-97	8.000.000

94

Passo 3 : Calcolo degli aggregati

- si calcolano `sum(Importo)` e `count(*)` per ciascun gruppo

CodCli	sum (Importo)	count (*)
1	13.500.000	2
3	32.500.000	2
4	8.000.000	1

95

Passo 4 : Estrazione dei gruppi

- si valuta il predicato `count(*) >= 2`

CodCli	sum (Importo)	count (*)
1	13.500.000	2
3	32.500.000	2
4	8.000.000	1

96

Passo 5 : Produzione del risultato

CodCli	sum (Importo)
1	13.500.000
3	32.500.000

97

Query con group by e target list

- Query scorretta:

```
select Importo
from Ordine
group by CodCli
```
- Query scorretta:

```
select O.CodCli, count(*), C.Città
from Ordine O join Cliente C
on (O.CodCli = C.CodCli)
group by O.CodCli
```
- Query corretta:

```
select O.CodCli, count(*), C.Città
from Ordine O join Cliente C
on (O.CodCli = C.CodCli)
group by O.CodCli, C.Città
```

98

where o having?

- Soltanto i predicati che richiedono la valutazione di funzioni aggregate dovrebbero comparire nell'argomento della clausola having
- Estrarre i dipartimenti in cui lo stipendio medio degli impiegati che lavorano nell'ufficio 20 è maggiore di 25:

```
select Dipart
from Impiegato
where Ufficio = '20'
group by Dipart
having avg(Stipendio) > 25
```

99

Query con raggruppamento e ordinamento

- È possibile ordinare il risultato delle query con raggruppamento

```
select ....
from ....
[ where .... ]
group by ....
[ having .... ]
order by ....
```

100

Raggruppamento e ordinamento

- Estrarre la somma degli importi degli ordini successivi al 10-6-97 per quei clienti che hanno emesso almeno 2 ordini, in ordine decrescente di somma di importo

```
select CodCli, sum(Importo)
from Ordine
where Data > 10-6-97
group by CodCli
having count(*) >= 2
order by sum(Importo) desc
```

101

Risultato dopo la clausola di ordinamento

CodCli	sum (Importo)
3	32.500.000
1	13.500.000

102

Doppio raggruppamento

- Estrarre la somma delle quantità dei dettagli degli ordini emessi da ciascun cliente per ciascun prodotto, purché la somma superi 50

```
select CodCli, CodProd, sum(Qta)
from Ordine as O, Dettaglio as D
where O.CodOrd = D.CodOrd
group by CodCli, CodProd
having sum(Qta) > 50
```

103

Situazione dopo il join e il raggruppamento

Ordine		Dettaglio			
CodCli	Ordine. CodOrd	Dettaglio. CodOrd	CodProd	Qta	
1	3	3	1	30	gruppo 1,1
1	4	4	1	20	
1	3	3	2	30	gruppo 1,2
1	5	5	2	10	
2	7	7	1	60	gruppo 2,1
3	1	1	1	40	gruppo 3,1
3	2	2	1	30	
3	6	6	1	25	

104

Estrazione del risultato

- si valuta la funzione aggregata **sum(Qta)** e il predicato **having**

CodCli	CodProd	sum(Qta)
1	1	50
1	2	40
2	1	60
3	1	95

105

Query binarie (set queries)

- Costruite concatenando due query SQL tramite operatori insiemistici
- Sintassi:
SelectSQL { < **union** | **intersect** | **except** > [**all**] *SelectSQL* }
- union** unione
- intersect** intersezione
- except (minus)** differenza
- Si eliminano i duplicati, a meno che non venga usata l'opzione **all**

106

Unione

- Estrarre i codici degli ordini i cui importi superano 500 euro oppure in cui qualche prodotto è presente con quantità superiore a 1000

```
select CodOrd
from Ordine
where Importo > 500
union
select CodOrd
from Dettaglio
where Qta > 1000
```

107

Nome degli attributi nel risultato

```
select Padre
from Paternita
union
select Madre
from Maternita
```

- Quali nomi per gli attributi del risultato?
 - Nessuno
 - Quelli del primo operando
 - ...

108

Notazione posizionale

```
select Padre, Figlio
from Paternita
union
select Figlio, Madre
from Maternita
```

```
select Padre, Figlio
from Paternita
union
select Madre, Figlio
from Maternita
```

- Sono interrogazioni diverse; esempio:

PADRE	FIGLIO
Luigi	Giorgio
Stefano	Giovanni

MADRE	FIGLIO
Anna	Giorgio
Paola	Giovanni

109

Notazione posizionale

```
select Padre, Figlio
from Paternita
union
select Figlio, Madre
from Maternita
```

```
select Padre, Figlio
from Paternita
union
select Madre, Figlio
from Maternita
```

Luigi	Giorgio
Stefano	Giovanni
Giorgio	Anna
Giovanni	Paola

Luigi	Giorgio
Stefano	Giovanni
Anna	Giorgio
Paola	Giovanni

110

Uso della parola chiave **all**

- Estrarre i padri di persone con nome "Giorgio" o "Giovanni", presentando due volte i padri che hanno due figli con ciascuno dei nomi

```
select Padre
from Paternita
where Figlio = 'Giorgio'
union all
select Padre
from Paternita
where Figlio = 'Giovanni'
```

111

Differenza

- Estrarre i codici degli ordini i cui importi superano 500 euro ma in cui nessun prodotto è presente con quantità superiore a 1000

```
select CodOrd
from Ordine
where Importo > 500
except
select CodOrd
from Dettaglio
where Qta > 1000
```

- Può essere rappresentata usando una query nidificata

112

Differenza

- Estrarre i codici degli ordini che presentano $n \geq 1$ linee d'ordine con quantità maggiore di 10 e non presentano un numero $m \geq n$ di linee d'ordine con quantità superiore a 1000

```
select CodOrd
from Dettaglio
where Qta > 10
except all
select CodOrd
from Dettaglio
where Qta > 1000
```

113

Intersezione

- Estrarre i codici degli ordini i cui importi superano 500 euro e in cui qualche prodotto è presente con quantità superiore a 1000

```
select CodOrd
from Ordine
where Importo > 500
intersect
select CodOrd
from Dettaglio
where Qta > 1000
```

- Anche in questo caso la query può essere rappresentata usando una query nidificata

114

Query nidificate

- Nella clausola **where** possono comparire predicati che:
 - confrontano un attributo (o un'espressione sugli attributi) con il risultato di una query SQL; sintassi:

ScalarValue Operator < **any** | **all** > *SelectSQL*

- any**: il predicato è vero se almeno una riga restituita dalla query *SelectSQL* soddisfa il confronto
- all**: il predicato è vero se tutte le righe restituite dalla query *SelectSQL* soddisfano il confronto
- Operator**: uno qualsiasi tra =, <, <=, >, >=
- La query che appare nella clausola **where** è chiamata query nidificata

115

Query nidificate semplici

- Estrarre gli ordini di prodotti con un prezzo superiore a 100

```
select CodOrd
from Dettaglio
where CodProd = any (select CodProd
                     from Prodotto
                     where Prezzo > 100)
```

- Equivalente a (senza query nidificata, a meno di duplicati)

```
select CodOrd
from Dettaglio D, Prodotto P
where D.CodProd = P.CodProd
and Prezzo > 100
```

116

Query nidificate semplici

- Estrarre i prodotti ordinati assieme al prodotto avente codice 'ABC'

– senza query nidificate:

```
select distinct D1.CodProd
from Dettaglio D1, Dettaglio D2
where D1.CodOrd = D2.CodOrd and
      D2.CodProd = 'ABC'
```

– con una query nidificata:

```
select distinct CodProd
from Dettaglio
where CodOrd = any
      (select CodOrd
       from Dettaglio
       where CodProd = 'ABC')
```

117

Negazione con query nidificate

- Estrarre gli ordini che non contengono il prodotto 'ABC':

```
select CodOrd
from Ordine
where CodOrd <> all (select CodOrd
                   from Dettaglio
                   where CodProd = 'ABC')
```

- In alternativa:

```
select CodOrd
from Ordine
except
select CodOrd
from Dettaglio
where CodProd = 'ABC'
```

118

Operatori in e not in

- L'operatore **in** è equivalente a = any

```
select distinct CodProd
from Dettaglio
where CodOrd in
      (select CodOrd
       from Dettaglio
       where CodProd = 'ABC')
```

- L'operatore **not in** è equivalente a <> all

```
select CodOrd
from Ordine
where CodOrd not in (select CodOrd
                    from Dettaglio
                    where CodProd = 'ABC')
```

119

Query nidificate

- Estrarre nome e indirizzo dei clienti che hanno emesso qualche ordine di importo superiore a 10.000

```
select Nome, Indirizzo
from Cliente
where CodCli in
      (select CodCli
       from Ordine
       where Importo > 10000)
```

120

Query nidificate a più livelli

- Estrarre nome e indirizzo dei clienti che hanno emesso qualche ordine che comprende il prodotto "Pneumatico"

```
select Nome, Indirizzo
from Cliente
where CodCli in
  select CodCli
  from Ordine
  where CodOrd in
    select CodOrd
    from Dettaglio
    where CodProd in
      select CodProd
      from Prodotto
      where Nome = 'Pneumatico'
```

121

La query equivalente

- La query precedente equivale (a meno di duplicati) a:

```
select C.Nome, Indirizzo
from Cliente as C, Ordine as O,
  Dettaglio as D, Prodotto as P
where C.CodCli = O.CodCli
  and O.CodOrd = D.CodOrd
  and D.CodProd = P.CodProd
  and P.Nome = 'Pneumatico'
```

122

max e min con query nidificate

- Gli operatori aggregati max e min possono essere espressi tramite query nidificate
- Estrarre l'ordine con il massimo importo

– usando max:

```
select CodOrd
from Ordine
where Importo in (select max(Importo)
  from Ordine)
```

– con una query nidificata:

```
select CodOrd
from Ordine
where Importo >= all (select Importo
  from Ordine)
```

123

Uso di any e all

```
select CodOrd
from Ordine
where Importo > any
  select Importo
  from Ordine
```

```
select CodOrd
from Ordine
where Importo >= all
  select Importo
  from Ordine
```

COD-ORD	IMPORTO	ANY	ALL
1	50	F	F
2	300	V	V
3	90	V	F

124

L'operatore exists

- Si può usare il quantificatore esistenziale sul risultato di una query SQL
- Sintassi:

exists *SelectStar*

- il predicato è vero se la query *SelectStar* restituisce un risultato non nullo (sempre **select *** perché è irrilevante la proiezione)

125

Query nidificate complesse

- La query nidificata può usare variabili della query esterna
 - Interpretazione: la query nidificata viene valutata per ogni tupla della query esterna
- Estrarre tutti i clienti che hanno emesso più di un ordine nella stessa giornata:

```
select distinct CodCli
from Ordine O
where exists (select *
  from Ordine O1
  where O1.CodCli = O.CodCli
    and O1.Data = O.Data
    and O1.CodOrd <> O.CodOrd)
```

126

Query nidificate complesse

- Estrarre tutte le persone che [non] hanno degli omonimi:

```
select *
from Persona P
where [not] exists
    (select *
     from Persona P1
     where P1.Nome = P.Nome
        and P1.Cognome = P.Cognome
        and P1.CodFisc <> P.CodFisc)
```

127

Costruttore di tupla

- Il confronto con la query nidificata può coinvolgere più di un attributo
- Gli attributi devono essere racchiusi da un paio di parentesi tonde (costruttore di tupla)
- La query precedente può essere espressa così:

```
select *
from Persona P
where (Nome,Cognome) [not] in
    (select Nome, Cognome
     from Persona P1
     where P1.CodFisc <> P.CodFisc)
```

128

Commenti sulle query nidificate

- L'uso di query nidificate può produrre query 'meno dichiarative', ma spesso si migliora la leggibilità
- La prima versione di SQL prevedeva solo la forma nidificata (o strutturata) con una sola relazione nella clausola **from**, il che è insoddisfacente
- Le sottointerrogazioni non possono contenere operatori insiemistici ("l'unione si fa solo al livello esterno"); la limitazione non è significativa, ed è superata da alcuni sistemi

129

Commenti sulle query nidificate

- Query complesse, che fanno uso di variabili, possono diventare molto difficili da comprendere
- L'uso delle variabili deve rispettare le regole di visibilità
 - una variabile può essere usata solamente all'interno della query dove viene definita o all'interno di una query che è ricorsivamente nidificata nella query dove è definita
 - se un nome di variabile è omissso, si assume il riferimento alla variabile più vicina

130

Visibilità delle variabili

- Query scorretta:

```
select *
from Cliente
where CodCli in
    (select CodCli
     from Ordine O1
     where CodOrd = 'AZ1020')
or CodCli in
    (select CodCli
     from Ordine O2
     where O2.Data = O1.Data)
```
- La query è scorretta poiché la variabile O1 non è visibile nella seconda query nidificata

131

Comandi di modifica in SQL

- Istruzioni per
 - inserimento (**insert**)
 - cancellazione (**delete**)
 - modifica dei valori degli attributi (**update**)
- Tutte le istruzioni possono operare su un insieme di tuple (set-oriented)
- Il comando può contenere una condizione, nella quale è possibile fare accesso a tabelle esterne

132

Inserimento

- Sintassi:

```
insert into NomeTabella [ (ListaAttributi) ]  
  <values (ListaDiValori) | SelectSQL>
```
- Usando **values**:

```
insert into Studente  
  values ('456878', 'Giorgio Rossi',  
         'Bologna', 'Logistica')
```
- Usando una query:

```
insert into Bolognesi  
  (select *  
   from Studente  
   where Città = 'Bologna')
```

133

Inserimento

- L'ordine degli attributi e dei valori è significativo (notazione posizionale, il primo valore viene associato al primo attributo, e così via)
- Se la *ListaAttributi* viene omessa, si considerano tutti gli attributi della relazione, nell'ordine in cui compaiono nella definizione della tabella
- Se la *ListaAttributi* non contiene tutti gli attributi della relazione, agli attributi rimanenti viene assegnato il valore di default (se definito, altrimenti il valore *null*)

134

Inserimento

- Usando **values** con *ListaAttributi*:

```
insert into Studente (Matr, Nome, Città, CDip)  
  values ('456878', 'Giorgio Rossi',  
         'Bologna', 'Logistica')
```
- Usando una query con *ListaAttributi*:

```
insert into Bolognesi (Matr, Nome, Città, CDip)  
  (select Matr, Nome, Città, CDip  
   from Studente  
   where Città = 'Bologna')
```

135

Cancellazioni

- Sintassi:

```
delete from NomeTabella [ where Condizione ]
```
- Cancellare lo studente con matricola 678678:

```
delete from Studente  
  where Matr = '678678'
```
- Cancellare gli studenti che non hanno sostenuto esami:

```
delete from Studente  
  where Matr not in (select Matr  
                    from Esame)
```

136

Cancellazioni

- L'istruzione **delete** cancella dalla tabella tutte le tuple che soddisfano la condizione
- Il comando può provocare delle cancellazioni in altre tabelle, se è presente un vincolo d'integrità referenziale con politica **cascade**
- Se si omette la clausola **where**, il comando **delete** cancella tutte le tuple
- Per cancellare tutte le tuple da STUDENTE (mantenendo lo schema della tabella):

```
delete from Studente
```
- Per cancellare completamente la tabella STUDENTE (contenuto e schema):

```
drop table Studente cascade
```

137

Modifiche

- Sintassi:

```
update NomeTabella  
  set Attributo = <Espressione | SelectSQL | null | default >  
  { , Attributo = <Espressione | SelectSQL | null | default > }  
  [ where Condizione ]
```
- Esempi:

```
update Esame  
  set Voto = 30  
  where Data = 1-4-03  
  
update Esame  
  set Voto = Voto + 1  
  where Matr = '787989'
```

138

Modifiche

- Poiché il linguaggio è set-oriented, è molto importante l'ordine dei comandi

```
update Impiegato
  set Stipendio = Stipendio * 1.1
  where Stipendio <= 30
update Impiegato
  set Stipendio = Stipendio * 1.15
  where Stipendio > 30
```

- Se i comandi sono scritti in questo ordine, alcuni impiegati possono ottenere un aumento doppio

139

Uso di in nelle modifiche

- Aumentare di 5 euro l'importo di tutti gli ordini che comprendono il prodotto 456

```
update Ordine
  set Importo = Importo + 5
  where CodOrd in
    select CodOrd
    from Dettaglio
    where CodProd = '456'
```

140

Uso di query nidificate nelle modifiche

- Assegnare a TotPezzi la somma delle quantità delle linee di un ordine

```
update Ordine O
  set TotPezzi =
    (select sum(Qta)
     from Dettaglio D
     where D.CodOrd = O.CodOrd)
```

141

Viste

- Offrono la "visione" di tabelle virtuali (schemi esterni)
- Classificate in:
 - semplici (selezione e proiezione su una sola tabella)
 - complesse
- Sintassi:

```
create view NomeVista [ (ListaAttributi) ] as SelectSQL
[ with [ local | cascaded ] check option ]
```

142

Esempio di vista semplice

- Ordini di importo superiore a 10.000

```
create view OrdiniPrincipali as
  select *
  from Ordine
  where Importo > 10000
```

Ordine

1	3	1-6-96	50.000
4	1	1-7-97	12.000
6	3	3-9-97	27.000

VISTA :
ordini principali

143

Viste semplici in cascata

```
create view ImpiegatoAmmin
  (Matr, Nome, Cognome, Stipendio) as
  select Matr, Nome, Cognome, Stipendio
  from Impiegato
  where Dipart = 'Amministrazione'
  and Stipendio > 10

create view ImpiegatoAmminJunior as
  select *
  from ImpiegatoAmmin
  where Stipendio < 50
  with check option
```

144

Viste

- Le viste in SQL-2 possono contenere nella definizione altre viste precedentemente definite, ma non vi può essere mutua dipendenza (la ricorsione è stata introdotta in SQL:1999)
- Le viste possono essere usate per formulare query complesse
 - Le viste decompongono il problema e producono una soluzione più leggibile
- Le viste sono talvolta necessarie per esprimere alcune query:
 - query che combinano e nidificano diversi operatori aggregati
 - query che fanno un uso sofisticato dell'operatore di unione

145

Ricorsione in SQL:1999

```
with recursive Raggiungibile (Orig, Dest, Costo) as
( select Orig, Dest, Costo
  from Volo
  union
  select R.Orig, V.Dest, R.Costo+V.Costo
  from Raggiungibile R join Volo V
    on R.Dest = V.Orig
  where (R.Orig, V.Dest) not in
    (select R1.Orig, R1.Dest from Raggiungibile R1
     where R1.Costo < R.Costo+V.Costo)

select distinct Dest, Costo
from Raggiungibile R
where Orig = 'Milano'
```

146

Viste e query

- Estrarre il cliente che ha generato il massimo fatturato (senza usare le viste):

```
select CodCli
from Ordine
group by CodCli
having sum(Importo) >= all
      (select sum(Importo)
       from Ordine
       group by CodCli)
```

- Questa soluzione può non essere riconosciuta da tutti i sistemi SQL

147

Viste e query

- Estrarre il cliente che ha generato il massimo fatturato (usando le viste):

```
create view CliFatt(CodCli, FattTotale) as
select CodCli, sum(Importo)
from Ordine
group by CodCli

select CodCli
from CliFatt
where FattTotale in (select max(FattTotale)
                    from CliFatt)
```

148

Viste e query

- Estrarre il numero medio di ordini per cliente:
 - Soluzione scorretta (SQL non permette di applicare gli operatori aggregati in cascata):

```
select avg(count(*))
from Ordine
group by CodCli
```

- Soluzione corretta (usando una vista):

```
create view CliOrd(CodCli, NumOrdini) as
select CodCli, count(*)
from Ordine
group by CodCli
```

```
select avg(NumOrdini)
from CliOrd
```

149

Viste e query

- Estrarre il numero medio di ordini per cliente:
 - Utilizzando query nidificate nella clausola from:

```
select avg(NumOrdini)
from (select count(*) as NumOrdini
      from Ordine
      group by CodCli)
```

150

Uso della vista per query

- Vista:

```
create view OrdiniPrincipali as
select *
from Ordine
where Importo > 10000
```
- Query:

```
select CodCli
from OrdiniPrincipali
```
- Composizione della vista con la query:

```
select CodCli
from Ordine
where Importo > 10000
```

151

Modifiche tramite le viste

- Vista:

```
create view OrdiniPrincipali as
select *
from Ordine
where Importo > 10000
```
- Modifica:

```
update OrdiniPrincipali
set Importo = Importo * 1.05
where CodCli = '45'
```
- Composizione della vista con la modifica:

```
update Ordine
set Importo = Importo * 1.05
where CodCli = '45'
and Importo > 10000
```

152

Check option

- La **check option** interviene quando viene aggiornato il contenuto di una vista, per verificare che la tupla inserita/modificata appartenga alla vista
- Se l'opzione è **local**, il controllo viene fatto solo rispetto alla vista su cui viene invocato il comando
- Se l'opzione è **cascaded**, il controllo viene fatto su tutte le viste coinvolte
- Es.:

```
create view OrdiniPrinc70 as
select *
from OrdiniPrincipali
where CodCli = '70'
with local check option
```

153

Check option

- ```
update OrdiniPrinc70
set CodCli = '71'
where CodOrd = '754'
```

  
viene rifiutato con check option **local** e **cascaded**
- ```
update OrdiniPrinc70
set Importo = 5000
where CodOrd = '754'
```


viene accettato dalla **local**, rifiutato dalla **cascaded**

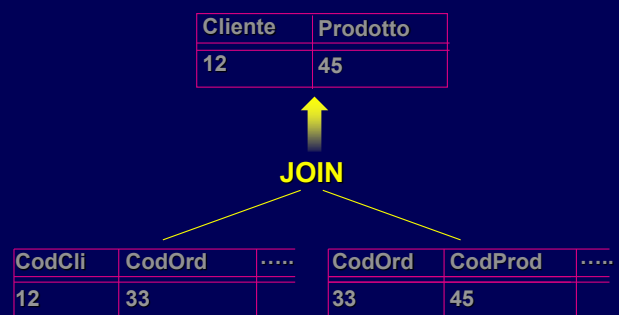
154

Esempio di vista complessa

```
create view CliPro(Cliente,Prodotto) as
select CodCli, CodProd
from Ordine join Dettaglio
on Ordine.CodOrd = Dettaglio.CodOrd
```

155

Vista complessa (JOIN)



156

Interrogazione sulla vista complessa

- Query:

```
select Cliente
from CliProd
where Prodotto = '45'
```
- Composizione della vista con la query:

```
select CodCli
from Ordine join Dettaglio
on Ordine.CodOrd = Dettaglio.CodOrd
where CodProd = '45'
```

157

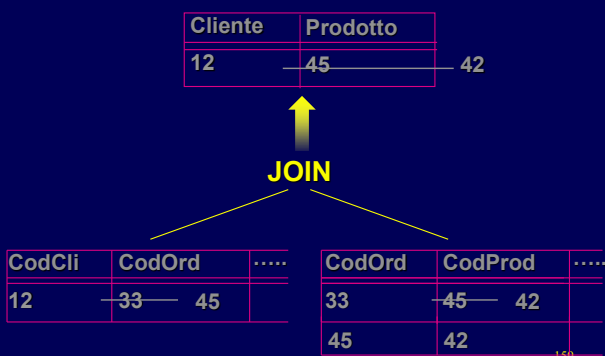
Modifiche sulla vista complessa

- Non è possibile modificare le tabelle di base tramite la vista perché la interpretazione è ambigua
- Es.:

```
update CliProd
set Prodotto = '42'
where Cliente = '12'
```
- Due alternative per la realizzazione sulle tabelle di base
 - il cliente ha cambiato l'ordine
 - il codice del prodotto è cambiato

158

Vista complessa (JOIN)



159

Aspetti evoluti del DDL

- Creazione di indici
- Autorizzazioni d'accesso
- Vincoli di integrità
- Procedure e regole attive

Creazione di indici

- Indici: meccanismi di accesso efficiente ai dati

```
create index
es.: create index DataIx
on Ordine (Data)

create unique index
es.: create unique index OrdKey
on Ordine (CodOrd)
```

161

Qualità dei dati

- Qualità dei dati:
 - correttezza, completezza, attualità
- In molte applicazioni reali i dati sono di scarsa qualità (5% - 40% di dati scorretti)
- Per aumentare la qualità dei dati:
 - Regole di integrità
 - Manipolazione dei dati tramite programmi predefiniti (procedure e trigger)

162

Vincoli di integrità generici

- Predicati che devono essere veri se valutati su istanze corrette (legali) della base di dati
- Espressi in due modi:
 - negli schemi delle tabelle
 - come asserzioni separate

163

Vincoli d'integrità generici

- La clausola **check** può essere usata per esprimere vincoli arbitrari nella definizione dello schema
- Sintassi:
 - check** (*Condizione*)
- *Condizione* è ciò che può apparire in una clausola *where* (comprese le query nidificate)
- Es., la definizione di un attributo *Superiore* nello schema della tabella IMPIEGATO:

```
Superiore character(6)
check (Matr like "1%" or
      Dipart in (select Dipart
                  from Impiegato I
                  where I.Matr = Superiore))
```

164

Esempio: gestione magazzino

Magazzino

CodProd	QtaDisp	QtaRiord
1	150	100
3	130	80
4	170	50
5	500	150

Riordino

CodProd	Data	QtaOrd

165

Esempio: definizione di Magazzino

```
create table Magazzino as
( CodProd    char(2) primary key,
  QtaDisp    integer not null
              check(QtaDisp >= 0),
  QtaRiord    integer not null
              check(QtaRiord > 10))
```

166

Asserzioni

- Le asserzioni permettono la definizione di vincoli al di fuori della definizione delle tabelle
- Utili in molte situazioni (es., per esprimere vincoli inter-relazionali di tipo generico)
- Una asserzione associa un nome a una clausola *check*; sintassi:


```
create assertion NomeAsserzione check (Condizione)
```
- Es., la tabella IMPIEGATO deve contenere almeno una tupla:

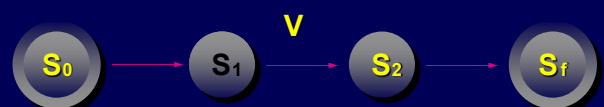

```
create assertion SempreUnImpiegato
  check (1 <= (select count(*)
                from Impiegato))
```

167

Significato dei vincoli

La verifica dei vincoli può essere:

- immediate** (immediata):
la loro violazione annulla l'ultima modifica
- deferred** (differita):
la loro violazione annulla l'intera applicazione



Modifica dinamica del significato dei vincoli

- Ogni vincolo è definito di un tipo (normalmente "immediate")
- L'applicazione può modificare il tipo iniziale dei vincoli:
 - **set constraints immediate**
 - **set constraints deferred**
- Tutti i vincoli vengono comunque verificati, prima o poi

169

Controllo dell'accesso

- Confidenzialità: protezione selettiva della base di dati in modo da garantire l'accesso solo agli utenti autorizzati
- Meccanismi per identificare l'utente (tramite *parola chiave* o *password*):
 - Quando si collega al sistema informatico
 - Quando accede al DBMS
- Utenti individuali e gruppi di utenti

170

Autorizzazioni

- Ogni componente dello schema può essere protetto (tabelle, attributi, viste, domini, etc.)
- Il proprietario di una risorsa (il creatore) assegna privilegi (autorizzazioni) agli altri utenti
- Un utente predefinito **_system** rappresenta l'amministratore di sistema e ha pieno accesso a tutte le risorse
- Un privilegio è caratterizzato da:
 - la risorsa
 - l'utente che concede il privilegio
 - l'utente che riceve il privilegio
 - l'azione che viene consentita sulla risorsa
 - la possibilità di passare il privilegio ad altri utenti

171

Tipi di privilegi

- SQL offre 6 tipi di privilegi
 - **insert**: per inserire un nuovo oggetto nella risorsa
 - **update**: per modificare il contenuto della risorsa
 - **delete**: per rimuovere un oggetto dalla risorsa
 - **select**: per accedere al contenuto della risorsa in una query
 - **references**: per costruire un vincolo di integrità referenziale che coinvolge la risorsa (può limitare la modificabilità della risorsa)
 - **usage**: per usare la risorsa in una definizione di schema (es., un dominio)
- **all privileges** li riassume tutti

172

grant e revoke

- Per concedere un privilegio a un utente:
grant <Privilegi | **all privileges** > **on** Risorsa **to** Utenti [**with grant option**]
 - **grant option** specifica se deve essere garantita la possibilità di propagare il privilegio ad altri utenti
- Per revocare un privilegio:
revoke Privilegi **on** Risorsa **from** Utenti [**restrict** | **cascade**]

173

Esempi

```
grant all privileges on Ordine to User1
grant update(Importo) on Ordine to User2
grant select on Ordine to User2, User3
```

```
revoke update on Ordine from User1
revoke select on Ordine from User3
```

174

Esempio di uso, grant option

- 1 Database administrator
`grant all privileges on Ordine to User1
with grant option`
- 2 User1
`grant select on Ordine to User2
with grant option`
- 3 User2
`grant select on Ordine to User3`

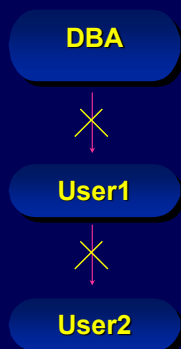
175

Revoca di un privilegio con cascata

- 1 Database administrator
`grant select on Ordine to User1
with grant option`
- 2 User1
`grant select on Ordine to User2`
- 3 Database administrator
`revoke select on Ordine from User1 cascade`

176

Revoca di un privilegio con cascata



177

Viste e autorizzazioni di accesso

Viste = unità di autorizzazione

- Consentono la gestione ottimale della privacy

178

Esempio: gestione dei conti correnti

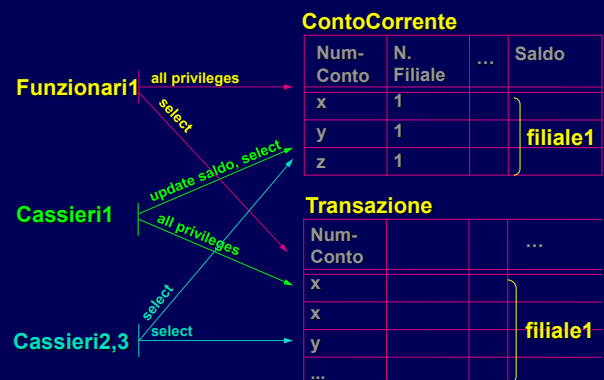


ContoCorrente(NumConto, Filiale,
 Cliente, CodFisc, DataApertura, Saldo)

Transazione(NumConto, Data, Progr.,
 Causale, Ammontare)

179

Requisiti di accesso



180

Viste relative alla prima filiale

```
create view Conto1 as
( select *
  from ContoCorrente
  where Filiale = 1)

create view Transazione1 as
( select *
  from Transazione
  where NumConto in
    ( select NumConto
      from Conto1 ) )
```

181

Autorizzazioni relative ai dati della prima filiale

```
grant all privileges on Conto1
  to Funzionari1
grant update(Saldo) on Conto1
  to Cassieri1
grant select on Conto1
  to Cassieri1, Cassieri2, Cassieri3
grant select on Transazione1
  to Funzionari1
grant all privileges on Transazione1
  to Cassieri1
grant select on Transazione1
  to Cassieri2, Cassieri3
```

182

Capitolo 5

Prof. Stefano Paraboschi

Funzioni scalari in SQL

- Costrutti definiti in SQL-2 che estendono la sintassi delle espressioni nella target list
 - `coalesce`
 - `nullif`
 - `case`
- Vengono valutate per ogni riga estratta dalla query
- Non aumentano il potere espressivo, ma semplificano la scrittura delle query (altrimenti bisognerebbe fare un'unione di diverse query)

`coalesce`

- Sintassi
 - `coalesce(list)`
- Descrizione
 - restituisce il primo valore non nullo nella lista
- Esempio:

```
select Nome,  
       coalesce(DataNasc,'Non disponibile')  
from Persona
```

`nullif`

- Sintassi
 - `nullif(input,value)`
- Descrizione
 - restituisce il valore nullo se l'espressione *input* ha come valore *value*, altrimenti restituisce *input*
- Esempio:

```
select Nome, nullif(DataNasc,'Ignota')  
from Persona
```

`case`

- Sintassi
 - `case when Condizione then Espr`
 `{ when Condizione then Espr }`
 `else Espr end`
- Descrizione
 - restituisce l'espressione associata alla prima condizione soddisfatta
- Esempio:

```
select Nome, case when Voto <= 10 then 'Grav. Ins.'  
                  when Voto <= 17 then 'Ins.'  
                  else to_char(Voto) end  
from Esame
```

Altre funzioni scalari

- Funzioni matematiche:
 - `abs()`, `degrees()`, `exp()`, `ln()`, `log()`,
 `radians()`, `round()`, `sqrt()`, `trunc()`,
 `float()`, `integer()`, `acos()`, `asin()`,
 `atan()`, `cos()`, `cot()`, `sin()`, `tan()`
- Funzioni su stringhe
 - `char_length()`, `lower()`, `upper()`,
 `substring()`, `trim()`, ...

Altre funzioni scalari

- Funzioni su dati temporali
 - now, today (variabili predefinite)
 - abstime(), age(), date_part(), date_trunc(), to_char()
- per date_part() e date_trunc() la porzione può essere
- year, month, day, hour, minute, second, decade, century, millennium, millisecond, microsecond, dow, week, epoch

Altre funzioni scalari

- Normalmente i sistemi estendono ulteriormente questo insieme di funzioni
 - Funzioni per la formattazione dell'output
 - Funzioni geometriche
 - Funzioni di accesso ad alcuni servizi del sistema operativo

Uso di SQL nelle applicazioni

- L'uso diretto di SQL è riservato a relativamente pochi utenti delle basi di dati
- Normalmente si accede a un DBMS tramite un'applicazione
- L'applicazione può essere realizzata con
 - Linguaggi di quarta generazione (4GL)
 - soluzioni specializzate, spesso integrate con un particolare DBMS
 - Normali linguaggi di programmazione (COBOL, C, C++, Java, Basic, ...) che fanno uso di SQL
 - Problema: come combinare due diversi linguaggi di programmazione?

SQL Embedded

- Le applicazioni "rinchiudono" (embed) comandi SQL all'interno delle istruzioni di un linguaggio di programmazione procedurale (COBOL, C, etc.)
- I comandi SQL embedded sono preceduti da 'EXEC SQL' e chiusi con ';'.
- Un precompilatore (o preprocessore) gestisce i comandi SQL, convertendoli in opportune chiamate di una libreria interna, normalmente specifica per il DBMS
- Il precompilatore può riconoscere eventuali anomalie nell'uso di SQL

Esempio di SQL Embedded

```
#include<stdlib.h>
main()
{
    exec sql begin declare section;
    char *NomeDip = "Manutenzione";
    char *CittaDip = "Pisa";
    int NumeroDip = 20;
    exec sql end declare section;

    exec sql connect to utente@librodb;
    if (sqlca.sqlcode != 0) {
        printf("Connessione al DB non riuscita\n"); }
    else {
        exec sql insert into Dipartimento
            values (:NomeDip, :CittaDip, :NumeroDip);
        exec sql disconnect all;
    }
}
```

Variabili in SQL Embedded

- Le variabili che realizzano il passaggio di informazioni tra il programma e l'ambiente SQL embedded devono essere dichiarate in una sezione opportuna
- Le variabili del programma sono precedute da ':' quando sono usate come parametri nei comandi SQL
- L'ambiente offre una variabile predefinita **sqlca** (SQL Communication Area) con una componente **sqlcode** che descrive l'esito dei comandi SQL (vale zero se il comando SQL è stato eseguito con successo)

Cursori

- Problema ulteriore: conflitto d'impedenza (impedance mismatch)
 - i linguaggi di programmazione tradizionali gestiscono i record uno alla volta (tuple-oriented)
 - SQL gestisce insiemi di tuple (set-oriented)
- I cursori fanno parte dello standard SQL-2 e risolvono questo problema
- Un cursore:
 - accede al risultato di una query in modo set-oriented
 - restituisce le tuple al programma una alla volta
- Sintassi per la definizione dei cursori:
declare *NomeCursore* [**scroll**] **cursor for** *SelectSQL*
[**for** <**read only** | **update** [**of** *Attributo* {*,* *Attributo*}>]

Operazioni sui cursori

- Per eseguire la query associata a un cursore:
open *NomeCursore*
- Per estrarre una tupla dal risultato della query:
fetch [*Posizione* **from**] *NomeCursore* **into** *ListaDiFetch*
- Per deallocare il cursore, scartando il risultato della query:
close *NomeCursore*
- Per accedere alla tupla corrente (quando un cursore accede a una tabella, nell'ambito di un comando di modifica) :
current of *NomeCursore* (nella clausola where)

Esempio d'uso dei cursori

```
void MostraStipendiDipart(char NomeDip[])
{
    exec sql begin declare section;
    char Nome[20], Cognome[20];
    long int Stipendio;
    exec sql end declare section;

    exec sql declare DipImp cursor for
        select Nome, Cognome, Stipendio
        from Impiegato
        where Dipart = :NomeDip;
    exec sql open DipImp;
    exec sql fetch DipImp into :Nome, :Cognome, :Stipendio;
    printf("Dipartimento %s\n", NomeDip);
    while (sqlca.sqlcode == 0)
    {
        printf("Nome: %s %s ", Nome, Cognome);
        printf("Stipendio: %d\n", Stipendio);
        exec sql fetch DipImp into :Nome, :Cognome, :Stipendio;
    }
    exec sql close DipImp;
}
```

SQL Dinamico

- SQL dinamico serve quando le applicazioni non possono sapere al momento della compilazione quale sarà il comando SQL da eseguire
- Problema principale: gestire il trasferimento di parametri tra il programma e l'ambiente SQL
- Per l'esecuzione diretta:
execute immediate *ComandoSQL*
- Se l'esecuzione è preceduta da un'analisi del comando:
prepare *NomeComando* **from** *ComandoSQL*
– seguito da:
execute *NomeComando* [**into** *TargetList*]
[**using** *ListaParametri*]

Call Level Interface (CLI)

- Con SQL Embedded il preprocessore traduce il comando in un insieme opportuno di chiamate di una libreria interna
- Un'alternativa consiste nell'offrire direttamente al programmatore una libreria di sottoprogrammi per gestire il dialogo con il DBMS
- Procedura classica
 - Si usa un sottoprogramma per creare una connessione
 - Si invia sulla connessione un comando SQL (come testo)
 - Si ottiene come risposta una struttura relazionale (la libreria offre un ricco insieme di strumenti per analizzare la struttura del risultato)
 - Al termine della sessione si chiude la connessione

ODBC

- Open DataBase Connectivity (ODBC) è una importante Call Level Interface, creata da Microsoft
- L'architettura di ODBC prevede 4 componenti
 - L'applicazione
 - specifica il driver da utilizzare e invia ad esso i comandi SQL
 - Il driver manager
 - carica il driver chiesto effettivamente dall'applicazione
 - Il driver
 - realizza il canale di comunicazione tra l'applicazione e una particolare sorgente dati
 - La sorgente dati
 - Mantiene i dati ed esegue i comandi che arrivano dall'applicazione

Caratteristiche di ODBC

- ODBC è una soluzione di grande successo per la costruzione di applicazioni in ambiente Windows
 - La sorgente dati spesso è remota e opera su una piattaforma software completamente diversa
 - gli aspetti di distribuzione e di eterogeneità sono completamente nascosti all'applicazione
 - Le applicazioni possono facilmente cambiare la sorgente dati
 - L'interfaccia però non sfrutta i vantaggi del paradigma di programmazione a oggetti ed è abbastanza complicata
 - soluzioni alternative, basate su ODBC: OLE DB e ADO

OLE DB e ADO

- OLE DB è un'interfaccia generale per l'accesso a sorgenti dati (non solo DB relazionali)
 - a seconda delle caratteristiche della sorgente dati, sarà più o meno ricco l'insieme di comandi di accesso che verranno accettati al momento dell'esecuzione
- Si basa sul modello a oggetti COM, che caratterizza la piattaforma software Microsoft
- ActiveX Data Object (ADO) è un'interfaccia semplice per l'accesso a sorgenti OLE DB

ADO

- Il modello a oggetti di ADO si basa su 4 concetti fondamentali
 - *Connection*
 - rappresenta il canale di comunicazione
 - la sua creazione richiede la specifica della sorgente dati, dell'utente e della password
 - contiene la collezione di errori *Errors*
 - *Command*
 - contiene il comando (SQL) che si vuole far eseguire sulla sorgente dati
 - *Record*
 - rappresenta la singola tupla
 - *RecordSet*
 - rappresenta l'insieme di tuple
 - Svolge il ruolo del cursore, che grazie al modello a oggetti non deve essere introdotto esplicitamente in SQL

ADO in VisualBasic (1)

```
Public Sub InterrogaBD()  
    Dim setImpiegati As ADOB.Recordset  
    Dim conn As ADOB.Connection  
    Dim stringaSQL As String  
    Dim comSQL As ADOB.Command  
    Dim messaggio As String  
  
    ' si stabilisce la connessione  
    Set conn = New ADOB.Connection  
    conn.Open "mioServer", "giovanni", "passwordsegreta"  
  
    ' si carica l'insieme degli impiegati in setImpiegati  
    Set setImpiegati = New ADOB.Recordset  
    stringaSQL = "select Nome, Cognome, Data " & _  
        "from Impiegato order by Cognome"  
    comSQL.CommandText = stringaSQL  
    setImpiegati.Open comSQL, conn, , ,  
    .....
```

ADO in VisualBasic (2)

```
.....  
' si stampano i dati di tutti gli impiegati  
Do While Not setImpiegati.EOF  
    messaggio = "Impiegato: " & setImpiegati!Nome & _  
        setImpiegati!Cognome & "(record " & _  
        setImpiegati.AbsolutePosition & _  
        " di " & setImpiegati.RecordCount & ")"  
    If MsgBox(messaggio, vbOkCancel) = vbCancel  
        Then Exit Do  
    setImpiegati.MoveNext  
Loop  
  
' si fa pulizia  
setImpiegati.Close  
conn.Close  
Set setImpiegati = Nothing  
Set conn = Nothing  
End Sub
```

JDBC

- Java è un moderno linguaggio di programmazione orientato agli oggetti, proposto dalla Sun Microsystems
 - È il cuore di un'architettura, basata sulla definizione rigorosa di un ambiente di esecuzione dei programmi scritti nel linguaggio (Java Virtual Machine)
 - Portabilità molto elevata
 - Dispone di una famiglia di librerie molto estesa, tra cui Java DataBase Connectivity (JDBC)
- JDBC ha una struttura simile a ODBC
 - il driver manager isola il driver responsabile di implementare il canale di comunicazione
 - esistono anche bridge JDBC/ODBC, per usare driver ODBC in un contesto JDBC
- Uso di JDBC
 - Si carica il driver
 - Si crea una connessione con la base di dati
 - Si compone il comando SQL e lo si invia alla base di dati
 - Si gestisce il risultato del comando SQL

Esempio d'uso di JDBC (1)

```
import java.sql.*;
public class PrimoJdbc {
    public static void main(String[] arg) {
        Connection conn = null;
        try {
            // Caricamento del driver
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
            // Apertura della connessione
            conn =
            DriverManager.getConnection("jdbc:odbc:Corsi");
        }
        catch (Exception e) { System.exit(1); }
        .....
    }
}
```

Esempio d'uso di JDBC (2)

```
.....
try {
    // Esecuzione dell'interrogazione
    Statement interrogazione = conn.createStatement();
    ResultSet risultato =
    interrogazione.executeQuery("select * from Corsi");
    while (risultato.next()) {
        String nomeCorso = risultato.getString("NomeCorso");
        System.out.println(nomeCorso);
    }
}
catch (Exception e) { System.exit(1); }
}
```

Procedure

- Moduli di programma che svolgono una specifica attività di manipolazione dei dati
- SQL-2 permette la definizione di procedure (anche note come *stored procedures*), ma solo in forma molto limitata
- La maggior parte dei sistemi offrono delle estensioni che permettono di scrivere procedure complesse (es., Oracle PL/SQL), con strutture di controllo, variabili, eccezioni, etc.
 - si ottiene un linguaggio di programmazione completo
- Due momenti:
 - dichiarazione (DDL)
 - invocazione (DML)
- Con architettura client-server sono normalmente:
 - invocate dai client
 - memorizzate ed eseguite presso i server

Esempio : prelievo dal magazzino

Magazzino

CodProd	QtaDisp	QtaRiord
1	150	100
3	130	80
4	170	50
5	500	150

Riordino

CodProd	Data	QtaOrd

Specifica

- L'utente indica un prelievo dando il codice del prodotto e la quantità da prelevare
- Se la quantità disponibile in magazzino non è sufficiente la procedura si arresta con una eccezione
- Viene eseguito il prelievo, modificando la quantità disponibile in magazzino
- Se la quantità disponibile in magazzino è inferiore alla quantità di riordino si predispone un nuovo ordine d'acquisto

Interfaccia

```
procedure Prelievo
( Prod integer,
  Quant integer )
```

Invocazione

```
Prelievo(4,150)
```

Stato iniziale nella base di dati

CodProd	QtaDisp	QtaRiord
4	170	50

Realizzazione della procedura

1. Dichiarazione variabili
2. Lettura dello stato
3. Se la quantità disponibile è insufficiente: eccezione
4. Aggiornamento dello stato
5. Se la nuova quantità disponibile è inferiore alla quantità di riordino: emissione di un ordine

Procedura

```
procedure Prelievo (Prod integer, Quant integer) is
begin
  Q1, Q2 integer;
  X exception;
  select QtaDisp, QtaRiord into Q1, Q2
    from Magazzino
   where CodProd = Prod;
  if Q1 < Quant then raise(X);
  update Magazzino
    set QtaDisp = QtaDisp - Quant
   where CodProd = Prod;
  if Q1 - Quant < Q2 then
    insert into Riordino
      values (Prod, sysdate, Q2)
  end;
```

Esempio di invocazione

Prelievo(4,150)

Prod=4, Quant=150

```
select QtaDisp, QtaRiord into Q1, Q2
  from Magazzino
 where CodProd = Prod;
```

CodProd	QtaDisp	QtaRiord
4	170	50

Q1 = 170, Q2 = 50

Invocazione (continua)

if Q1 < Quant then raise(X) non scatta

```
update Magazzino
 set QtaDisp = QtaDisp - Quant
 where CodProd = Prod
```

CodProd	QtaDisp	QtaRiord
4	20	50

Q1 - Quant < Q2 è vero:
insert into Riordino
values(Prod, sysdate, Q2)

CodProd	Data	QtaRiord
4	10-10-97	50

Problemi del progetto di procedure

- Decomposizione modulare delle applicazioni
- Aumento di:
 - efficienza
 - controllo
 - riuso
- Aumenta la responsabilità dell'amministratore della base di dati (rispetto al programmatore applicativo)
- Si sposta "conoscenza" dalle applicazioni allo schema della base di dati (indipendenza di conoscenza)
- Attenzione a non abusarne:
 - è una soluzione efficiente se il problema richiede di fare computazioni semplici su grandi quantità di dati
 - è inefficiente se su un dato estratto dalla base di dati si devono svolgere computazioni complesse

FASI NEL PROGETTO DEI DATI

Il progetto della base di dati

si inserisce nel:

Ciclo di vita del sistema informativo

comprendente in generale le seguenti attività:

- Raccolta ed analisi dei requisiti
- Progettazione (di schemi e applicazioni)
- Implementazione
- Validazione e collaudo
- Funzionamento

Ci concentriamo sulla parte più tecnica, specifica di questo corso: la progettazione degli schemi

Prerequisiti della progettazione

- L'analisi dei requisiti è condotta dalla una specifica figura professionale (analista) tramite interviste dell'utente (che deve essere per quanto possibile parte attiva della definizione dei requisiti)
- Produce una descrizione che esprimiamo tramite un testo riassuntivo (tipicamente ambiguo).
- Nella realtà si aggiungono anche:
 - Descrizioni terminologiche, glossari, raccolte informali di definizioni
 - Descrizioni astratte dei programmi (dataflow diagrams) e del loro uso da parte degli utenti (use case diagrams).

Assunzione tecnologica

- Architettura client-server con un unico database server cui si collegano le varie applicazioni

(di fatto questa scelta non è vincolante e architetture più complesse sono discusse nell'ambito del corso di Sistemi Informativi)

Fasi della progettazione

- la progettazione concettuale
- la progettazione logica
- la progettazione fisica

La progettazione concettuale

Ha per scopo tradurre il risultato dell'analisi dei requisiti in una **DESCRIZIONE FORMALE** che dovrà essere indipendente dal DBMS

La descrizione formale è espressa tramite uno **SCHEMA CONCETTUALE**, costruito utilizzando un **MODELLO CONCETTUALE DEI DATI**

La progettazione logica

Ha per scopo tradurre lo **SCHEMA CONCETTUALE** in uno **SCHEMA LOGICO**, scelto all'interno dei modelli logici dei dati:

- Gerarchico
- Reticolare
- Relazionale
- Orientato ad oggetti
- XML

Lo schema logico è dipendente dal **DBMS** ma non dallo specifico prodotto

La progettazione fisica

Ha per scopo produrre un **PROGETTO FISICO** della base dei dati, cioè un progetto che ottenga prestazioni ottimali tramite scelta e dimensionamento di strutture fisiche di accesso.

Il progetto fisico viene eseguito in modo differente su ciascun prodotto.

Dipendenze da MODELLO e DBMS

	Dipende dal MODELLO	Dipende dal DBMS
Progetto concettuale	NO	NO
Progetto logico	SI	NO
Progetto fisico	SI	SI

LE ASTRAZIONI NEI MODELLI CONCETTUALI PER BASI DI DATI

Ingredienti dei modelli concettuali

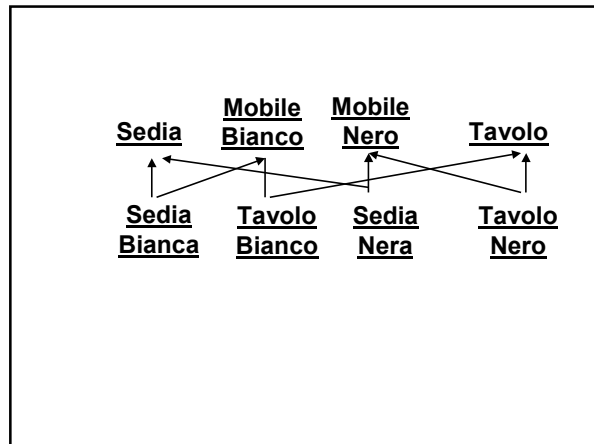
- **ASTRAZIONI**: capacità di evidenziare caratteristiche comuni ad insiemi di oggetti
- Tre **ASTRAZIONI** di base per la rappresentazione della conoscenza:
 - Classificazione
 - Aggregazione
 - Generalizzazione

Classificazione

Capacità di definire classi di oggetti o fatti del mondo reale

- LIBRO
- BICICLETTA
- PERSONA
- APPARTAMENTO

Per ogni classe esiste un implicito "test di appartenenza" che consente di dire se un oggetto o fatto del mondo reale è una istanza della classe.

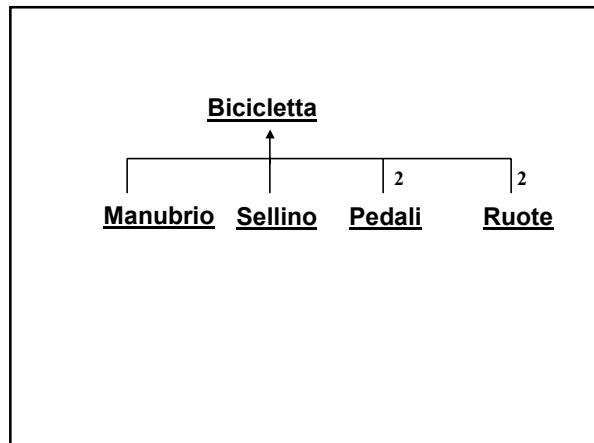


Aggregazione

Costruzione di una classe complessa aggregando classi più semplici (componenti)

- BICICLETTA
- RUOTE
- PEDALI
- MANUBRIO

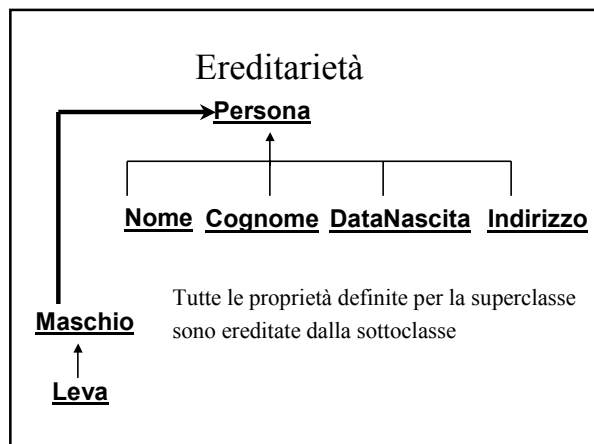
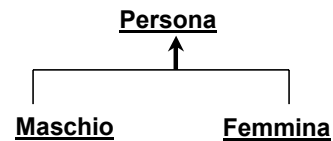
Per ogni componente si indica quante istanze sono presenti in una istanza della classe aggregata



Generalizzazione

Stabilisce legami di sottoinsieme fra classi

- FEMMINA < PERSONA
- MASCHIO < PERSONA



IL MODELLO ENTITA'-RELAZIONE (ENTITY-RELATIONSHIP)

MODELLO ENTITA' RELAZIONE

- il modello Entity-Relationship (ER), (P.P.Chen 1976) si è affermato come standard industriale di buona parte delle metodologie e degli strumenti per il progetto concettuale di basi di dati.
- **Attenzione:** Relationship = Associazione (pero' poi si dice informalmente "relazione")

Entità

- Rappresenta una classe di oggetti (es., automobili, impiegati, studenti) o di fatti (es., conti correnti, corsi universitari)
- Devono essere oggetti rilevanti per la applicazione
- Ogni entità è caratterizzata da un nome

simbolo grafico per rappresentare entità



esempio:



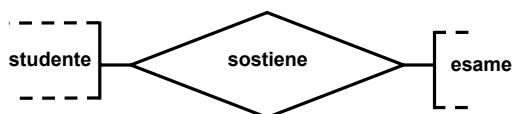
Relazione (o Associazione)

- Rappresenta una aggregazione di entità di interesse per l'applicazione
- Ogni istanza di una associazione è una ennupla tra istanze di entità (es., legame tra un automobile e il suo proprietario)
- Ogni associazione è caratterizzata da un nome

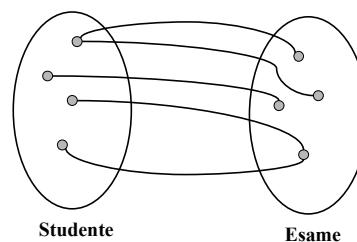
simbolo grafico per rappresentare associazioni



esempio:



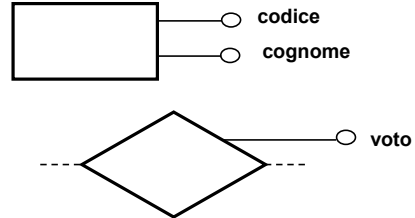
Rappresentazione grafica delle istanze



Attributi

- Rappresentano caratteristiche delle entità e delle associazioni di interesse per l'applicazione
- Ogni attributo è caratterizzato da un nome

simbolo grafico per rappresentare attributi



Linee guida per il progetto

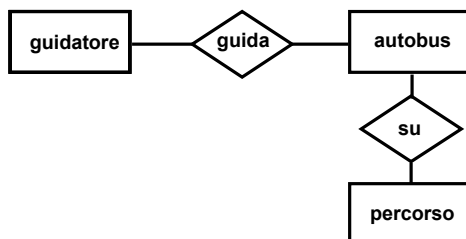
- Se il concetto è significativo per il contesto applicativo: entità
- Se il concetto è descrivibile tramite un dato elementare: attributo
- Se il concetto definisce un legame tra entità: associazione

Corrispondenza tra concetti e elementi del modello ER

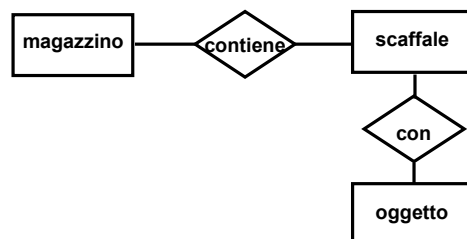
La corrispondenza tra oggetti e fatti del mondo reale e entità, associazioni e attributi non è assoluta ma dipende dal contesto:

Es.: l'auto BOF34675 ha colore rosso
il colore rosso ha lunghezza d'onda = ~700 nm

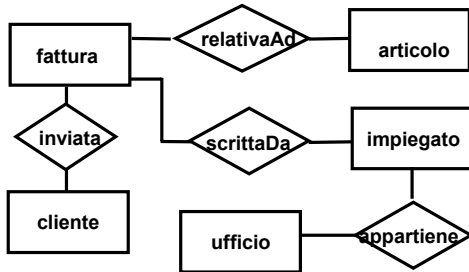
Esempio: gestione viaggi



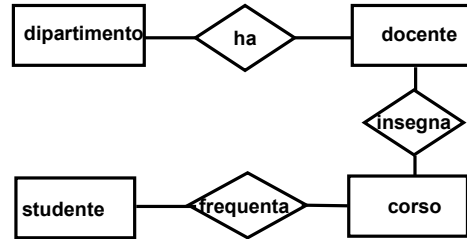
Esempio: gestione magazzino



Esempio: gestione fatture



Esempio: università



ASSOCIAZIONI NEL MODELLO ER

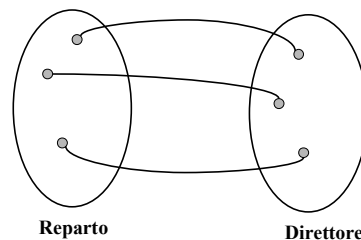
Cardinalità delle associazioni

- Per cardinalità si intende un vincolo sul numero di istanze di associazione cui ciascuna istanza di entità deve partecipare.
- È una coppia (MIN-CARD, MAX-CARD)
 - MIN-CARD = 0 (opzionale)
 - = 1 (obbligatoria)
 - MAX-CARD = 1 (uno)
 - = N (molti)
- In base alla sola cardinalità massima si hanno associazioni uno-uno, uno-molti, molti-molti

Associazione 1:1



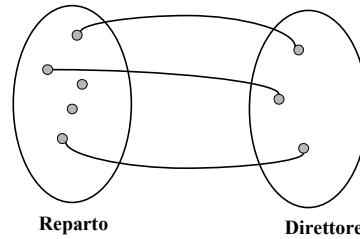
- un reparto deve essere diretto da uno e un solo direttore (1,1)
- un direttore deve dirigere uno ed un solo reparto (1,1)



Associazione 1:1 con opzionalità



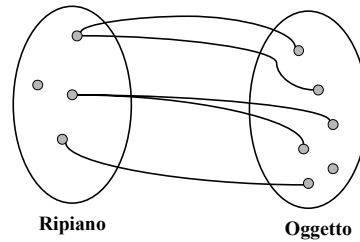
- un reparto può essere diretto da uno e un solo direttore (0,1)
un direttore deve dirigere uno ed un solo reparto (1,1)



Associazione 1:N



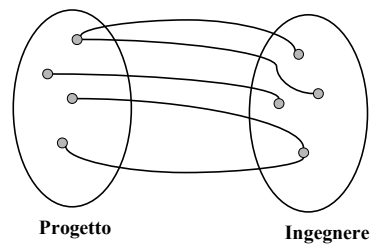
- un reparto può contenere molti oggetti (0,n)
un oggetto può essere contenuto al più su un ripiano (0,1)



Associazione N:M

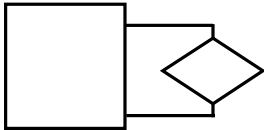


- un progetto può essere fatto da molti ingegneri (0,n),
un ingegnere deve partecipare ad uno o più progetti (1,m)

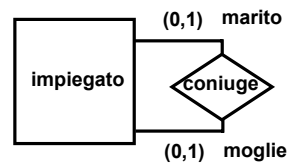


Auto-associazioni

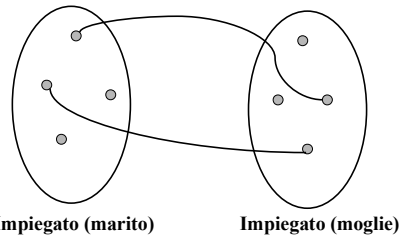
associazioni aventi come partecipanti istanze provenienti dalla stessa entità (chiamate anche “ad anello”):



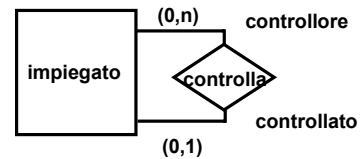
Auto-associazioni 1:1



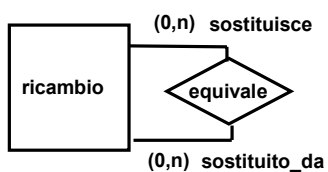
può essere riportato il “ruolo” sul ramo



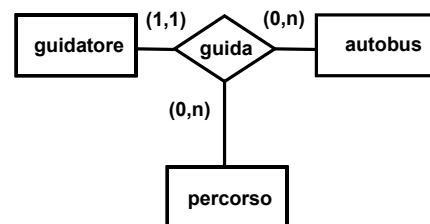
Auto-associazioni 1:N

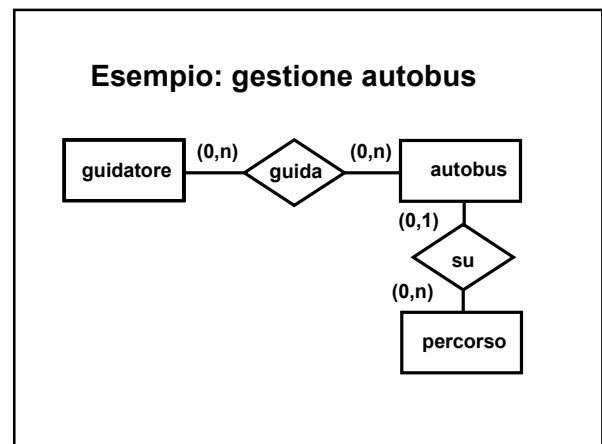
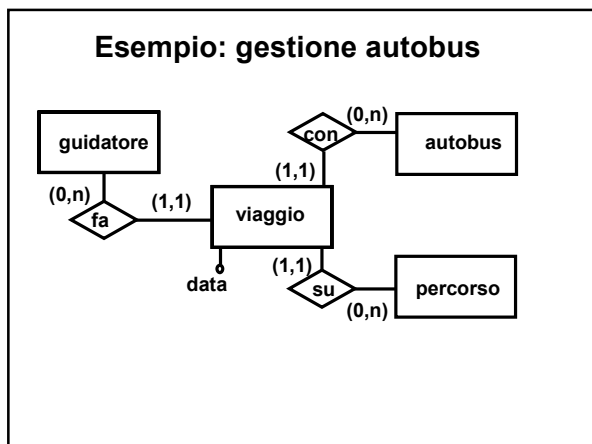
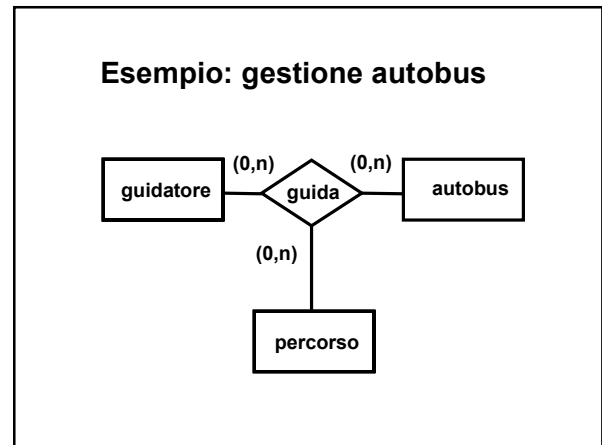
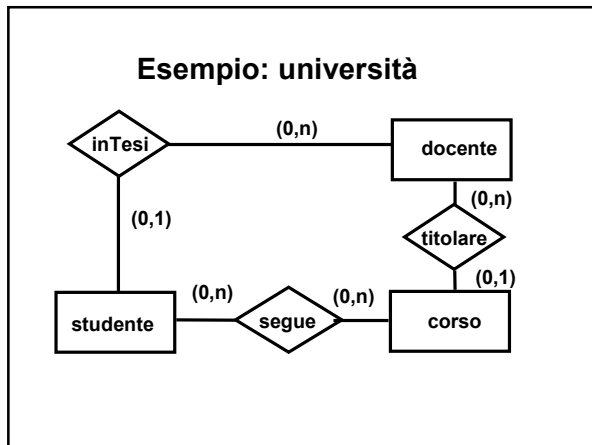
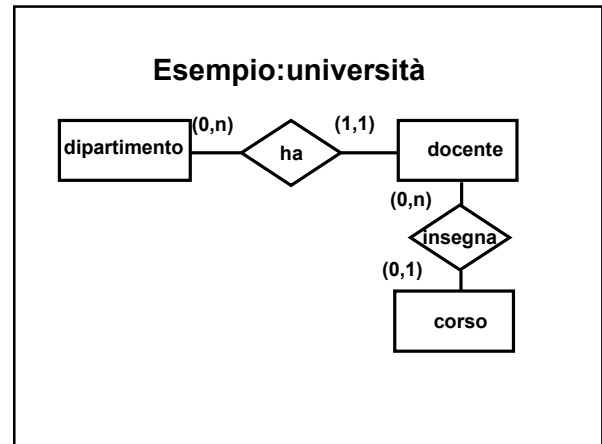
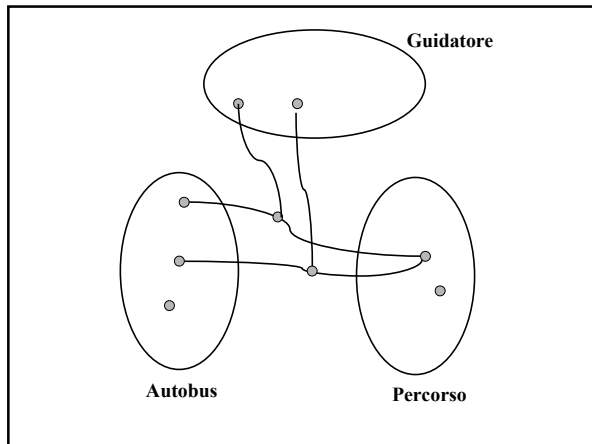


Auto-associazioni N:M



Associazioni ternarie





ATTRIBUTI E IDENTIFICATORI NEL MODELLO ER

Cardinalità degli attributi

- una prima classificazione:

attributo scalare (semplice, ad un sol valore)

es.: matricola, cognome, voto

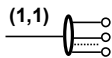
attributo multiplo (sono ammessi n valori)

(1,n) es.: qualifica, titolo, specialità

il simbolo (n,m) esprime la cardinalità dell'attributo.

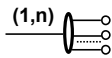
Attributi composti

attributo composto



es.: data (gg,mm,aaaa),
indirizzo (via, numero civico,
città, provincia, cap)

attributo multiplo composto



es.: telefono (stato,
città, numero)

Opzionalità

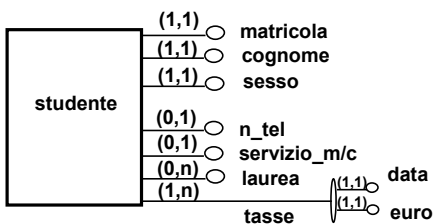
Attributo opzionale (è ammessa la "non esistenza del valore")



es.: tel., qualifica, targa

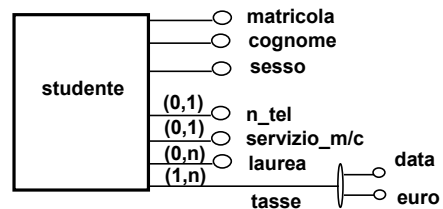
Esempio

Esempio:



Esempio con default

Esempio:

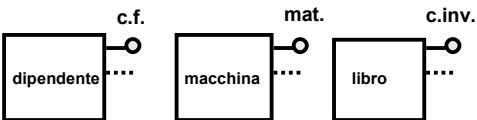


Identificatore

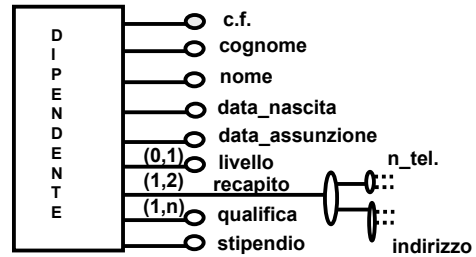
Un identificatore caratterizza in modo univoco ciascuna singola istanza di entità

simbolo —○

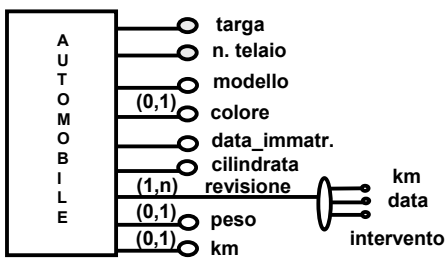
non è modificabile (in generale...)



esempio

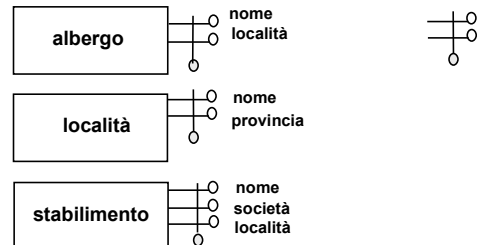


esempio

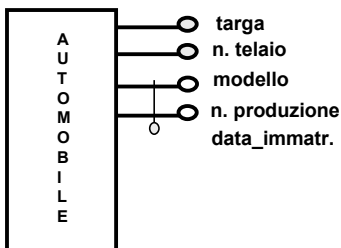


Identificatori composti

L'identificatore di un'entità può essere composto



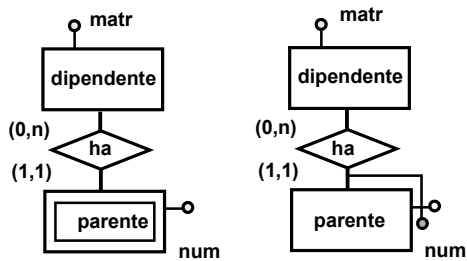
esempio



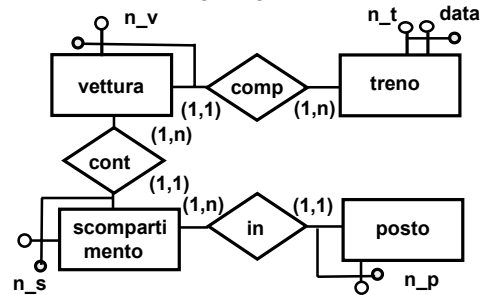
Le entità deboli

- Le entità deboli possono esistere se e solo se sono presenti entità "forti" da cui queste dipendono
 - In caso di eliminazione dell'istanza "forte" di riferimento le istanze deboli collegate devono essere eliminate

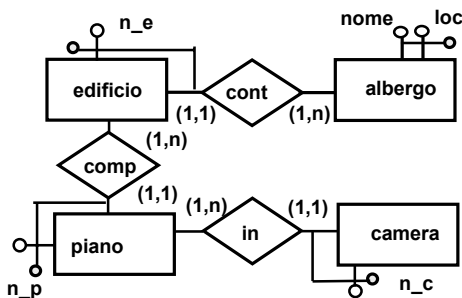
Simboli usati



esempio: posti treno



Esempio: gestione camere di un albergo



Esempi: commento

- le entità con identificatore esterno sono deboli poiché a tutti i livelli la cancellazione di una entità provoca la cancellazione delle entità deboli collegate

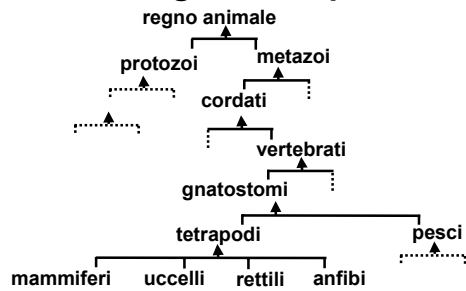
(eliminazione di vettura, chiusura di scompartimento, inagibilità edificio o piano, ecc.)

GERARCHIE DI GENERALIZZAZIONE

Gerarchie di generalizzazione

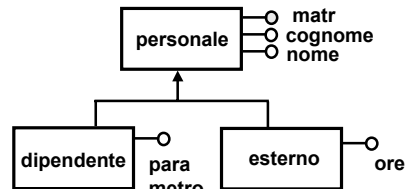
- Una gerarchia di generalizzazione è un legame logico tra un'entità padre E ed alcune entità figlie $E_1 E_2 \dots E_n$ dove:
 - E è la generalizzazione di $E_1 E_2 \dots E_n$
 - $E_1 E_2 \dots E_n$ sono specializzazioni di E tale per cui:
 - ogni istanza di E_k è anche istanza di E
 - una istanza di E può essere una istanza di E_k
- Le entità figlio ereditano le proprietà (attributi, relazioni, identificatori) dell'entità padre.

Una delle gerarchie più note



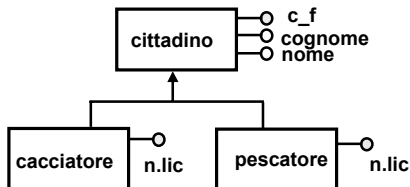
Esempio di gerarchia

un'azienda si avvale dell'opera di professionisti esterni, quindi il suo personale si suddivide in esterni e dipendenti:



Esempio di gerarchia

un comune gestisce l'anagrafe ed i servizi per i suoi cittadini alcuni di questi richiedono la licenza di caccia o pesca :



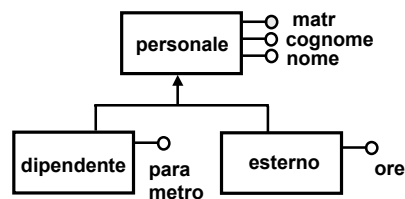
Proprieta' delle gerarchie

- **t** sta per totale: ogni istanza dell'entità padre deve far parte di una delle entità figlie
 - nell'esempio il personale si divide (completamente) in esterni e dipendenti
- **p** sta per parziale: le istanze dell'entità padre possono far parte di una delle entità figlie
 - nell'esempio i cacciatori e pescatori sono un sottoinsieme dei cittadini

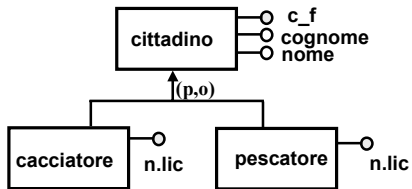
Proprieta' delle gerarchie

- **e** sta per esclusiva: ogni istanza dell'entità padre non può far parte di più di una delle entità figlie
 - nell'esempio si esclude che una istanza di personale possa appartenere ad entrambe le sottoclassi
- **o** sta per overlapping: ogni istanza dell'entità padre può far parte di più entità figlie
 - nell'esempio un cittadino può essere al tempo stesso cacciatore e pescatore

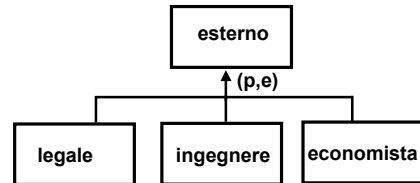
Default: (t,e)



Indicazione della proprietà

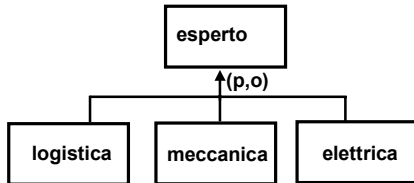


Un'ulteriore specializzazione



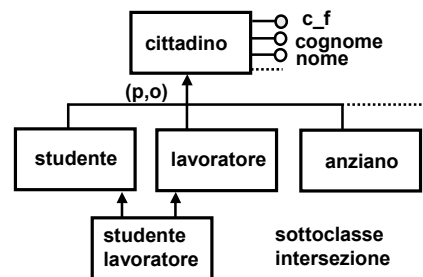
p: possono esistere esterni generici che non sono né legali, né ingegneri, né economisti ma non interessa stabilire una sottoclasse ad hoc

Un'ulteriore specializzazione

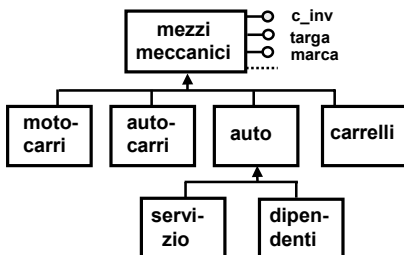


p : possono esistere esperti sia meccanici, sia elettrici, sia della logistica
O: le tre qualifiche non si escludono

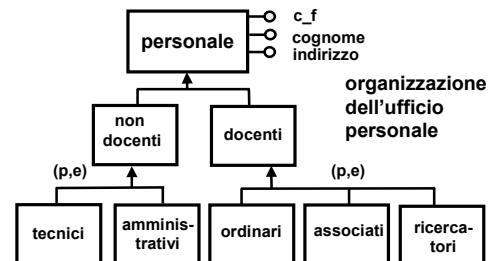
Esempio: un comune



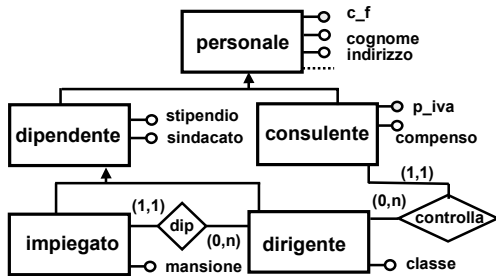
Esempio: parco mezzi meccanici



Esempio: università



Esempio: personale d'azienda



STRATEGIE DI PROGETTO

Strategie di progetto

- Lo sviluppo dello schema si può eseguire seguendo due strategie fondamentali:
 - Top-Down
 - Bottom-Up
- Per quanto riguarda le strategie bottom-up, vedremo i casi:
 - “A macchia d’olio”
 - Mista

Progetto top-down e bottom-up

- Top-down: si procede per raffinamenti a partire da una descrizione che comprende TUTTA la realtà di interesse
- Bottom-up: si disegnano separatamente aspetti della realtà e poi li si integra costruendo un unico schema

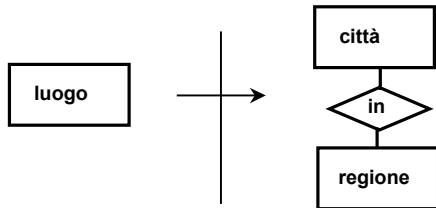
Confronto

- Top down: il progetto è più ordinato e razionale, ma più difficile (il progettista deve possedere una “visione d’insieme”)
- Bottom-up: si possono prendere decisioni differenti nell’affrontare i sotto-problemi, che si tradurranno in “conflitti” (modelli diversi della stessa realtà).

Strategia top-down “pura”

- A partire dalle specifiche si costruisce uno schema iniziale
- Dallo schema iniziale si arriva per raffinamenti successivi a schemi intermedi e poi allo schema finale
- I raffinamenti prevedono l’uso di trasformazioni elementari (primitive) che operano sul singolo concetto per descriverlo con maggior dettaglio

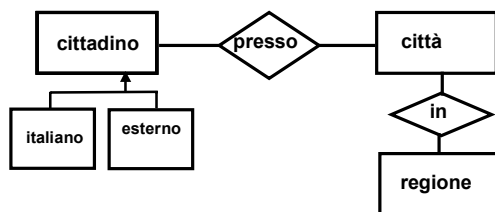
Trasformazione elementare top-down



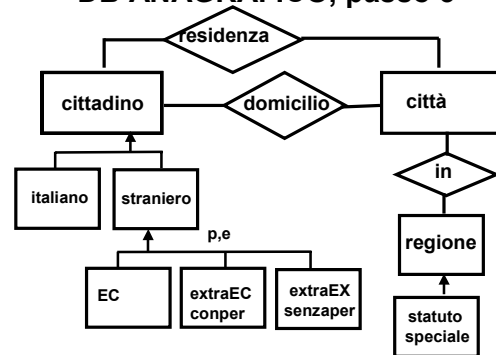
DB ANAGRAFICO, passo 1



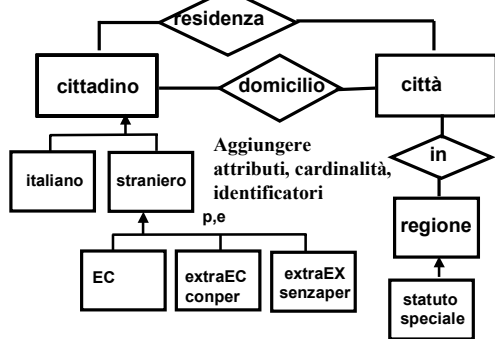
DB ANAGRAFICO, passo 2



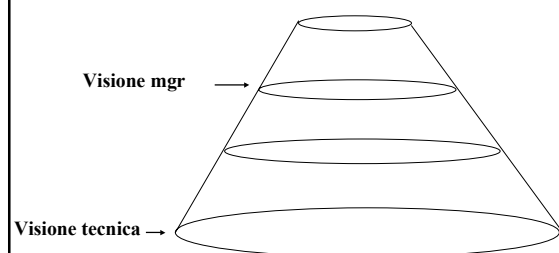
DB ANAGRAFICO, passo 3



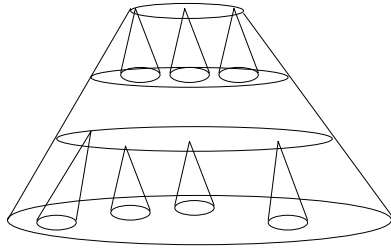
DB ANAGRAFICO, passo 4



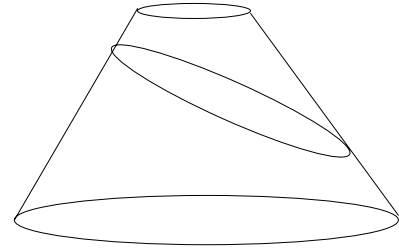
“Piani” del progetto



Progetto “equilibrato”



Progetto “squilibrato”



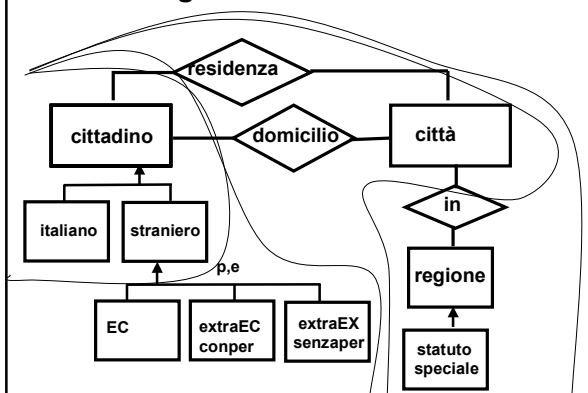
Valutazione di una strategia top down

- vantaggi:
 - il progettista descrive inizialmente lo schema trascurando i dettagli
 - precisa lo schema gradualmente
- problema:
 - non va bene per applicazioni complesse perché è difficile avere una visione globale precisa iniziale di tutte le componenti del sistema

Strategia “a macchia d’olio”

- Le specifiche nascono progressivamente, affrontando i requisiti fino al massimo dettaglio e “avanzando” per sottoproblemi
- La tecnica è adatta a tradurre pian piano una descrizione testuale in un diagramma

Strategia a macchia d’olio



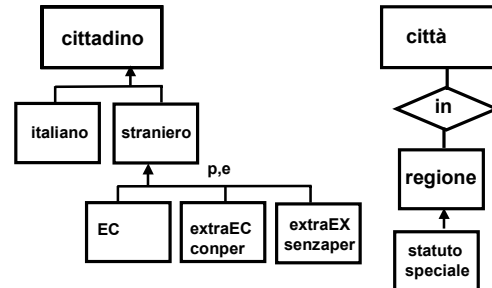
Valutazione della strategia “a macchia d’olio”

- La tecnica è adatta a tradurre pian piano una descrizione testuale in un diagramma
- Pur essendo bottom-up, il progettista analizza le specifiche in modo “stratificato” e le aggiunge progressivamente a un unico schema, perciò i conflitti sono meno probabili

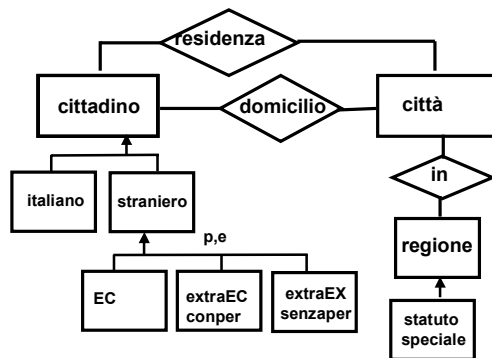
Strategia Mista

- Adatta di fronte a progetti ampi
- Si suddividono le specifiche in parti (ad esempio: le funzioni amministrazione, personale, marketing, vendita, produzione di una azienda)
- Si realizzano top-down le varie parti
- Si realizza (bottom-up) l'integrazione delle varie parti sviluppate

Sviluppo di due sottoschemi



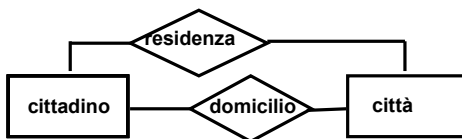
Integrazione



Valutazione della strategia mista

- **vantaggi:**
 - diversi progettisti elaborano gli schemi parziali, il singolo progettista ha una visione più precisa del proprio settore
- **problema:**
 - conflitti e difficoltà di integrazione
- **soluzione possibile:**
 - sviluppare un piccolo schema dei soli concetti principali (schema scheletro) in modo top-down e attenersi alle scelte presenti nello schema scheletro in tutti gli altri schemi.

Schema Scheletro



Sintesi

- Un progettista esperto procede (inconsiamente) sia in modo top-down che in modo bottom-up
- Per affrontare gli esercizi, la tecnica a macchia d'olio viene usata spesso
- In ogni caso, è possibile (e molto conveniente) DOCUMENTARE un progetto in modo top-down a posteriori

QUALITA' DI UNO SCHEMA CONCETTUALE

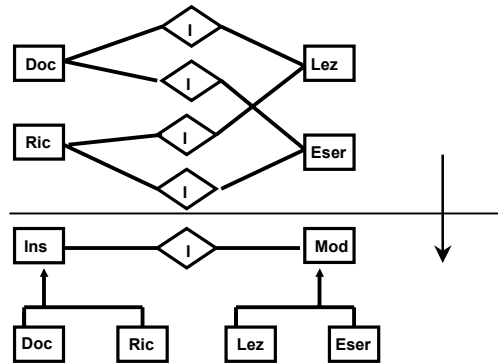
Qualità di uno schema concettuale

- Completezza
- Correttezza
- Leggibilità
- Minimalità
- Auto-Esplicatività

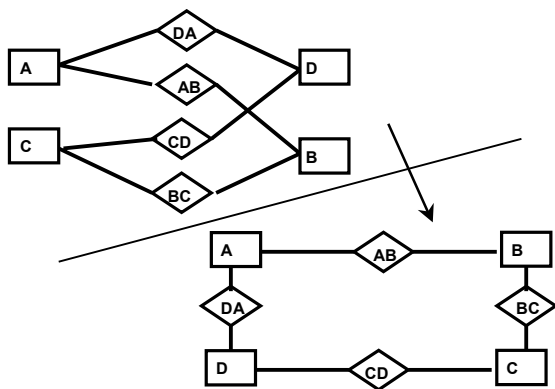
Completezza e correttezza

- Rappresentare in modo completo e corretto i requisiti
- Sono proprietà ovvie ma sulle quali c'è poco da aggiungere
 - Per la completezza: assicurarsi che i dati consentano di eseguire tutte le applicazioni
 - Per la correttezza: assicurarsi che sia possibile popolare la base di dati anche con informazione parzialmente incompleta durante fasi iniziali della sua evoluzione

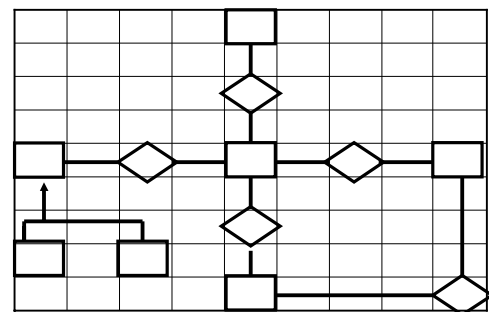
Leggibilità concettuale



Leggibilità grafica



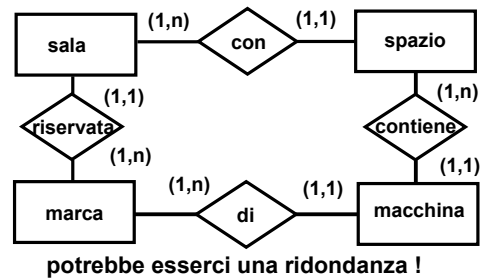
Disegnare in una griglia



Ridondanze negli schemi

- Una società gestisce delle sale di esposizione
- le sale di esposizione sono riservate a marche di macchine
- le sale comprendono spazi di esposizione
- gli spazi contengono macchine
- le macchine appartengono ad una certa marca

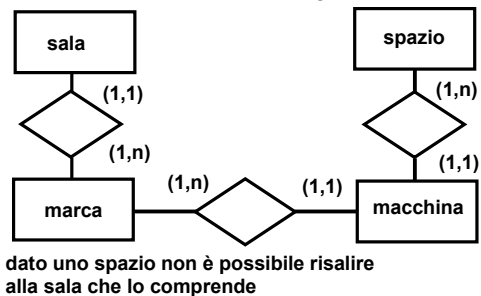
Ridondanze negli schemi



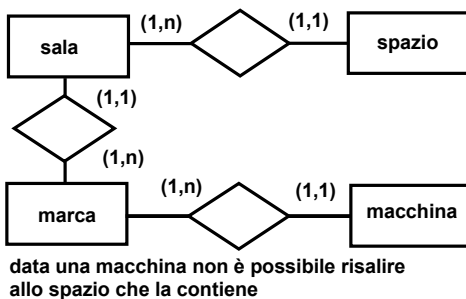
Ridondanze negli schemi

- il ciclo è ridondante se la sistemazione delle macchine negli spazi viene effettuata nel rispetto del vincolo che una sala sia assegnata per intero ad una sola marca
- proviamo ad eliminare le 4 associazioni a turno e verificare il rispetto delle specifiche

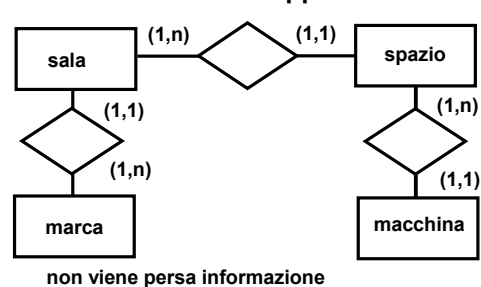
Eliminazione di: comprendere

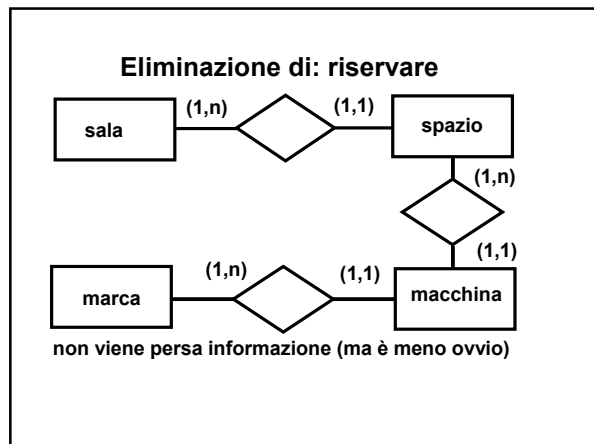


Eliminazione di: contenere



Eliminazione di: appartenere



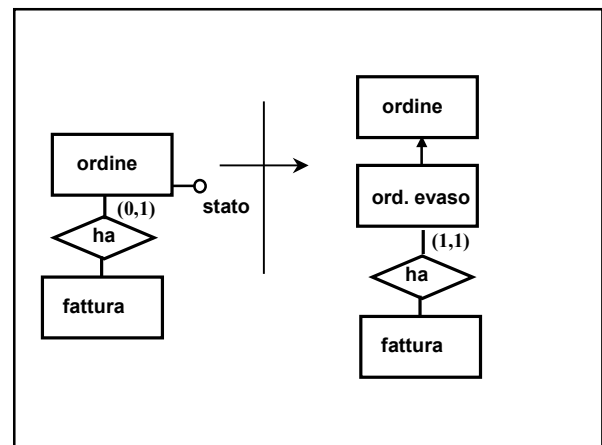


Discussione

- Ragionevole la prima eliminazione
- Piuttosto irragionevole la seconda eliminazione
- E in ogni caso: occorre conoscere bene il significato delle associazioni!

Auto-esplicitività

- Fare in modo che lo schema rappresenti esplicitamente il massimo di conoscenza sulla realtà



A questo punto....

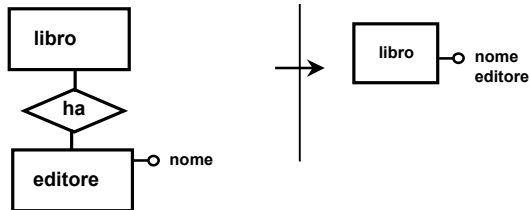
- Il progetto è stato condotto (o documentato) top-down
- Il progetto risponde ai requisiti di qualità

Ultimo passo: post-processing

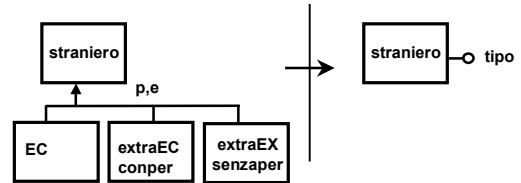
Post-processing

- Verificare che:
 - Tutte le entità abbiano un identificatore
 - Tutte le associazioni abbiano cardinalità ben definite
 - Le entità siano significative (consentano di rappresentare più di un attributo o siano collegate ad altre entità tramite associazioni)
 - Le generalizzazioni siano utili (consentano di ereditare proprietà)

Esempio di entità inutile



Esempio di gerarchia inutile



ESERCIZIO DI PROGETTAZIONE CONCETTUALE

...

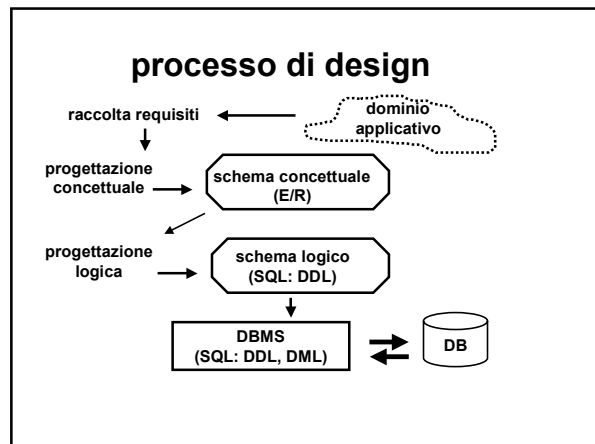
Progetto logico

Progetto logico

- Lo schema E/R descrive un dominio applicativo a un dato livello di astrazione
- Lo schema E/R serve per:
 - fornire una descrizione sintetica e visiva
 - rappresentare buona parte della semantica dell'applicazione
 - scambiare informazioni sull'attività progettuale tra i membri del team di progetto e mantenere una documentazione

Progetto logico

- Non esistono DBMS in grado di operare direttamente sui concetti di schemi E/R
 - è quindi necessario tradurli in altri schemi di dati (logico relazionale in queste lezioni)
 - questa traduzione può essere eseguita in modo semi-automatico
 - le scelte alternative devono tenere conto dell'efficienza dello schema logico risultante e delle operazioni da effettuare (derivanti da flussi e processi)



scelte alternative

si possono individuare alcune linee guida:

- considerare le proprietà logiche comunque primarie rispetto ai motivi di efficienza
- tenere sulla stessa entità informazioni che verranno di frequente consultate insieme
- tenere su entità separate informazioni che verranno consultate separatamente
- limitare l'incidenza di valori nulli per attributi opzionali

fasi del progetto

il progetto produce trasformazioni e traduzioni dello schema E/R con le seguenti fasi:

- 1 eliminazione delle gerarchie isa
- 2 selezione delle chiavi primarie, eliminazione delle identificazioni esterne
- 3 normalizzazione degli attributi composti o multipli
- 4 traduzione di entità e associazioni in schemi di relazioni
- 5 verifica di normalizzazione

eliminazione delle gerarchie

il modello relazionale non rappresenta le gerarchie, le gerarchie sono sostituite da entità e associazioni:

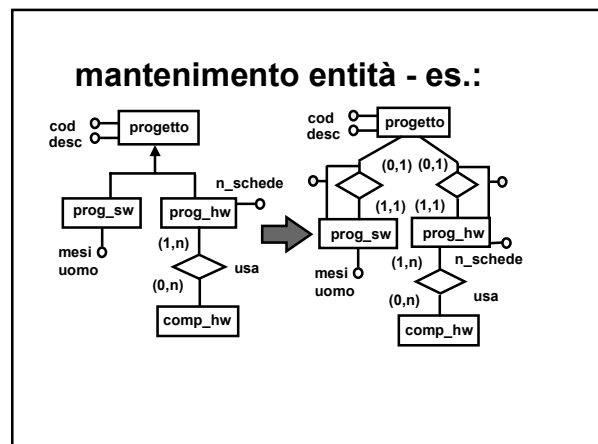
- 1) mantenimento delle entità con associazioni
- 2) collasso verso l'alto
- 3) collasso verso il basso

l'applicabilità e la convenienza delle soluzioni dipendono dalle proprietà di copertura e dalle operazioni previste

mantenimento delle entità

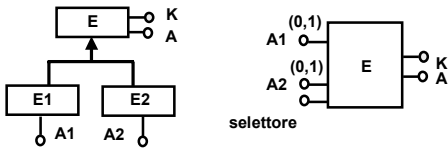
- tutte le entità vengono mantenute
- le entità figlie sono in associazione con l'entità padre
- le entità figlie sono identificate esternamente tramite l'associazione

questa soluzione è sempre possibile, indipendentemente dalla copertura



eliminazione delle gerarchie

- Il collasso verso l'alto riunisce tutte le entità figlie nell'entità padre



selettore è un attributo che specifica se una istanza di E appartiene a una delle sottoentità

isa: collasso verso l'alto

- il collasso verso l'alto favorisce operazioni che consultano insieme gli attributi dell'entità padre e quelli di una entità figlia:
 - in questo caso si accede a una sola entità, anziché a due attraverso una associazione
- gli attributi obbligatori per le entità figlie divengono opzionali per il padre
 - si avrà una certa percentuale di valori nulli

isa: collasso verso l'alto

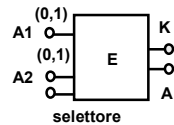
Copertura dell'ISA

totale esclusiva:

selettore ha N valori,
quante sono le sottoentità

parziale esclusiva:

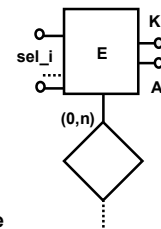
selettore ha N+1 valori; il valore in più serve per le istanze che non appartengono ad alcuna sottoentità



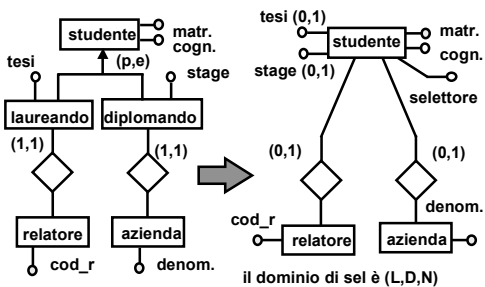
isa: collasso verso l'alto

Copertura dell'ISA:

overlapping: occorrono tanti selettori booleani quante sono le sottoentità, sel_i è "vero" per ogni istanza di E che appartiene a E_i se la copertura è parziale i selettori possono essere tutti "falsi"
le associazioni connesse alle sottoentità si trasportano su E, le eventuali cardinalità minime diventano 0



isa: collasso verso l'alto



isa: collasso verso il basso

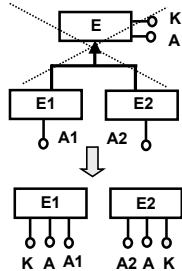
Collasso verso il basso:

- si elimina l'entità padre trasferendone gli attributi su tutte le entità figlie
 - una associazione del padre è replicata, tante volte quante sono le entità figlie
 - la soluzione è interessante in presenza di molti attributi di specializzazione (con il collasso verso l'alto si avrebbe un eccesso di valori nulli)
 - favorisce le operazioni in cui si accede separatamente alle entità figlie

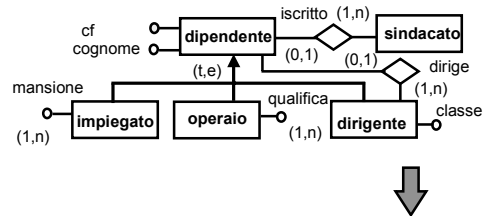
isa: collasso verso il basso

limiti di applicabilità:

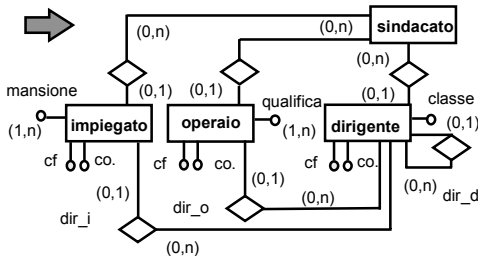
- se la copertura è parziale non si può fare: dove mettere gli E che non sono né E1, né E2 ?
- se la copertura è overlapping introduce ridondanza: per una istanza presente sia in E1 che in E2 si rappresentano due volte gli attributi di E



collasso verso il basso: es.



collasso verso il basso: es.

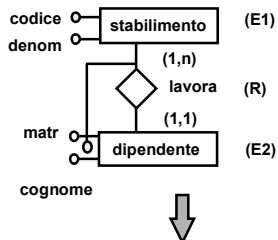


Scelta della chiave primaria

- È necessario che tra i diversi identificatori di una entità venga designata una chiave primaria: per la chiave primaria occorrerà, infatti, che il DBMS sia provvisto di strumenti per garantire l'unicità dei valori
- criteri euristici di scelta:
 - primo: scegliere la chiave che è usata più frequentemente per accedere all'entità
 - secondo: si preferiscono chiavi semplici a chiavi composte, interne anziché esterne

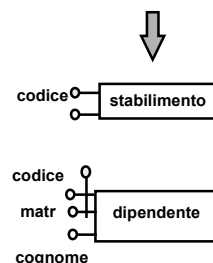
identificatori esterni

- una componente di identificazione esterna di una entità E2 da una entità E1 attraverso una associazione R comporta il trasporto della chiave primaria di E1 su E2



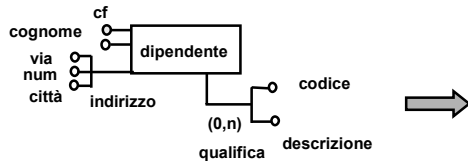
identificatori esterni

- in questo modo l'associazione è rappresentata attraverso la chiave, e può essere eliminata
- la chiave trasportata è chiave esterna
- in presenza di più identificazioni in cascata, è necessario iniziare la propagazione dall'entità che non ha identificazioni esterne



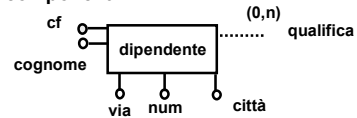
attributi composti/ripetuti

le relazioni non possono contenere attributi composti o attributi ripetuti, ma solamente attributi "atomici"



attributi composti

- Due possibili soluzioni
 - eliminare l'attributo composto e considerare i suoi componenti come attributi semplici
 - in questo modo si perde la visione unitaria ma si mantiene l'articolazione dei componenti



attributi composti

- eliminare i componenti e considerare l'attributo come semplice
- in questo modo lo schema risulta semplificato, perdendo parte della struttura



attributi ripetuti

la definizione di relazione impone che, se una entità E ha un attributo A ripetuto, si crei una nuova entità che contenga l'attributo e sia collegata a E:

Caso a) - un valore può comparire una volta sola nella ripetizione:

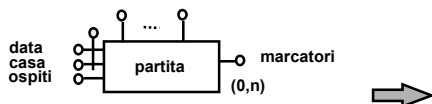
la nuova entità EA ha l'identificatore composto dall'identificatore di E più l'attributo A



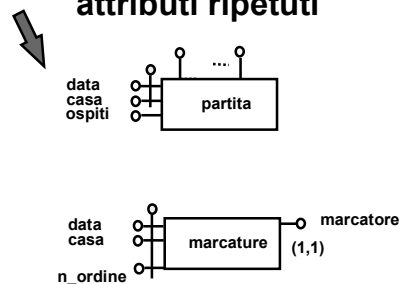
attributi ripetuti

Caso b) - un valore può comparire più volte nella ripetizione:

la nuova entità EA ha l'identificatore composto dall'identificatore di E più un valore identificante sintetico (ad esempio, un numero d'ordine)



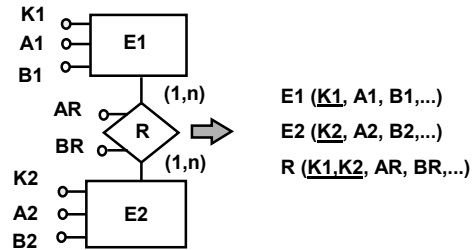
attributi ripetuti



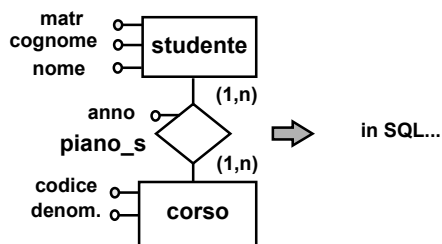
Traduzione standard

- ogni entità è tradotta con una relazione con gli stessi attributi
 - la chiave è l'identificatore dell'entità stessa (già visto)
- ogni associazione è tradotta con una relazione con gli stessi attributi, cui si aggiungono gli identificatori di tutte le entità che essa collega (già visto)
 - la chiave è composta dalle chiavi delle entità collegate

traduzione standard



traduzione standard: es.



traduzione standard: es.

```
CREATE TABLE STUDENTE (MATR... NOT NULL,
..., NOME..., PRIMARY KEY (MATR));

CREATE TABLE CORSO (CODICE... NOT NULL,
DENOM ..., PRIMARY KEY (CODICE));

CREATE TABLE PIANO_ST (MATR... NOT NULL,
CODICE... NOT NULL, ANNO...
PRIMARY KEY (MATR, CODICE),
FOREIGN KEY (MATR) REFERENCES STUDENTE
FOREIGN KEY (CODICE) REFERENCES CORSO);
```

altre traduzioni

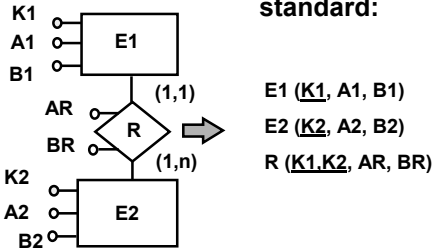
- La traduzione standard è sempre possibile ed è l'unica possibilità per le associazioni N a M
- Altre forme di traduzione delle associazioni sono possibili per altri casi di cardinalità (1 a 1, 1 a N)
- Le altre forme di traduzione fondono in una stessa relazione entità e associazioni

altre traduzioni

- Le altre forme di traduzione:
 - danno origine a un minor numero di relazioni e generano quindi uno schema più semplice
 - richiedono un minor numero di join per la navigazione attraverso un'associazione, ovvero per accedere alle istanze di entità connesse tramite l'associazione
 - penalizzano le operazioni che consultano soltanto gli attributi di una entità che è stata fusa

Associazione binaria 1 a N

- traduzione standard:



associazione binaria 1 a N

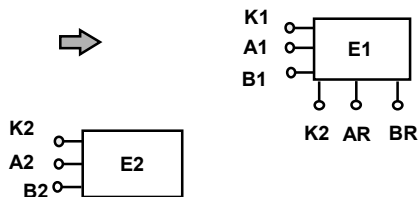
- Se E1 partecipa con cardinalità (1,1) può essere fusa con l'associazione, ottenendo una soluzione a due relazioni:

$E1(\underline{K1}, A1, B1, K2, AR, BR)$
 $E2(\underline{K2}, A2, B2)$

- Se E1 partecipa con cardinalità (0,1) la soluzione a due relazioni ha valori nulli in K2, AR, BR per le istanze di E1 che non partecipano all'associazione

Associazione binaria 1 a N

- equivale a:

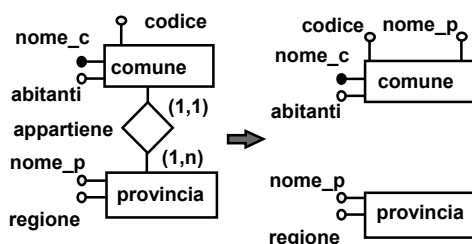


associazione binaria 1 a N

- Attenzione : in questo caso, poiché la partecipazione di E1 è 0,1 o 1,1, si nota facilmente che ad un dato valore di K1 corrisponde uno e un sol valore di K2 (non è vero il contrario), quindi si può dire che K1 implica K2 o, anche, che esiste una dipendenza funzionale da K1 a K2
- nella soluzione a 3 relazioni la chiave della relazione che traduce l'associazione è riducibile a K1:

$E1(\underline{K1}, A1, B1)$, $E2(\underline{K2}, A2, B2)$
 $R(\underline{K1}, K2, AR, BR)$

ass. binaria 1 a N es.



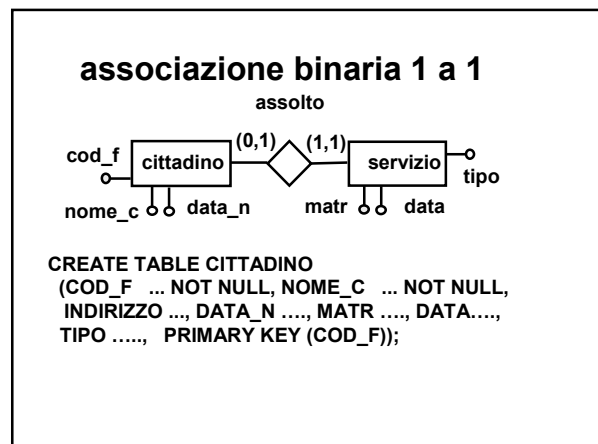
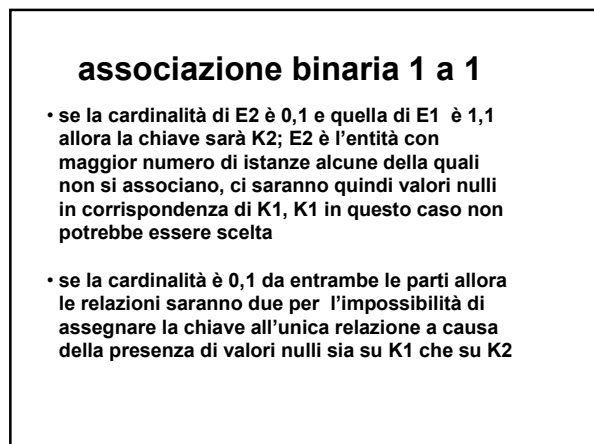
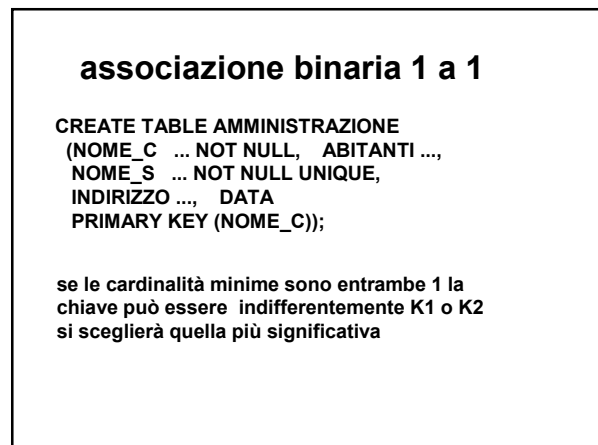
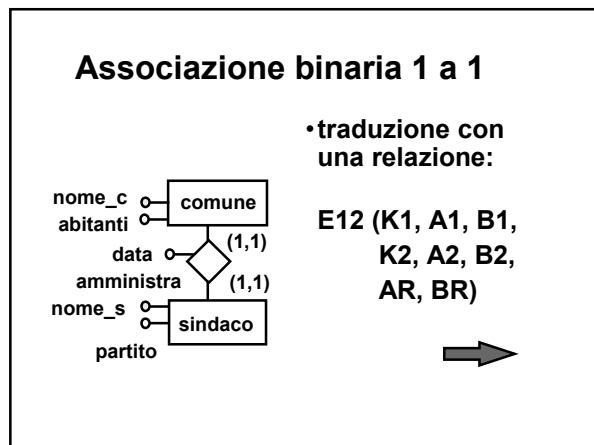
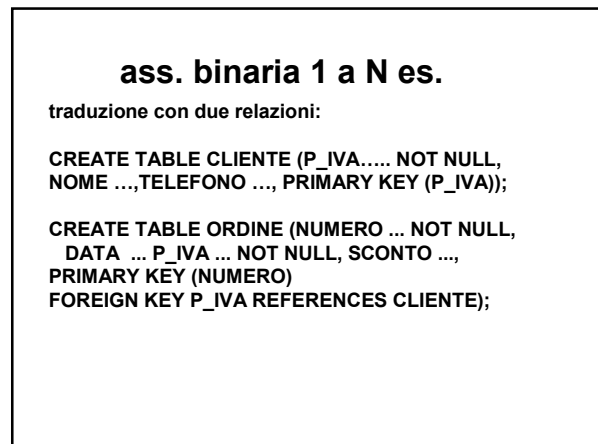
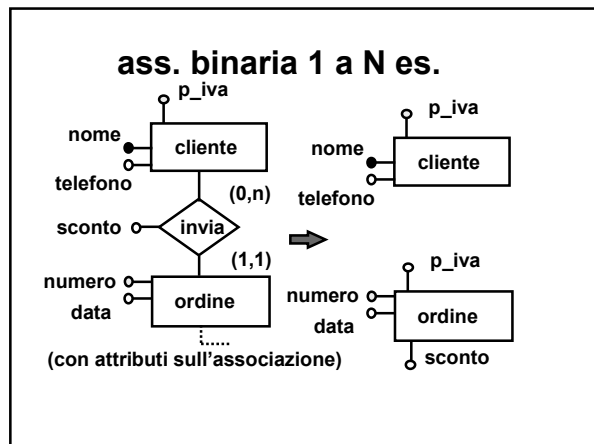
(senza attributi sull'associazione)



ass. binaria 1 a N es.

```
CREATE TABLE PROVINCIA
(NOME_P ... NOT NULL,
REGIONE ... PRIMARY KEY (NOME_P));
```

```
CREATE TABLE COMUNE
(CODICE ... NOT NULL, NOME_C ...
ABITANTI ..., NOME_P ... NOT NULL
PRIMARY KEY (CODICE)
FOREIGN KEY NOME_P
REFERENCES PROVINCIA);
```



associazione binaria 1 a 1

- Traduzione con due relazioni
 - l'associazione può essere compattata con l'entità che partecipa obbligatoriamente (una delle due se la partecipazione è obbligatoria per entrambe) la discussione sulla chiave è analoga al caso di traduzione con una relazione

E1 (K1, A1, B1,...)

E2 (K2, A2, B2,... K1, AR, BR)

associazione binaria 1 a 1

- Traduzione con tre relazioni
 - la chiave della relazione che traduce l'associazione può essere indifferentemente K1 o K2, non ci sono problemi di valori nulli

E1 (K1, A1, B1,...)

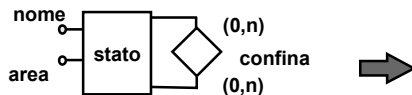
E2 (K2, A2, B2,...)

R (K1, K2, AR, BR,...)

Auto associazione N a M

viene tradotta con:

- una relazione per l'entità ed
- una per l'associazione,
 - quest'ultima contiene due volte la chiave dell'entità, è necessario però modificare i nomi degli attributi, per non avere omonimia



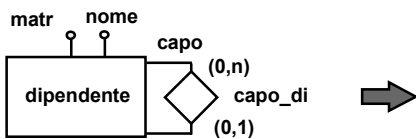
auto associazione N a M

```
CREATE TABLE STATO
(NOME ... NOT NULL, AREA ...
PRIMARY KEY (NOME));
```

```
CREATE TABLE CONFINA
STATO_A ... NOT NULL, STATO_B ... NOT NULL,
PRIMARY KEY (STATO_A, STATO_B)
FOREIGN KEY (STATO_A)
REFERENCES STATO
FOREIGN KEY (STATO_B)
REFERENCES STATO);
```

auto associazione 1 a N

- è traducibile con una sola relazione che contiene due volte l'attributo chiave: una volta come chiave ed una come riferimento all'istanza connessa, con nome diverso per specificare il ruolo

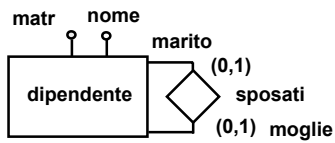


auto associazione 1 a N

```
CREATE TABLE DIPENDENTE
(MATR ... NOT NULL, NOME ..., CAPO ...
PRIMARY KEY (MATR)
FOREIGN KEY (CAPO)
REFERENCES DIPENDENTE);
```

- nel caso di associazione 1 ad 1 il concetto di ruolo assume maggiore importanza:

auto associazione 1 a 1



- su entrambi i rami è bene specificare il ruolo: conviene la soluzione con due relazioni per evitare ridondanze, vincoli ed eccesso di valori nulli.

auto associazione 1 a 1

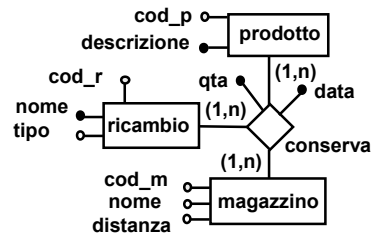
```
CREATE TABLE DIPENDENTE (MATR ... NOT NULL, NOME ..., PRIMARY KEY (MATR))
```

```
CREATE TABLE SPOSATI (MOGLIE ... NOT NULL, MARITO ... NOT NULL, PRIMARY KEY (MOGLIE), FOREIGN KEY (MOGLIE) REFERENCES DIPENDENTE, FOREIGN KEY (MARITO) REFERENCES DIPENDENTE);
```

Associazione n-aria

- segue la traduzione standard
- talvolta, nella relazione che traduce l'associazione, la chiave ottenuta componendo le chiavi di tutte le entità partecipanti è una superchiave, cioè una chiave composta il cui set di componenti non è minimale (la chiave vera è un sottoinsieme)
- Esempio: prodotti-ricambi-magazzini

associazione n-aria



associazione n-aria

```
CREATE TABLE PRODOTTO (COD_P... NOT NULL, DESCRIZIONE..., PRIMARY KEY (PRD));
CREATE TABLE RICAMBIO (COD_R ... NOT NULL, NOME..., TIPO..., PRIMARY KEY (COD_R));
CREATE TABLE MAGAZZINO (COD_M.... NOT NULL, NOME ..., DISTANZA..., PRIMARY KEY (COD_M));
```

associazione n-aria

l'associazione diventa:

```
CREATE TABLE CONSERVA (COD_P ... NOT NULL, COD_R... NOT NULL, COD_M... NOT NULL, DATA... NOT NULL, QTA ..., PRIMARY KEY (COD_P, COD_R, COD_M), FOREIGN KEY (COD_P) REFERENCES PRODOTTO, FOREIGN KEY (COD_M) REFERENCES MAGAZZINO, FOREIGN KEY (COD_R) REFERENCES RICAMBIO);
```

ipotizziamo che COD_M sia ridondante $\Rightarrow$

associazione n-aria

- un ricambio esiste in un solo magazzino, quindi COD_R è associato ad un solo COD_M, cioè determina COD_M, allora la presenza di COD_M nella chiave è ridondante:

```
CREATE TABLE CONSERVA (COD_P ... NOT NULL,  
COD_R... NOT NULL, COD_M... NOT NULL,  
DATA... , QTA ...  
PRIMARY KEY (COD_P, COD_R)  
FOREIGN KEY (COD_P) REFERENCES PRODOTTO  
FOREIGN KEY (COD_R) REFERENCES RICAMBIO);
```

- COD_M non e' piu' parte della chiave

commento

- nel caso precedente la dipendenza tra magazzino e ricambio non era stata espressa sulla associazione n-aria; abbiamo ipotizzato di scoprirla nella fase di progetto logico
- se il progetto concettuale è ben fatto casi del genere non sono frequenti
- il ricontrollo delle chiavi delle relazioni è quindi importante e se ne occupa la teoria della normalizzazione

LA NORMALIZZAZIONE DELLE RELAZIONI

Nelle lezioni precedenti

- Abbiamo visto la conversione degli schemi E/R in schemi logici relazionali
- questa attività, che va sotto il nome di progetto logico, prevede una serie di fasi che applicano regole di trasformazione e traduzione

In questa lezione

- continueremo a risolvere i problemi legati alla costruzione di schemi relazionali, vedremo, in particolare, come produrre schemi relazionali esenti da anomalie e non suscettibili di perdita di informazioni nelle operazioni di join
- riprenderemo il concetto di dipendenza funzionale
- introdurremo il concetto di forma normale

impiegato	stipendio	progetto	budget	funzione
Rossi	2	biella	300	tecnico
Verdi	3	valvola	500	progettista
Verdi	3	albero	1500	progettista
Neri	7	albero	1500	direttore
Neri	7	valvola	500	consulente
Neri	7	biella	300	consulente
Mori	6	biella	300	direttore
Mori	6	albero	1500	progettista
Bianchi	6	albero	1500	progettista
Bianchi	6	biella	300	progettista

ridondanze e anomalie

1) ridondanza :

- si ripete più volte la notizia che un impiegato percepisce un certo stipendio
- si ripete più volte che un progetto ha un certo budget
- i valori di progetto e di impiegato si ripetono e quindi non possono singolarmente essere presi come chiave
- la chiave è (progetto, impiegato) : non si hanno ripetizioni

ridondanze e anomalie

2) aggiornamento :

- poiché si ripete più volte la notizia che un impiegato percepisce un certo stipendio, se lo stipendio viene aggiornato questo deve essere fatto su tutte le tuple che riguardano un certo impiegato
- poiché si ripete più volte che un progetto ha un certo budget, se il budget viene aggiornato lo si deve fare su tutte le tuple che riguardano un certo progetto

ridondanze e anomalie

3) cancellazione :

- supponendo che un impiegato lasci l'azienda o non partecipi a progetti rischiamo di perdere i dati sui progetti se era l'ultimo impiegato del progetto
- analogamente per i dati degli impiegati se un progetto viene eliminato
- se la chiave è (progetto, impiegato) in entrambi i casi di eliminazione si potrebbero avere valori nulli nella chiave

ridondanze e anomalie

4) inserimento :

- se la chiave è (progetto, impiegato) non è possibile inserire i dati di un impiegato se non è stato assegnato ad almeno un progetto, analogamente per un nuovo progetto a cui non è stato ancora assegnato un impiegato
- accettare un inserimento di (progetto) o, (impiegato) vuol dire che si inseriscono valori nulli (incompatibili con la chiave)

ridondanze e anomalie

- casi così eclatanti non succedono se si è seguita la prassi corretta di progettazione: prima lo schema E/R e poi la traduzione in schema relazionale
- può però succedere che carenze di specifiche o errori di schematizzazione possano portare a relazioni con anomalie
- i casi sono invece più frequenti quando si esaminano vecchi DB scarsamente documentati o, addirittura, si cerca di intuire la natura dei dati da documenti che sintetizzano le informazioni su moduli cartacei

dipendenze funzionali

- La dipendenza funzionale è un vincolo di integrità per il modello relazionale
- dall'osservazione della relazione ricaviamo che:
 - ogni volta che in una tupla compare un certo impiegato lo stipendio è sempre lo stesso
 - possiamo dire che il valore dell'impiegato determina il valore dello stipendio, cioè:
 - esiste una funzione che associa ad ogni valore nel dominio impiegato uno ed un solo valore nel dominio stipendio
 - analogamente per un valore di progetto

ricordiamo che:

Schema

Dominio: 2,3,4,5.... 20

R
e
l
a
z
i
o
n
e

impiegato	stipendio	progetto	budget	funzione
Rossi	2	biella	300	tecnico
Verdi	3	valvola	500	progettista
Verdi	3	albero	1500	progettista
Neri	7	albero	1500	direttore
Neri	7	valvola	500	consulente
Neri	7	biella	300	consulente

dipendenze funzionali

- La dipendenza funzionale si può definire formalmente :
 - data una relazione R definita su uno schema $S(X)$ e due sottoinsiemi di attributi Y e Z non vuoti di X, esiste una dipendenza funzionale $Y \rightarrow Z$, se, per ogni coppia di tuple t1 e t2 aventi lo stesso valore di Y risulta che hanno lo stesso valore di Z
- dall'osservazione della relazione ricaviamo che:
 - impiegato $\rightarrow$ stipendio e progetto $\rightarrow$ budget

dipendenze funzionali

- Attenzione : se prendiamo la chiave K della relazione R si verifica facilmente che esiste una dipendenza funzionale tra K ed ogni attributo dello schema
- infatti per definizione di chiave esiste un solo valore di K in R e quindi la dipendenza di cui sopra è banalmente soddisfatta
- nell'esempio:
 - impiegato, progetto $\rightarrow$ stipendio, budget, funzione

dipendenze funzionali

Però:

- impiegato, progetto $\rightarrow$ funzione è una dipendenza completa,
- mentre
 - impiegato, progetto $\rightarrow$ stipendio e
 - impiegato, progetto $\rightarrow$ budget sono in realtà
 - impiegato $\rightarrow$ stipendio e progetto $\rightarrow$ budget queste sono dipendenze parziali che causano anomalie

dipendenze funzionali

- Le ridondanze e le anomalie sono causate da dipendenze $X \rightarrow Y$ che permettono ripetizioni all'interno della relazione (impiegato,stipendio e progetto, budget si ripetono nella relazione), in altre parole :
- Le ridondanze e le anomalie sono causate da dipendenze $X \rightarrow Y$ tali che X non contiene la chiave della relazione
- Una relazione R è in forma normale (Boyce e Codd) se, per ogni dipendenza $X \rightarrow Y$ in R, X contiene una chiave K di R (X è superchiave)

dipendenze funzionali

- Una relazione non in forma normale è possibile che venga decomposta in due o più relazioni in forma normale
- la decomposizione si può attuare effettuando proiezioni in modo tale da ottenere che ciascuna dipendenza funzionale corrisponda ad una relazione separata
- nell'esempio :
 - FUNZIONI per impiegato, progetto $\rightarrow$ funzione
 - IMPIEGATI per impiegato $\rightarrow$ stipendio
 - PROGETTI per progetto $\rightarrow$ budget

impiegato	stipendio
Rossi	2
Verdi	3
Neri	7
Mori	6
Bianchi	6

progetto	budget
biella	300
valvola	500
albero	1500

impiegato	progetto	funzione
Rossi	biella	tecnico
Verdi	valvola	progettista
Verdi	albero	progettista
Neri	albero	direttore
Neri	valvola	consulente
Neri	biella	consulente
Mori	biella	direttore
Mori	albero	progettista
Bianchi	albero	progettista
Bianchi	biella	direttore

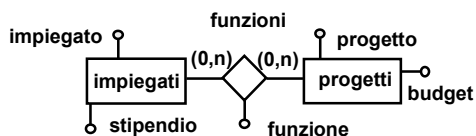
dipendenze funzionali

- FUNZIONI, IMPIEGATI e PROGETTI sono normalizzate perché soddisfano la definizione di forma normale
- la relazione non decomposta può essere ricostruita con il join:

```
SELECT *
FROM IMPIEGATI I, PROGETTI P, FUNZIONI F
WHERE I.IMPIEGATO = F.IMPIEGATO
AND F.PROGETTO = P.PROGETTO
```

dipendenze funzionali

- Quando la relazione originale è ricostruibile con il join la decomposizione è corretta e si dice essere senza perdita
- notare che lo schema corretto corrisponde alla traduzione di:

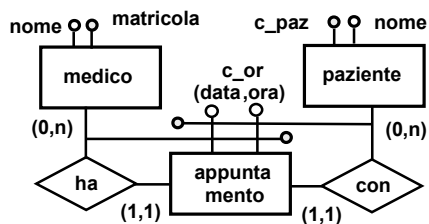


dipendenze funzionali

- Schemi E/R corretti producono in generale buoni schemi relazionali senza problemi di anomalie e ridondanze e corrispondono a decomposizioni senza perdita
- Schemi E/R dove non tutti i vincoli sono espressi nello schema e/o in presenza di associazioni n_arie possono però venire tradotti (non intenzionalmente) in schemi relazionali non ben normalizzati
- È quindi importante ricontrollare la normalizzazione: operazione questa non sempre facile o, possibile per carenza di specifiche

dipendenze funzionali

ad esempio, lo schema già visto in precedenza:



dipendenze funzionali

Per mezzo dell'identificazione esterna individua correttamente le due dipendenze:

matricola, data, ora → c_paz
c_paz, data, ora → matricola

e si traduce nello schema relazionale:

medico (matricola, nome)
paziente (c_paz, cognome)
appuntamento (matricola, data, ora, c_paz)

oppure

appuntamento (c_paz, data, ora, matricola)

dipendenze funzionali

- Non sempre le decomposizioni producono effetti desiderabili
- decomposizioni errate possono generare relazioni che, ricongiunte con il join, producono relazioni con dati incerti; si ha quindi una perdita di informazione

- consideriamo un esempio di relazione:

SEDI (impiegato, progetto, sede)
con le dipendenze:

impiegato, progetto → sede
impiegato → sede e progetto → sede

dipendenze funzionali

chiave di SEDI :
impiegato, progetto

impiegato	progetto	sede
Rossi	biella	milano
Verdi	valvola	torino
Verdi	albero	torino
Bianchi	cinghia	milano
Neri	valvola	torino

Vincolo:
gli impiegati hanno
come sede la sede
dei loro progetti

decomponendo secondo le due dipendenze:



dipendenze funzionali

impiegato	sede	progetto	sede
Rossi	milano	biella	milano
Verdi	torino	valvola	torino
Bianchi	milano	albero	torino
Neri	torino	cinghia	milano

il join sull'attributo comune:

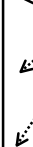
SELECT I.IMPIEGATO, P.PROGETTO, P.SEDE
FROM IMPIEGATI I, PROGETTI P
WHERE I.SEDE = P.SEDE



dipendenze funzionali

impiegato	progetto	sede
Rossi	biella	milano
Rossi	cinghia	milano
Verdi	valvola	torino
Verdi	albero	torino
Bianchi	biella	milano
Bianchi	cinghia	milano
Neri	valvola	torino
Neri	albero	torino

←..... crea tuple
che non
esistevano!



dipendenze funzionali

- Inoltre , anche se sono corrette, le due relazioni non rispettano il vincolo che la sede di un impiegato è la sede dei suoi progetti, un progetto potrebbe cambiare sede indipendentemente dagli impiegati

- Regola:
una buona decomposizione deve prevedere la ricostruzione della relazione di partenza con operazioni di join su chiavi

- Osservazione :
i join su attributi che si corrispondono n a m sono rischiosi

dipendenze funzionali

progetto	sede	impiegato	progetto
biella	milano	Rossi	biella
valvola	torino	Verdi	valvola
albero	torino	Verdi	albero
cinghia	milano	Bianchi	cinghia
		Neri	valvola

il join sull'attributo comune:

SELECT I.IMPIEGATO, P.PROGETTO, P.SEDE
FROM IMPIEGATI I, PROGETTI P
WHERE I.PROGETTO = P.PROGETTO



dipendenze funzionali

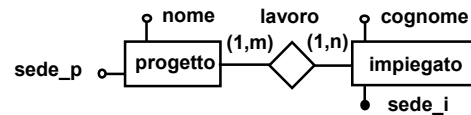
la decomposizione è corretta
però abbiamo perso
nello schema precedente la
dipendenza
impiegato → sede

impiegato	progetto	sede
Rossi	biella	milano
Verdi	valvola	torino
Verdi	albero	torino
Bianchi	cinghia	milano
Neri	valvola	torino

problema: che ne è di Verdi se il progetto Albero va a Roma?

dipendenze funzionali

Se fossimo partiti dallo schema E/R?



avremmo comunque dovuto dichiarare, a parte dallo schema E/R, il vincolo che la sede di un impiegato deve essere uguale alla sede dei progetti in cui lavora, e lo schema relazionale: ➡

progetti

progetto	sede
biella	milano
valvola	torino
albero	torino
cinghia	milano

impiegati

impiegato	progetto
Rossi	biella
Verdi	valvola
Verdi	albero
Bianchi	cinghia
Neri	valvola

lavoro

impiegato	sede
Rossi	milano
Verdi	torino
Bianchi	milano
Neri	torino

questa soluzione consente anche (0,n) nell'associazione

dipendenze funzionali

il join adesso sarà:

```

SELECT I.IMPIEGATO, P.PROGETTO, P.SEDE
FROM IMPIEGATI I, PROGETTI P, LAVORO L
WHERE I.IMPIEGATO = L.IMPIEGATO
AND L.PROGETTO = P.PROGETTO
    
```

che ottiene la relazione richiesta senza perdita perché lavora su chiavi

però non c'è garanzia sull'uguaglianza di sede

conclusioni generali

- Progettare i dati è difficile
- il lavoro di gruppo è importantissimo per evitare differenze di percezione e di visione dei problemi
- DFD, schemi E/R, dipendenze funzionali sono utilissimi per descrivere e capire i problemi
 - tanta più conoscenza si riesce a descrivere negli schemi, tanta meno verrà espressa con vincoli meno leggibili, o dispersa in programmi di difficile lettura e aggiornamento

conclusioni generali

- è bene però anche non eccedere con schemi particolarmente complicati contenenti un eccesso di concetti fittizi e di collegamento che rendono difficile la lettura e la soluzione innaturale
- la documentazione di progetto è pertanto fondamentale